GARBAGE MANAGEMENT SYSTEM

SHAHRA M.A

E2447027

Dissertation submitted to the Faculty of Information Technology,

University of Moratuwa, Sri Lanka in partial fulfillment of the requirements of the Degree of Bachelor of Information Technology(External) in Information Technology.

June 2026

Declaration and copyright statement

"I declare that this is my own work and this report does not incorporate without acknowledgement any material previously submitted for a degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to University of Moratuwa the non exclusive right to reproduce and distribute my report, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works."

Signature: SHAHRA MUSAM ALI Date:24/06/2026

Declaration of the supervisor

The above candidate has carried out work for the Bachelors of Information Technology Degree report under my supervision.

Name of the supervisor: Mr.Thathsara Liyanage

Signature of the supervisor: Date:

Acknowledgment

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project. First and foremost, I extend my heartfelt thanks to the University of Moratuwa, Center for Open and Distance Learning, for providing the opportunity to undertake this project. My heartfelt gratitude goes to MS. Dulmi Fonseka, our module coordinator for her continuous support. I would also like to extend my gratitude to MS. Vishaka Nanayakara for her invaluable input during the project preparation phase.

I am especially grateful to my supervisor, Mr. Thathsara Liyanage, for his invaluable guidance, constructive feedback, encouragement, and continuous support throughout the project period. His guidance plays an important role in shaping this project.

I would also like to thank my family for their constant encouragement, patience, and moral support, which motivated me to complete this project successfully. Additionally, I appreciate my colleagues and friends who directly or indirectly supported me by sharing ideas, feedback, and assistance during the development of this project.

Finally, I would like to extend my sincere thanks to everyone who contributed in any way toward the completion of this work.

Abstract

Most developing countries face significant challenges in garbage management, resulting in environmental pollution, disease spread, climate change impacts, and resource wastage. In Sri Lanka, the responsibility for garbage management lies with municipal councils. However, many councils lack adequate resources and continue to rely on manual and outdated methods. These methods including paper based records, improper scheduling, manual complaint handling, leading to poor coordination, lack of transparency and public dissatisfaction.

This project present the design and development of the CleanLand Garbage Management System a centralized, web based platform that digitize the garbage collection process in Sri Lanka The system supports seamless communication among residents, administrators, and truck drivers. Key features include garbage truck tracking, QR code based bin identification and status updates, digital complaint submission and tracking, email notifications, collection schedule management system, public awareness content, and a feedback and rating system. The system is developed using React.js for the frontend, Node.js for the backend, and MySQL as the relational database management system. The development process follows the agile methodology, progressing through iterative cycles of requirement analysis, system design, implementation, and continuous feedback to ensure that both functional and non functional requirements are met.

A stakeholder survey conducted among 36 participants revealed widespread dissatisfaction with existing systems, with only 2% of respondents reporting satisfaction and 86% experiencing missed garbage pickups. These findings strongly justify the need for the proposed system. The expected outcomes include improved operational efficiency, enhanced transparency, better inter stakeholder communication, and increased public satisfaction. Overall, CleanLand offers a practical, scalable solution for modernizing garbage management in Sri Lanka, contributing to a cleaner environment and promoting sustainable garbage management practices.

Table of Contents

Table of Contents

Acknowledgment iii

Abstract iv

Table of Contents v

List of Figures viii

List of Tables xi

List of abbreviations xi

CHAPTER 01 1

1. Introduction 1

1.1 Background 2

1.2 Problem Statement 2

1.3 Motivations and Significance of the Project 3

1.4 Aim and Objectives 4

1.4.1 Aim 4

1.4.2 Objectives 4

1.5 Summary 5

CHAPTER 02 6

2. Literature Review 6

2.2 Introduction 6

2.2 MYWASTE Garbage Collection System 6

2.3 Cleantech Garbage Management System 7

2.4 Colombo Municipal Council Garbage Collection System 7

2.5 Kurunegala Municipal Council Garbage Collection System 7

2.6 Galigamuva Municipal Council Garbage Collection System 8

2.7 Galle Municipal Council Garbage Management System 8

2.8 Background Analysis and Survey Among stakeholder 8

2.9 Comparison With Existing Systems 13

2.10 Summary 14

CHAPTER 3 15
3.The System Specification and Design Methodology 15
3.1 Introduction 15
3.2 Proposed Methodology 15
3.3 Requirement Analysis 17
3.3.1 Functional Requirements 17
3.3.2 Non Function Requirement 18
3.4 Components of the System 19
3.4.1 Software Components and Tasks 19
3.4.2 Database Component 34
3.4.3 Hardware Component 37
3.5 Technologies Adapted 38
3.6 System Design 39
3.6.1 Use Case Diagram 39
3.6.2 Activity Diagram 40
3.7 Summary 46
CHAPTER 04 47
4. System Implementation 47
4.1 Introduction 47
4.2 Code Features 47
4.3 Database Connection 48
4.4 Backend Server Implementation 48
4.5 Frontend Implementation 49
4.6 Login 51
4.7 SignUp 53
4.8 Complaint Page 57
4.9 Admin Dashboard 64
4.10 Bin Management 67
4.11 Special Pickup 72

4.12 Driver management page 74
4.13 Rout Page 78
4.14 Schedule page 79
4.15 Data Validation 82
4.16 Reusable Components of the System 82
4.17 Difficulties of the Software 83
4.18 Summary 83
CHAPTER 05 85
5. RESULTS AND EVALUATION 85
5.1 Introduction 85
5.2 Test and Deployment Plan 86
5.3 Testing Procedure 87
5.4 Test Cases 88
5.4.1 Test Cases for Registration 88
5.4.2 Test Case for login 89
5.4.3 Special pickup request test case 90
5.4.4 Complaint submission test case 92
5.4.5 Feedback submission test case 94
5.4.6. Add Schedule test case 95
5.4.7 Test case for route registration 97
5.5 Summary 99
CHAPTER 06 100
6. Future Work 100
6.1 Introduction 100
6.3 Summary 101
CHAPTER 07 102
7. Conclusion 102
CHAPTER 8 103
8. Reflection 103

9. Reference 103

List of Figures

Figure 1:Geographical Distribution of Survey Respondents 9
Figure 2: Satisfaction Level with Current Garbage Collection Services 9
Figure 3: Awareness of Garbage Collection Schedules 10
Figure 4: Methods Used to Receive Collection Schedule Information 10
Figure 5:Frequency of Missed Garbage Collections 11
Figure 6: Methods Used for Complaint Submission 11
Figure 7:Access to Garbage Truck Tracking Facilities 12
Figure 8:Availability of Garbage Bin Updates 12
Figure 9: Methods of Receiving Garbage Bin Updates 13
Figure 10: User login page 20
Figure 11: Resident registration page 21
Figure 12: Admin dashboard 1 21
Figure 13: Admin dashboard 2 22
Figure 14: Add new driver page 22
Figure 15:Driver management page 23
Figure 16: Update drive page 23
Figure 17:Driver delete page 24
Figure 18: Add QR code page 24
Figure 19: Special pickup request page 25
Figure 20: Schedule management page 25
Figure 21:Resident management page 26
Figure 22: Add truck page 26
Figure 23: Truck management page 27
Figure 24: Add route page 27
Figure 25: Route management page 28
Figure 26: Add bin page 28
Figure 27: Bin management page 29
Figure 28: Bin update page 29
Figure 29: Add content page 30
Figure 30: Home page 1 30
Figure 31: Home page 2 31
Figure 32: Home page 3 31
Figure 33: Home page 4 32

Figure 34:Schedule page 32
Figure 35: Complaint management page 33
Figure 36: Feedback page 33
Figure 37: Garbage pickup request page 34
Figure 38: ER diagram 36
Figure 39: Use case diagram 39
Figure 40:User login activity diagram 40
Figure 41: User registration activity diagram 41
Figure 42: Garbage collection schedule activity diagram 42
Figure 43: Special pickup request activity diagram 43
Figure 44: Awarness content activity diagram 44
Figure 45: Feedback activity diagram 45
Figure 46: Complaint activity diagram 46
Figure 47: Database connection 48
Figure 48: Backend server implementation 49
Figure 49: Frontend implementation 49
Figure 50: Frontend implementation 50
Figure 51: Frontend implementation 50
Figure 52: Login page 1 51
Figure 53: Login page 2 51
Figure 54: Login page 3 52
Figure 55: Login page 52
Figure 56: Login page 53
Figure 57: Login page backend 53
Figure 58: Signup page 1 54
Figure 59: Signup page 2 54
Figure 60: Signup page 3 55
Figure 61: Signup page 4 55
Figure 62: Signup page 5 56
Figure 63: Signup page backend 56
Figure 64: complaint page 1 57
Figure 65: Complaint page 2 57
Figure 66: Complaint page 3 58
Figure 67: Complaint page 4 58
Figure 68: Complaint page 5 59
Figure 69: Complaint page 6 59
Figure 70: Complaint page 7 60
Figure 71: Complaint page 8 60
Figure 72:Complaint management page 1 61
Figure 73: Complaint management page 2 61

Figure 74: Complaint management page 3 62
Figure 75: Complaint backend 62
Figure 76: Complaint backend 63
Figure 77: Complaint backend 63
Figure 78: Admin dashboard 1 64
Figure 79: Admin dashboard 2 64
Figure 80: Admin dashboard 3 65
Figure 81: Admin dashboard 4 65
Figure 82: Admin dashboard 5 66
Figure 83: Admin dashboard 6 66
Figure 84: Bin management 1 67
Figure 85: Bin management 2 67
Figure 86: Bin management 3 68
Figure 87: Bin management 4 68
Figure 88: Bin management 5 69
Figure 89: Add bin 1 69
Figure 90: Add bin 2 70
Figure 91: Add bin 3 70
Figure 92: Bin backend 70
Figure 93: Bin backend 71
Figure 94: Bin backend 71
Figure 95: Special pickup page 1 72
Figure 96: : Special pickup page 2 72
Figure 97: Special pickup page 3 73
Figure 98: Special pickup page 4 73
Figure 99: Add driver 1 74
Figure 100: Add driver 2 74
Figure 101: Driver management 1 75
Figure 102: Driver management 2 75
Figure 103: Driver management 3 76
Figure 104: Driver management 4 76
Figure 105:: Driver backend 77
Figure 106: : Driver backend 77
Figure 107 : Driver backend 78
Figure 108: Route backend 78
Figure 109: Route backend 79
Figure 110: Schedule backend 1 79
Figure 111: Schedule backend 2 80
Figure 112: Schedule backend 3 80
Figure 113: Schedule backend 4 81

Figure 114: Schedule backend 5 81
Figure 115: Schedule backend 6 82

List of Tables

Table 1: Comparison with existing systems..... 14
Table 2: Relational schema 35
Table 3: Test case for registration 1 88
Table 4: Test case for registration 2 89
Table 5: Test case for login 1 90
Table 6: Test case for login 2 90
Table 7: Test case for special pickup request 1 91
Table 8: Test case for special pickup request 2 92
Table 9: Test case for complaint submission 1 93
Table 10: Test case for complaint submission 2 94
Table 11: Test case for feedback submission 1 94
Table 12: Test case for feedback submission 2 95
Table 13: Test case for add schedule 1 96
Table 14: Test case for add schedule 2 97
Table 15: Test case for route registration 1 98
Table 16: Test case for route registration 2 98

List of abbreviations

GPS- Global Positioning System

QR- Quick Response

MC- Municipal Council

CMC- Colombo Municipal Council

CHAPTER 01

1. Introduction

Garbage management has become one of the most significant environmental and public health concerns in many developing countries. Rapid urbanization, population growth, and increased consumption have led to a substantial rise in the amount of garbage generated by households, commercial establishments, and industries. Improper garbage management practices contribute to environmental pollution, the spread of diseases, unpleasant living conditions, and the wastage of valuable resources. As a result, local authorities are under increasing pressure to improve garbage collection and disposal services while ensuring environmental sustainability.

In Sri Lanka, municipal councils are responsible for managing garbage collection and disposal activities within their respective jurisdictions. However, many councils continue to rely on traditional methods such as paper based record keeping, manual scheduling, and conventional complaint handling procedures. These outdated approaches often result in missed garbage collections, delayed responses to public complaints, poor coordination among stakeholders, and limited transparency in service delivery. Consequently, residents frequently experience dissatisfaction with existing garbage management services.

The advancement of information technology has created opportunities to modernize public services through digital transformation. Web based management systems have proven effective in improving operational efficiency, enhancing communication, and automating administrative processes. By applying these technologies to municipal garbage management, it is possible to create a more efficient, transparent, and user friendly system that benefits both local authorities and residents.

To address these challenges, the CleanLand Garbage Management system was developed as a centralized web based platform that digitizes and streamlines garbage collection management processes. The system facilitates communication among residents, municipal administrators, and garbage collection drivers while providing features such as collection schedule management, QR code based bin identification, complaint submission and tracking, email notifications, awareness content management, feedback collection, and truck monitoring capabilities. Through the integration of modern web technologies, the proposed system aims to improve service quality, operational efficiency, and public satisfaction while supporting sustainable waste management practices.

1.1 Background

Garbage management is a critical component of urban development and environmental protection. Effective garbage collection system help maintain clean communities, reduce health risks, and minimize environmental damage caused by improper garbage disposal. However, many municipalities in developing countries face challenges related to limited resources, inadequate infrastructure, and inefficient management practices.

In Sri Lanka, municipal councils are tasked with organizing garbage collection schedules, handling resident complaints, and monitoring garbage disposal activities. Despite these responsibilities, many councils continue to depend on manual systems that make information management difficult and time consuming. Residents often lack access to accurate collection schedules, while administrators face difficulties in monitoring operations and responding efficiently to service related issues.

Municipal councils and residents face numerous challenges in managing waste collection activities effectively. Common issues include missed garbage collection, inefficient complaint handling procedures, limited access to collection schedules, and inadequate communication between stakeholders. These challenges highlight the need for modern digital solution capable of improving operational efficiency and service transparency

The increasing availability of internet access, smartphones, and web technologies provides an opportunity to modernize municipal waste management systems. By utilizing digital platforms, local authorities can improve communication, automate administrative processes, and provide residents with convenient access to waste management services. The CleanLand Garbage Management System was developed to leverage these technologies and provide a comprehensive solution for modern garbage management.

1.2 Problem Statement

Effective garbage management is essential for maintaining environmental sustainability, public health, and the overall cleanliness of communities. However, many municipal councils in Sri Lanka continue to rely on traditional and manual methods for managing garbage collection operations. These methods include paper based record keeping, manual scheduling of collection route, and conventional complaint handling procedures. As urban populations continue to grow and garbage generation increases, these approaches have become increasingly inefficient and difficult to manage.

Resident frequently experience issues such as missed garbage collections, lack of awareness regarding collection schedules, delayed responses to complaints, and limited communication with

municipal authorities. In many cases, citizens have no convenient method of reporting service related issues or tracking the progress of submitted complaints. This often results in public dissatisfaction and reduced confidence in garbage management services.

Municipal administrators also face several operational challenges. The absence of a centralized digital system makes it difficult to manage coaction schedules, monitor service activities, maintain accurate records, and coordinative effectively with collection personnel. Furthermore, the lack of real time monitoring and tracking capabilities limits transparency and makes it difficult to evaluate the performance of garbage collection operations.

Improper management of garbage collection service can contribute to environmental pollution, unpleasant living conditions, health risks, and inefficient use of municipal resources. Although garbage generation continues to increase, many local authorities still lack modern technological solutions to support efficient service delivery.

Therefore, there is a need for a centralized, technology driven garbage management system that can improve communication among stakeholders, streamline garbage collection processes, enhance transparency, and provide a more efficient and user friendly approach to municipal garbage management. The CleanLand Garbage Management System was developed to address these challenges by digitizing key garbage management operations and providing an integrated platform for residents, administrators, and drivers.

1.3 Motivations and Significance of the Project

The growing challenges associated with municipal garbage management and the increasing dissatisfaction among residents regarding existing garbage collection services became the primary motivation for this project. As urban population continue to grow, the amount of garbage generated by households, business, and public institutions also increases significantly. However, many municipal councils in Sri Lanka continue to rely on traditional methods such as paper based record keeping, manual scheduling, and conventional complaint handling processes. These outdated approaches often result in missed garbage collection, poor communication between stakeholders, delayed complaint resolution, and a lack of transparency in service delivery.

Efficient garbage management plays a vital role in protecting public health, maintaining environmental cleanliness, and promoting sustainable urban development. When garbage collection services re not managed effectively, garbage accumulation can contribute to environmental pollution, unpleasant living conditions, the spread of diseases, and negative impacts on overall quality of life within communities. Therefore, there is a growing need for a modern and technology driven solution capable of improving the efficiency and reliability of municipal waste management operations.

This project aims to address these challenges through the development of the CleanLand Garbage Management System, a centralized web based platform that digitizes key garbage management activities. By integrating features such as collection schedule management, complaint submission and tracking, QR code based bin identification, email notifications, awareness content management, feedback collection, and truck tracking capabilities, the system provides a more efficient and transparent approach to garbage management.

The significance of this project lies in its ability to improve communication and coordination among residents, municipal administrators, and garbage collection drivers. Resident benefit from easier access to information and services, while administrators gain better tools for monitoring operations and managing resources effectively. Furthermore, the system promotes environmental awareness and encourages responsible garbage disposal practice within communities.

Overall, the CleanLand Garbage Management System contributes to the modernization of municipal garbage management services by utilizing contemporary web technologies to improve operational efficiency, enhances service quality, increase public satisfaction, and support environmental sustainability. The project establishes a strong foundation for future enhancements such as real time vehicle tracking, QR bin integration.

1.4 Aim and Objectives

1.4.1 Aim

The aim of this project is to design and develop a centralized, web based garbage management system that will make service of municipal garbage collection more efficient, transparent, and reliable. The system seek to bridge the gap between residents, garbage drivers, and municipal council administrators by providing real time truck information, effective communication, and improved service monitoring for cleaner environments and sustainable waste management practices.

1.4.2 Objectives

- Enhance communication and coordination among residents, garbage collectors, and municipal council administrator on a single digital platform.
- Increase transparency in garbage collection operations by enabling real time tracking of garbage trucks and insight into collection schedules.
- Improve service efficiency by reducing missed pickups, delayed collections, and unmanaged garbage bins through effective monitoring and reporting
- Provide residents with convenient access to garbage collection information, complaint submission, feedback mechanism, and email notifications.

- Support municipal councils with the monitoring operations, managing complaints, tracking, and making decisions. through the use of accurate and timely data.
- Promote responsible and eco friendly garbage disposal practice by increasing public awareness and providing proper guidance on garbage management.
- Establish a scalable and adaptable system that can support future enhancement and contribute to smart city and sustainable development initiatives.

1.5 Summary

This chapter introduce the CleanLand Garbage Management System and discuss the challenges associated with existing garbage management practices in Sri Lanka. The increasing volume of garbage generation, reliance on manual management processes, poor communication between stakeholders, inefficient complaint handling, and lack of transparency in garbage collection services are identified as key issues affecting garbage management. The chapter highlights the need for a technology driven solution to improve operational efficiency and service quality.

The proposed web based system aims to digitize management operations through features such as collection schedule management, complaint handling, QR code based bin identification, feedback collection, awareness content management, and truck tracking. By improving communication between residents, administrators, and drivers, the system expected to enhance transparency, increase public satisfaction, and support sustainable garbage management practices.

CHAPTER 02

2. Literature Review

2.2 Introduction

The rapid growth of urban populations and the increasing volume of garbage generated by households, commercial establishments, and industries have created significant challenges for municipal garbage management authorities. Traditional garbage management methods often rely on manual record keeping, static collection schedules, and inefficient communication mechanism, resulting in operational inefficiencies and reduced public satisfaction.

Recent advances in information technology have enabled the development of digital garbage management solutions thar improve operational efficiency, transparency, and stakeholder communication. Modern garbage management systems incorporate technologies such as web applications, GPS tracking, QR code identification and notification services to streamline garbage collection activities and enhance service delivery.

This chapter presents a comprehensive review of existing garbage management systems implemented both locally and internationally. The strengths, weaknesses, functionalities, and technological approaches of each system are analyzed to identify current limitations and opportunities for improvement. Furthermore, a stakeholder survey was conducted to understand user expectations and challenges associated with existing garbage management services. The findings of this chapter provides the foundation for the development of the proposed CleanLand Garbage Management System.

2.2 MYWASTE Garbage Collection System

The MYWASTE [1] platform is a cloud based digital garbage management solution designed to support garbage traceability, operational control, and reporting throughout the garbage lifecycle. The system provides garbage tracking capabilities, mobile collection registration, route monitoring, automated document generation, and GPS supported collection verification.

The system demonstrates strong capabilities in garbage tracking and operational monitoring. Features such as GPS based collection registration, digital signatures, photographic evidence, and automated documentation improve accountability and regulatory compliance. However, the platform primarily targets business and garbage transport operators rather than individual residents.

Although MYWASTE provides excellent operational visibility, it lacks citizen focused features such as complaint tracking, personalized notifications, awareness content, and resident feedback

mechanisms. Therefore, while the system performs well in logistics and compliance management, its support for community engagement remains limited.

2.3 Cleantech Garbage Management System

The Cleantech [2] Garbage Management System focuses on providing garbage collection, recyclable material handling, sanitation services, environmental awareness campaigns, and garbage management planning services. The organization contributes significantly to urban cleanliness and environmental sustainability through its operational services.

The system offers comprehensive garbage management services, however it lacks in dedicated digital platform that allows residents to interact with garbage management activities. Features such as online complaint submission, collection tracking, automated notifications, and services monitoring are not available.

As a result, user participation remains largely passive. Although operational services are extensive, and absence of digital engagement limits transparency and service accessibility for residents.

2.4 Colombo Municipal Council Garbage Collection System

The Colombo Municipal Council[3] provides an online platform that publishes area wise garbage collection schedules, contractor details, and contact information for garbage collection supervisors. The system improves public access to official garbage collection information through a web based interface.

Despite providing useful information, the platform functions primarily as a static information portal. Users must manually search schedules and contact authorities directly to submit complaints or inquiries. There are no facilities for complaint tracking, real time updates, notifications, or service monitoring.

Consequently, the platform offers limited interactivity and does not fully support modern user expectations for digital public services.

2.5 Kurunegala Municipal Council Garbage Collection System

The Kurunegala Municipal Council[4] provides collection schedules, municipal service information, and complaint submission facilities through its website. The platform allows residents to access collection schedules based on administrative divisions and complaints using an online form.

Although the system provides greater functionality than many traditional council websites, several limitations remain. Complaint submissions occasionally encounter technical issues, and user are

not provided with complaint status tracking or automated notifications. Furthermore, collection schedules remain static and do not support real time updates.

These limitations reduce user satisfaction and highlight the need for a more integrated and responsive garbage management platform.

2.6 Galigamuva Municipal Council Garbage Collection System

The Galigamuva Municipal Council[5] website provides information regarding garbage collection routes, resource allocation, composing activities, recycling initiatives, and garbage management awareness programs. Educational content regarding the 3R concept (Reduce, Reuse, Recycle) is also available.

The platform contributes positively to public awareness and transparency regarding municipal garbage management operations. However, the information is largely static and lacks interactive functionality. Residents cannot track complaints, receive notifictaions, or monitor collection activities in real time.

As a result, user engagement remains limited despite the availability of useful informational content.

2.7 Galle Municipal Council Garbage Management System

The Galle Municipal Council [6] website outlines the responsibilities and services associated with municipal garbage management, including garbage removal, recycling initiatives, complaint reporting, and town cleaning activities.

While the website provides useful information regarding municipal operations, it lacks interactive services such as online complaint management, collection tracking, schedule notifications, and resident dashboards. Communication between residents and municipal authorities remains largely dependent on telephone based interactions.

The absence of modern digital services limits the effectiveness of the platform in addressing contemporary waste management challenges.

2.8 Background Analysis and Survey Among stakeholder

To evaluate existing garbage collection services and identify user requirements for the proposed CleanLand Garbage Management System, a stakeholder survey was conducted among 36 participants from urban, suburban and rural areas. The survey was distributed through Google

Forms and gathered feedback on current waste management practices and expectations for a digital solution.

The survey demographics showed that 52.8% of respondents were from suburban areas, 27.8% from urban areas, and 19.4% from rural areas, providing a balanced representation of different communities.

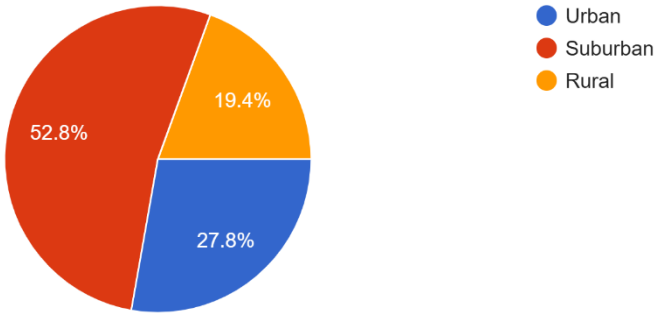


Figure 1:Geographical Distribution of Survey Respondents

The survey revealed considerable dissatisfaction with current garbage collection services. Most respondents reported low satisfaction levels due to unreliable collection services, poor communication, and limited transparency.

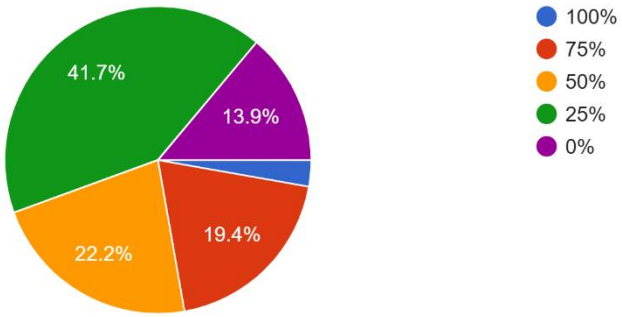


Figure 2: Satisfaction Level with Current Garbage Collection Services

Communication was identified as a major issue. Only 29.4% of respondents were aware of garbage collection schedules, while 70.6% reported receiving no schedule related information. Most

participants indicated that they do not receive updates regarding collection schedules through any communication channel.

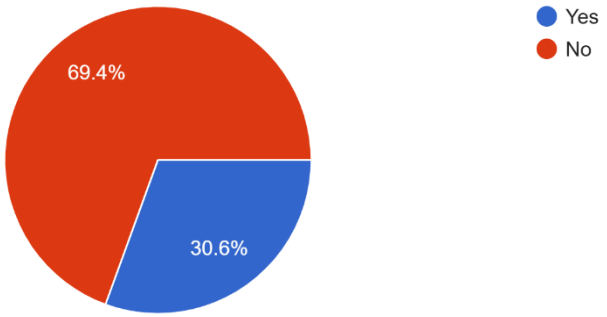


Figure 3: Awareness of Garbage Collection Schedules

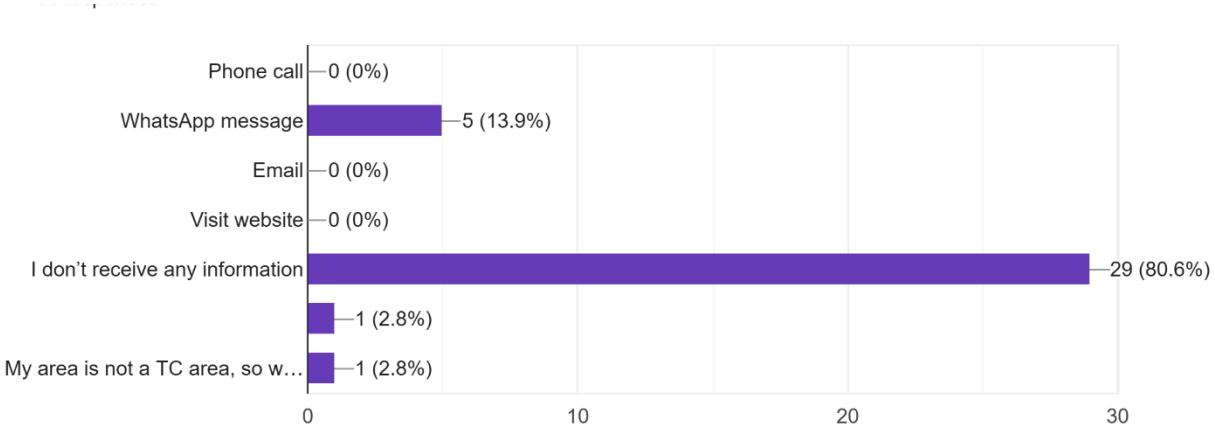


Figure 4: Methods Used to Receive Collection Schedule Information

Service reliability was another significant concern. Approximately 86% of respondents experienced missed garbage collections within the previous three months, highlighting weaknesses in existing waste management operations.

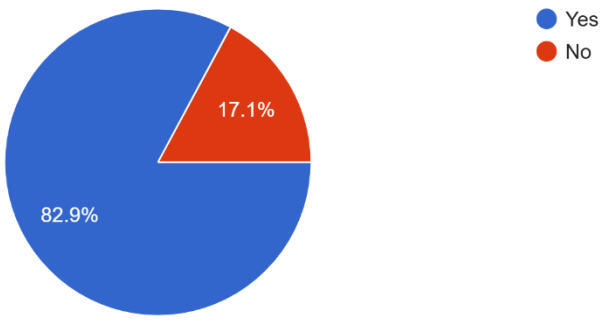


Figure 5:Frequency of Missed Garbage Collections

The survey also found that complaint management processes are ineffective. About 62.7% of respondents do not report service issues due to inconvenient reporting methods. Those who submit complaints generally rely on phone calls or in-person visits, indicating the lack of accessible digital complaint facilities.

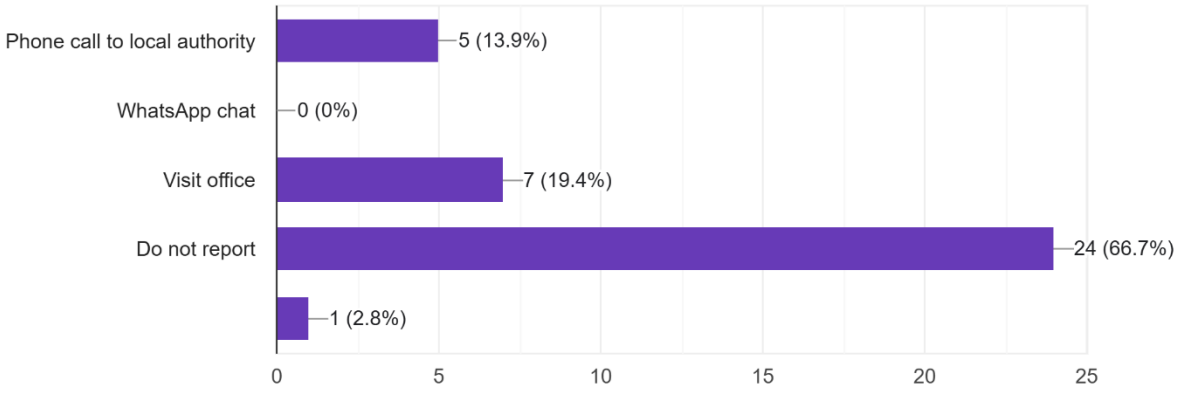


Figure 6: Methods Used for Complaint Submission

Most respondents reported having no access to truck tracking or garbage bin monitoring features. Approximately 92.2% lacked access to collection vehicle tracking, while 94% did not receive any updates regarding garbage bin status. Additionally, no respondents reported the use of QR code-based bin management systems.

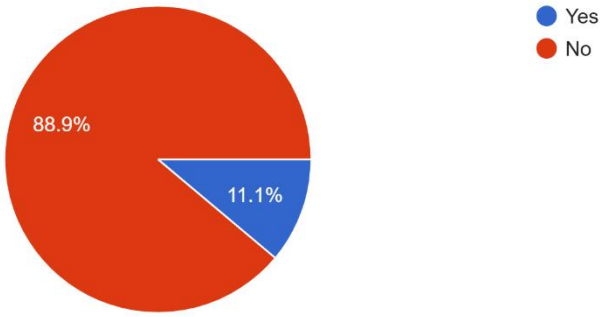


Figure 7:Access to Garbage Truck Tracking Facilities

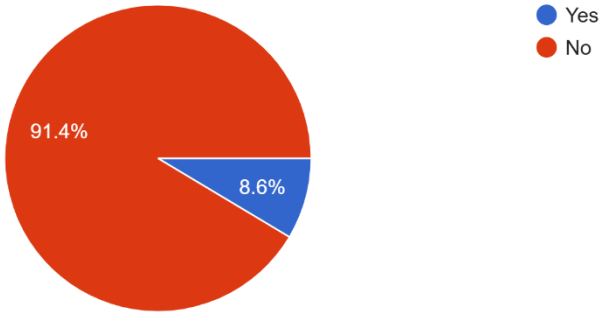


Figure 8:Availability of Garbage Bin Updates

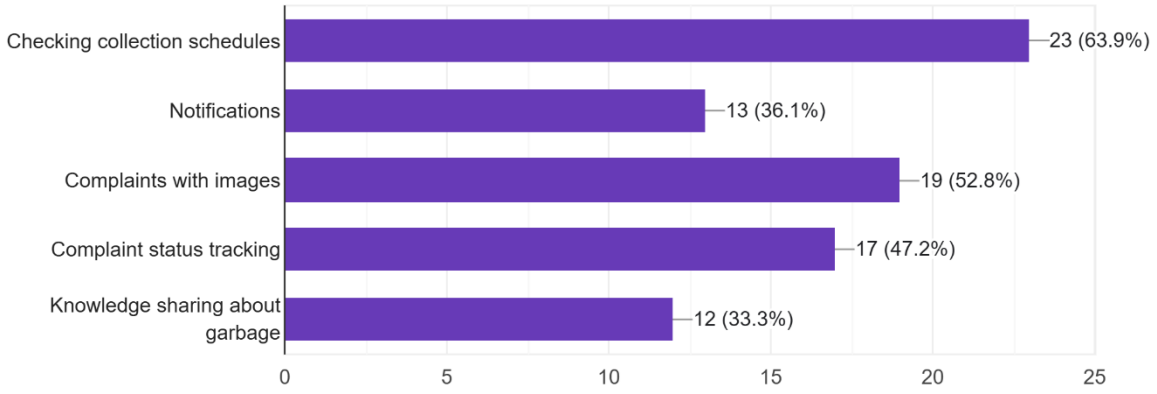


Figure 9: Methods of Receiving Garbage Bin Updates

Participants identified several important features for a modern waste management platform, including online complaint submission, complaint status tracking, collection schedule viewing, and automated notifications. These findings demonstrate a strong demand for a transparent and user-friendly digital solution.

Overall, the survey highlighted major shortcomings in existing garbage management services, particularly in communication, reliability, complaint handling, and operational transparency. These findings strongly support the development of the CleanLand Garbage Management System, which aims to improve waste management through digital complaints, schedule notifications, QR code-based bin management, and truck monitoring capabilities.

2.9 Comparison With Existing Systems

	Colombo Municipal Council	**Kurunegala Municipal Council**	**Galigamuwa Municipal Council**	**myWASTE**	**Cleantech**	**Galle Municipal Council**	**CleanLand**
Collection schedule	Yes	Yes (incomplete)	Yes (incomplete)	-	No	No	Yes
Real time truck tracking	No	No	No	Yes	No	No	Yes

Complaint management	No	Yes	Yes	Yes	No	Yes	Yes
Notification	No	No	No	No	No	No	Yes
Special request	Yes	Yes	No	Yes	Yes	No	Yes
QR based bin identification	No	No	No	NO	No	No	Yes
Public feedback and rating	No	No	No	No	N0	No	Yes

Table 1: Comparison with existing systems

2.10 Summary

This chapter reviewed and critically evaluated several existing garbage management systems implemented by municipal councils and private organizations. The analysis highlighted both the strengths and limitations of current solutions and identified gaps in functionality, user engagement, and service transparency. A stakeholder survey further confirmed the need for a modern digital garbage management platform. The findings presented in this chapter provide the foundation for the system requirements and design decisions discussed in the subsequent chapters.

CHAPTER 3

3.The System Specification and Design Methodology

3.1 Introduction

Agile is a software development methodology that emphasizes iterative development, continuous feedback, and flexibility throughout the project lifecycle. Unlike traditional development approaches, Agile allows requirements, designs, and solutions to evolve through collaboration between stakeholders and developers. The methodology focuses on delivering functional components of the system in incremental stages while continuously improving the product based on feedback and changing requirements.

In developing the CleanLand Garbage Management System, the Agile methodology was adopted to ensure that the system meets the needs of residents, administrators, and garbage collection drivers effectively. The project was divided into several iterations with each iteration focusing on specific system functionalities such as user management, complaint management, collection schedule management, QR code based bin identification, feedback management, and truck tracking.

At the beginning of each iteration requirement were gathered and analyzed based on stakeholder needs identified through surveys and discussions. The selected features were then designed, implemented, and tested before being reviewed and refined. Continuous testing and stakeholder feedback helped identify issues early and allowed improvement to be incorporated throughout the development process.

The Agile approach provided flexibility during system development, enabling new requirements and enhancements to be integrated without affecting the overall progress of the project. This iterative process ensured that each component of the CleanLand System was developed, evaluated, and improved incrementally, resulting in a reliable, user friendly, and efficient garbage management solution.

In the project planning and system design section, the proposed solution is developed using the Agile Methodology. Through iterative development cycles, the CleanLand Garbage Management System is designed and implemented in a structured manner, ensuring that both functional and non functional requirements are successfully achieved while maintaining high system quality and user satisfaction.

3.2 Proposed Methodology

The CleanLand Garbage Management System was developed using the Agile methodology, which emphasizes iterative development, continuous feedback, and incremental delivery of system

functionalities. Agile allows to develop, test, and improve features through multiple development cycles while adapting to changing requirements and stakeholder feedback.

The project is divided into several iterations, each focusing on a specific set of functionalities. At the end of every iteration, the developed features were tested and reviewed before proceeding to the next stage

Iteration 1: Requirement Gathering and Analysis

The initial iteration focused on collection and analyzing system requirements through stakeholder surveys, discussions, and research. Functional and non functional requirements were identified, including user management, complaint management, collection scheduling, QR code base bin identification, feedback management, notification service, and truck monitoring functionalities.

Iteration 2: System Design

Based on the gathered requirements, the system architecture, database structure, user interfaces were designed. The interaction between residents, administrator, and drivers were defined, and the database was deigned using MySQL. User interface prototype also prepared to support system development.

Iteration 3: System Development

The functionalities of the system were implemented using React.js for the frontend and Node.js for the backend. Features developed during this iteration included user registration and authentication, resident management, driver management, complaint submission, collection schedule management, and feedback management.

Iteration 4: System Development

This iteration focused on implementing QR code based bin identification and management features. Additional functionalities such as awareness content management, notifications, and complaint management also developed and integrated into the system.

Iteration 5: Development and Integration

The truck tracking and remaining system components were integrated with the existing functionalities. Data flow between different modules was tested to ensure smooth communication and consistent system performance.

Iteration 6: Testing

Testing was conducted throughout development and during the final iteration. Unit testing, integration testing were performed to identify and resolve system defects.

Iteration 7: Deployment and Maintenance

After successful testing, the system was prepared for deployment. Future maintenance activities include performance improvements, feature enhancements, and ongoing technical support.

The Agile methodology enabled continuous improvement throughout the development process and ensured that the CleanLand Garbage Management System effectively addressed stakeholder requirements while maintaining flexibility for future enhancements.

3.3 Requirement Analysis

Requirement Analysis is an important part of system development that helps identify and define the needs of stakeholders and system users. It involves gathering, analyzing, documenting, validating, and managing both functional and non functional requirements to ensure that the proposed solution effectively addresses existing problems. For the CleanLand Garbage Management System, requirements were identified through stakeholder surveys, observations and literature review.

3.3.1 Functional Requirements

- User Registration and Login - Enable residents to register and admin & drivers to securely access the system using their credentials.
- User Management - Allow admin to manage resident and driver accounts, including adding, updating, viewing, and removing user information.
- Complaint Management – Enable residents to submit complaints regarding garbage collection issues, upload supporting images, and track complaint status. Admin can review, update, and resolve complaints.
- Collection Schedule Managements – Allow admin to create, update, and publish garbage collection schedules while enabling residents to view schedule information.
- QR Code Based Bin Management- Adding unique QR code to garbage bins and allow collection personnel to scan and update bin collection records and residents to check bin details.
- Truck Monitoring – Allow residents and admin to track garbage collection truck.
- Notification Management – Send notifications and updates to regarding collection schedules, complaint status changes.

- Feedback and Rating Management – Allow residents to submit feedback and rate garbage collection services to support service quality Improvement.
- Awareness Content Management – Enable administrators to publish garbage management guidelines, recycling information, and environmental awareness content for residents.

3.3.2 Non Function Requirement

Security and Privacy

- Access control – The system shall implement role based access control to ensure user can access only authorized functionalities.
- Secure authentication - User authentication mechanism shall prevent unauthorized access to the system.

Usability and User Experience

- User friendly interface – The system shall provide an intuitive and easy to use interface for residents, admin, and drivers.
- Consistency in design – All system pages shall maintain a consistency layout, navigation structure, and visual appearance.
- Error handing – The system shall display meaningful error messages and validation feedback to assist user in correcting mistakes.

Performance and Scalability

- Responsiveness – The system shall provide fast response times for user requests and system operations.
- Scalability – The system shall support increasing numbers of user, complaints, and schedules without significant performance degradation.
- Efficient data management – Database operations shall be optimized to quick storage and retrieval of information.

Reliability and Availability

- System Reliability – The system shall operate consistently with minimal failures and maintain data accuracy.
- Availability – The web application shall be accessible whenever users require system services.

Maintainability

• Modular Architecture – The system shall be developed using a modular structure to facilitate future updates, maintenance, and feature enhancements.

• Ease of Enhancement – New functionalities can be integrated with minimal impact on existing modules.

3.4 Components of the System

The CleanLand Garbage Management System consist of two main components, software, and database. These components work together to provide an efficient and centralized platform for managing garbage collection operations, complaint handling, collection schedules, QR code based bin management, feedback management, and truck tracking.

The software component includes web based interfaces used by residents, administrator, and drivers. The database component stores and manages all system information, including user accounts, complaints, schedules, QR codes, feedback records, and notification data. Supporting technologies such as email services and QR code generation libraries enable additional system functionalities and improve communication between stakeholders.

3.4.1 Software Components and Tasks

The software component of the CleanLand Garbage Management System consist of the frontend application, backend server, admin dashboard, and integrated third party services.

The system is developed using React.js for the frontend, Node.js and Express.js for the backend, and MySQL for database management. These technologies work together to provide the core functionalities of the system.

Frontend Application

The frontend provides user interfaces for residents, drivers, and admin. Users can register and log in to the system, submit complaints, view garbage collection schedules, provide feedback, access awareness content, and receive notification through a responsive web interface.

Backend Server

The backend manages user authentication, complaint processing, schedule management, QR code generation, notification service, and communication with the database.

Admin Dashboard

The administrative dashboard enables admin to mange residents, drivers, complaints, collection schedules, garbage bins, feedback, and awareness content. It provides centralized control and monitoring of system operations.

Google Maps Integration

Google Maps API is integrated into the system to support truck tracking functionality. It enables admin and residents to view garbage truck locations on an interactive map, improving operational visibility and helping residents stay informed about collection activities.

QR Code Integration

QR code technology is used to uniquely identify garbage bins. Drivers can scan QR codes during collection activities to update bin records.

Email Notification

The system integrates email services to send notifications regarding complaint status updates, collection schedules, and important announcement to residents.

This software architecture ensures efficient communication between stakeholders while improving transparency, accountability, and overall garbage collection management.

Login Page

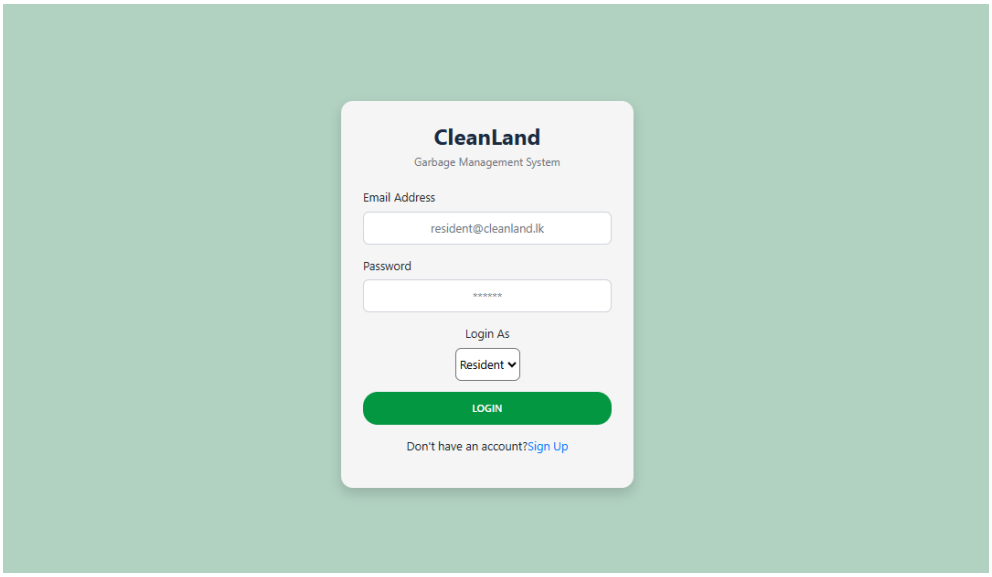


Figure 10: User login page

Registration Page

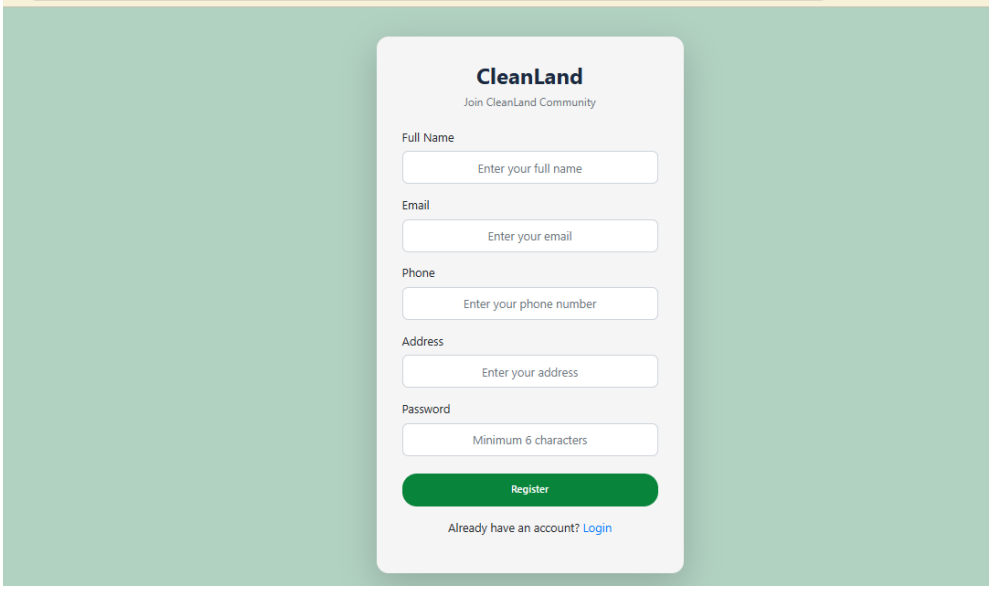


Figure 11: Resident registration page

Admin Dashboard

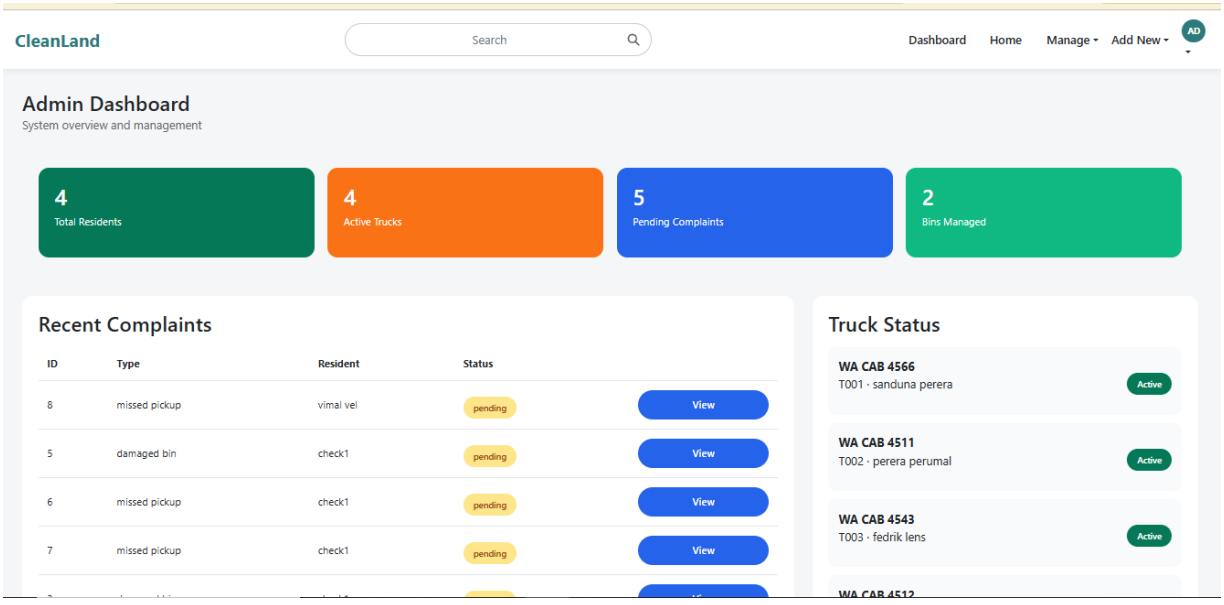


Figure 12: Admin dashboard 1

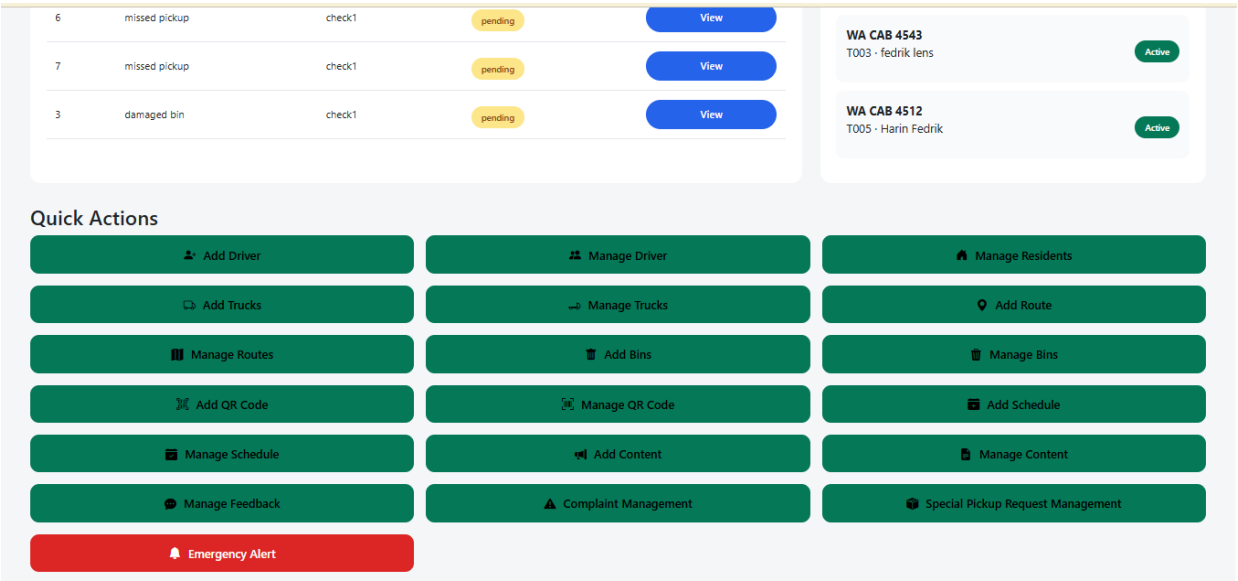


Figure 13: Admin dashboard 2

Add New Driver

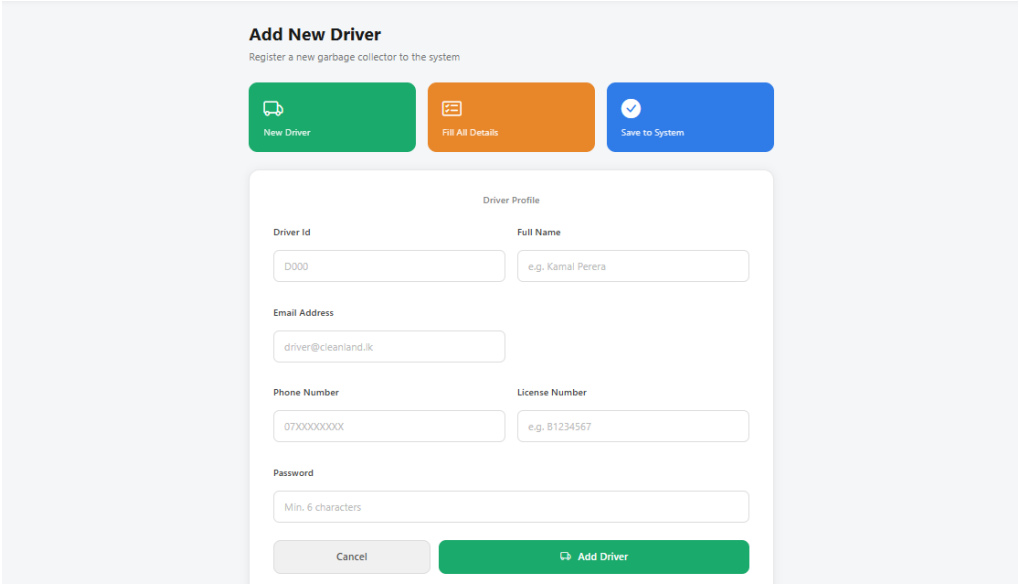


Figure 14: Add new driver page

Driver Managemaent Page

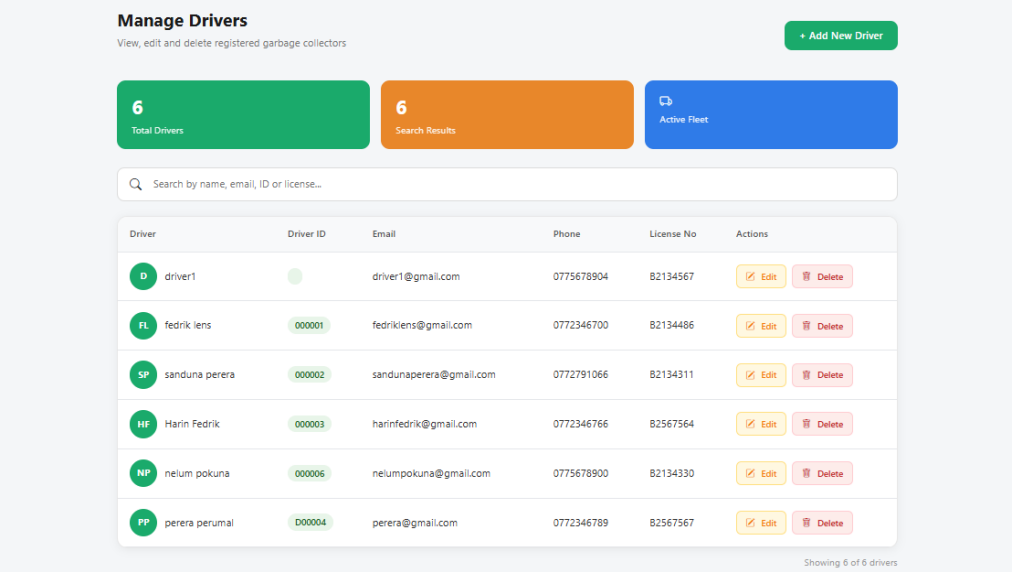


Figure 15:Driver management page

Update Driver Page

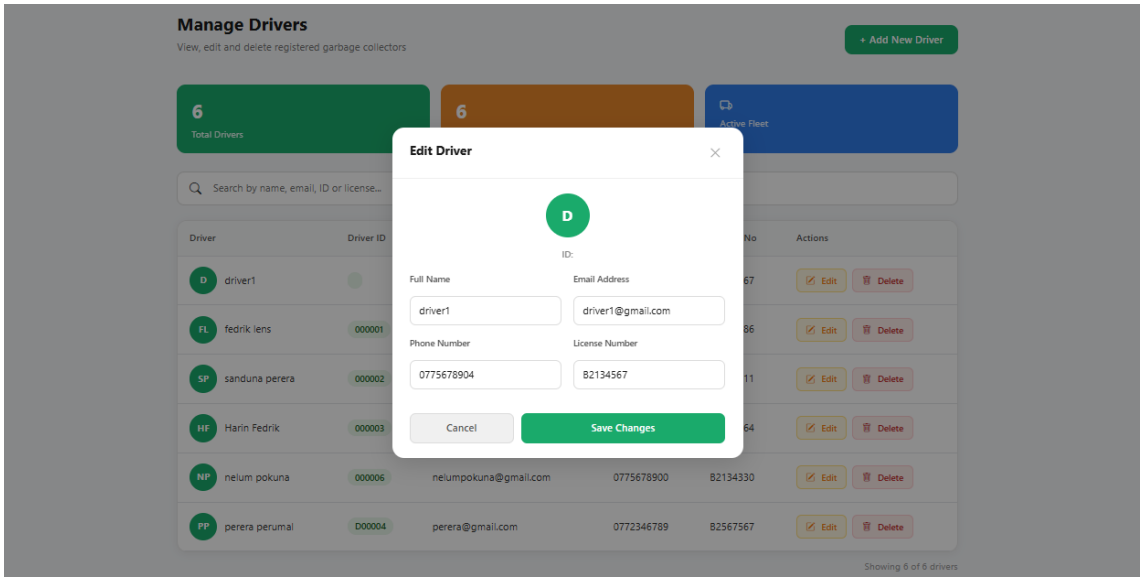


Figure 16: Update drive page

Delete Driver Page

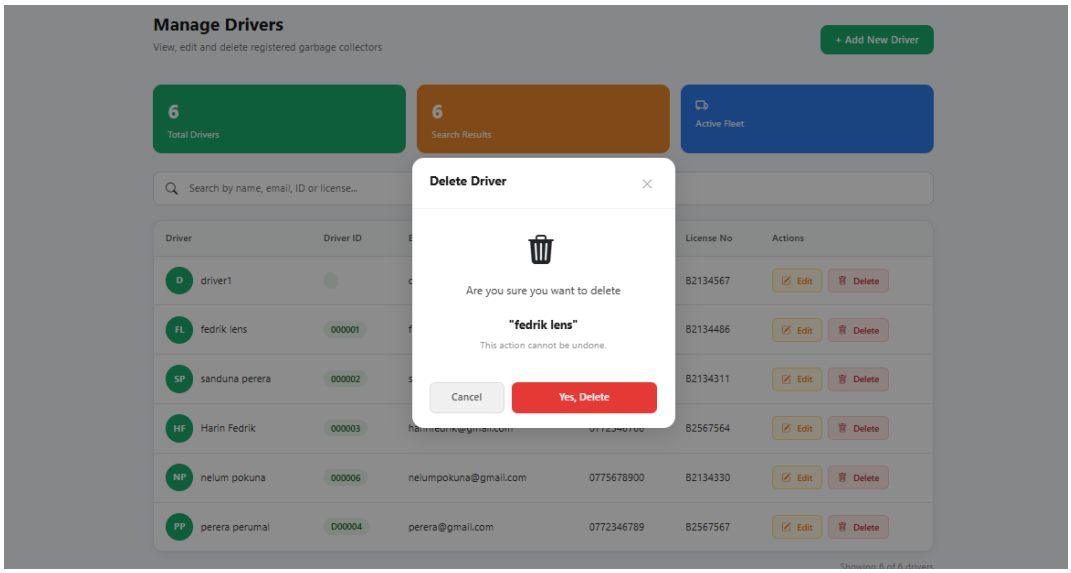


Figure 17:Driver delete page

QR Code Page

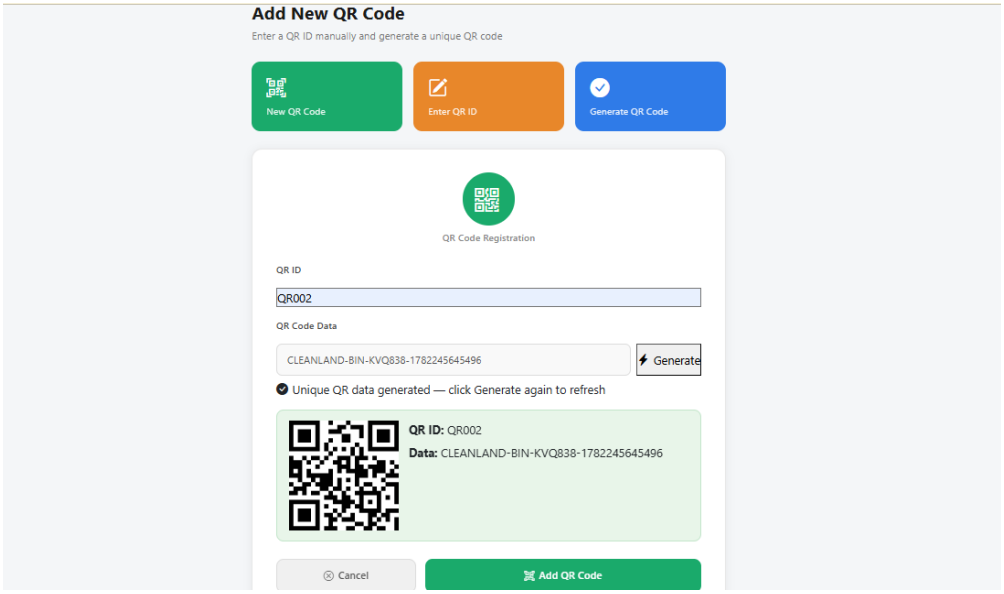


Figure 18: Add QR code page

Special Pickup Request Page

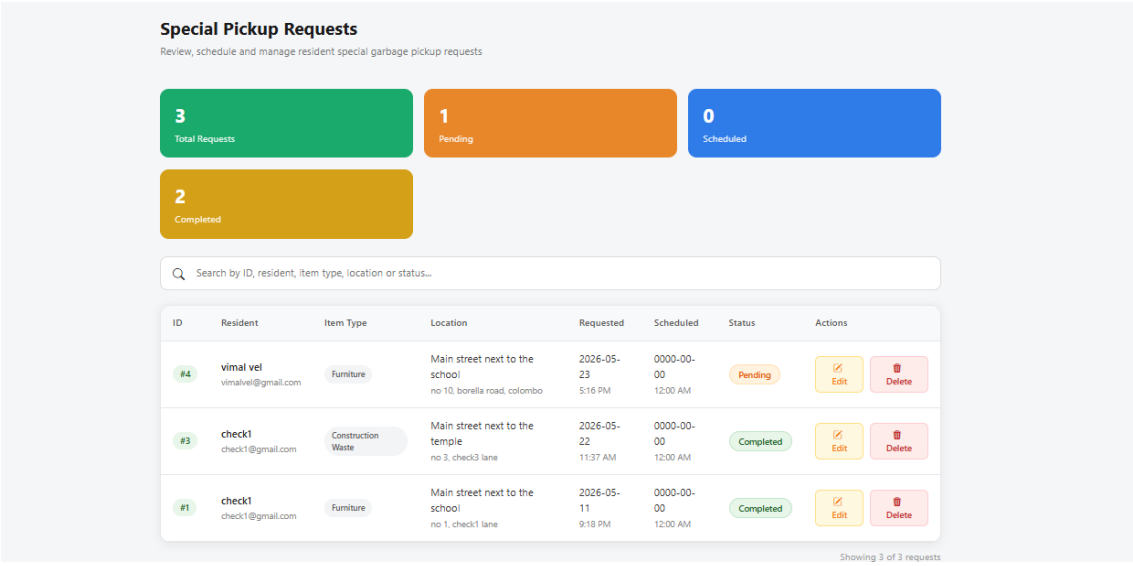


Figure 19: Special pickup request page

Schedule management page

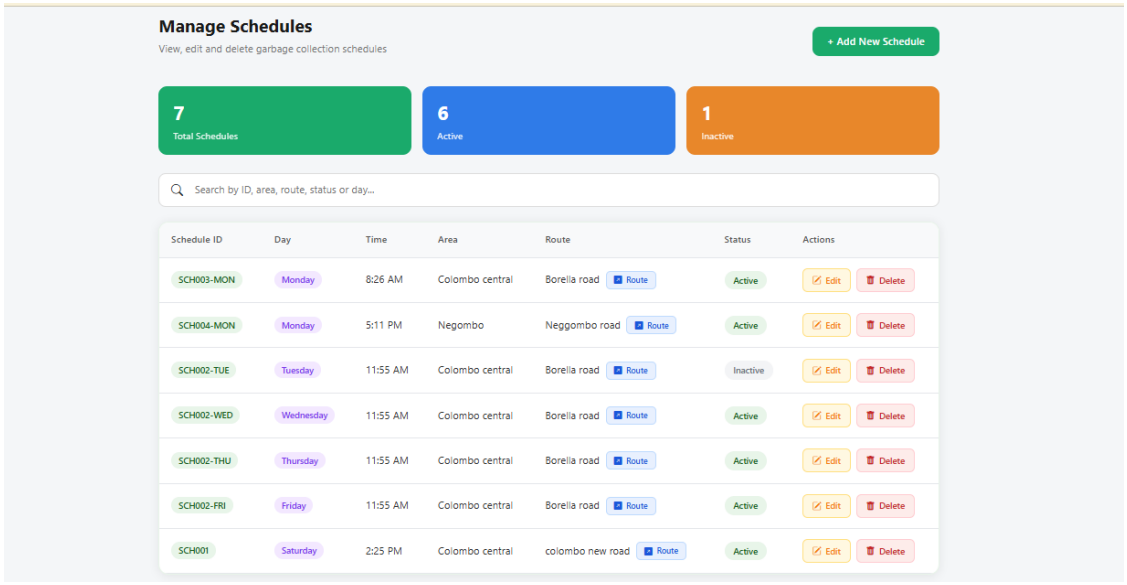


Figure 20: Schedule management page

Resident management page

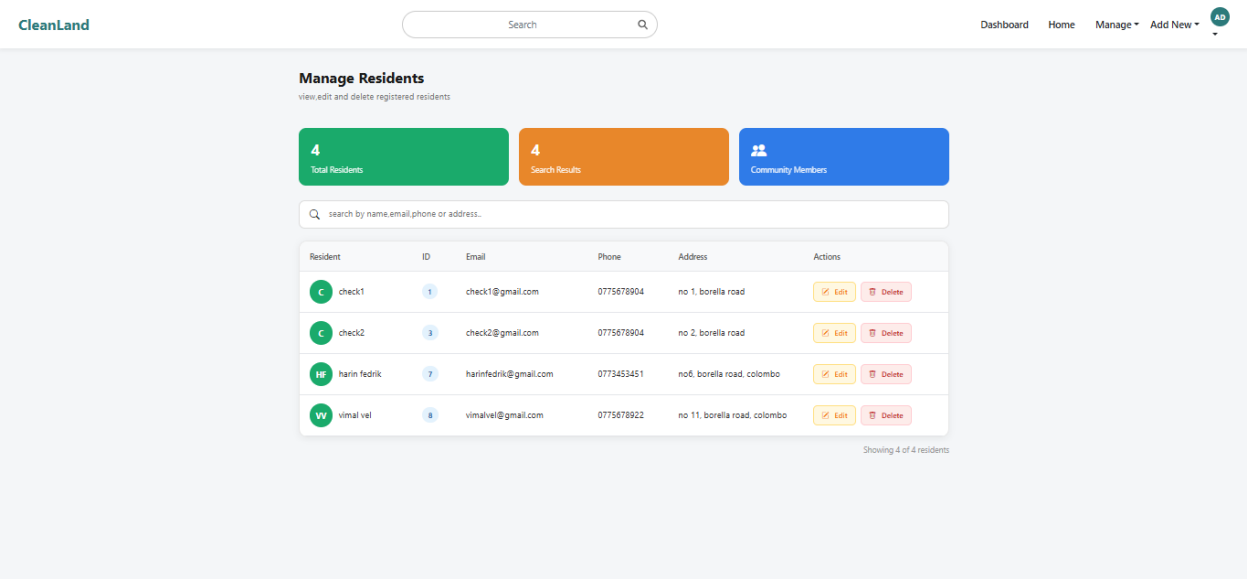


Figure 21:Resident management page

Add Truck Page

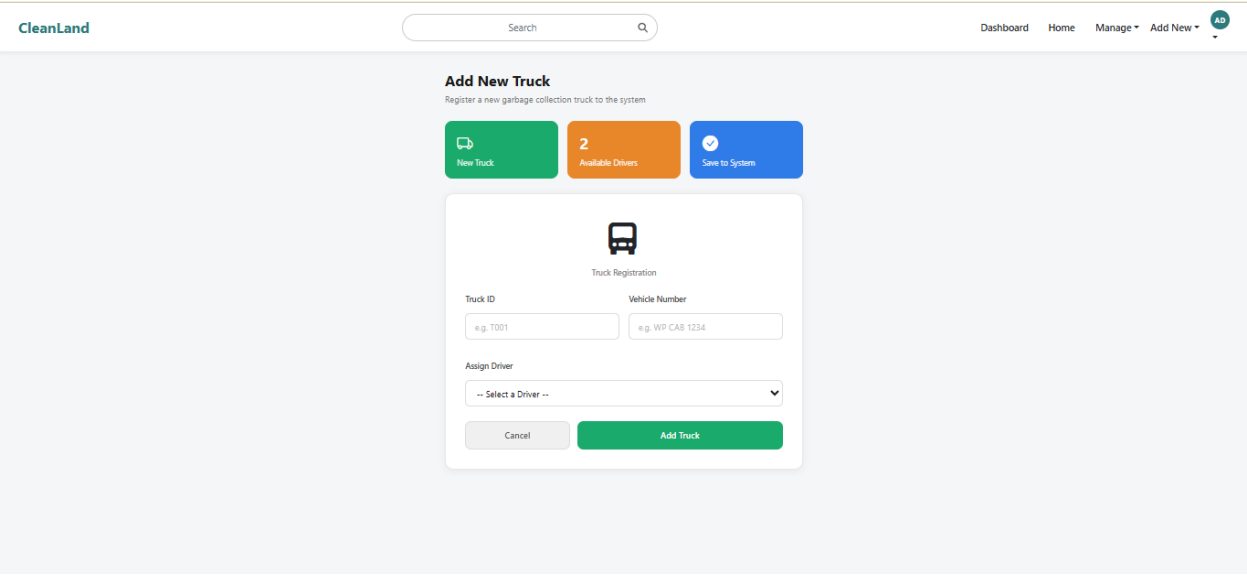


Figure 22: Add truck page

Truck management page

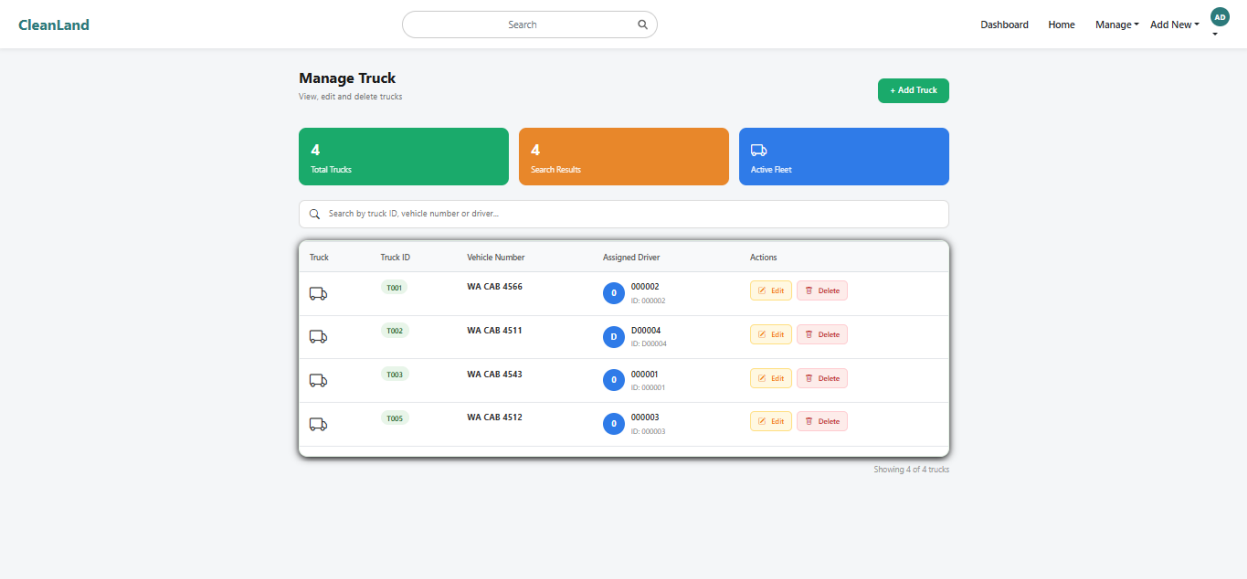


Figure 23: Truck management page

Add Route Page

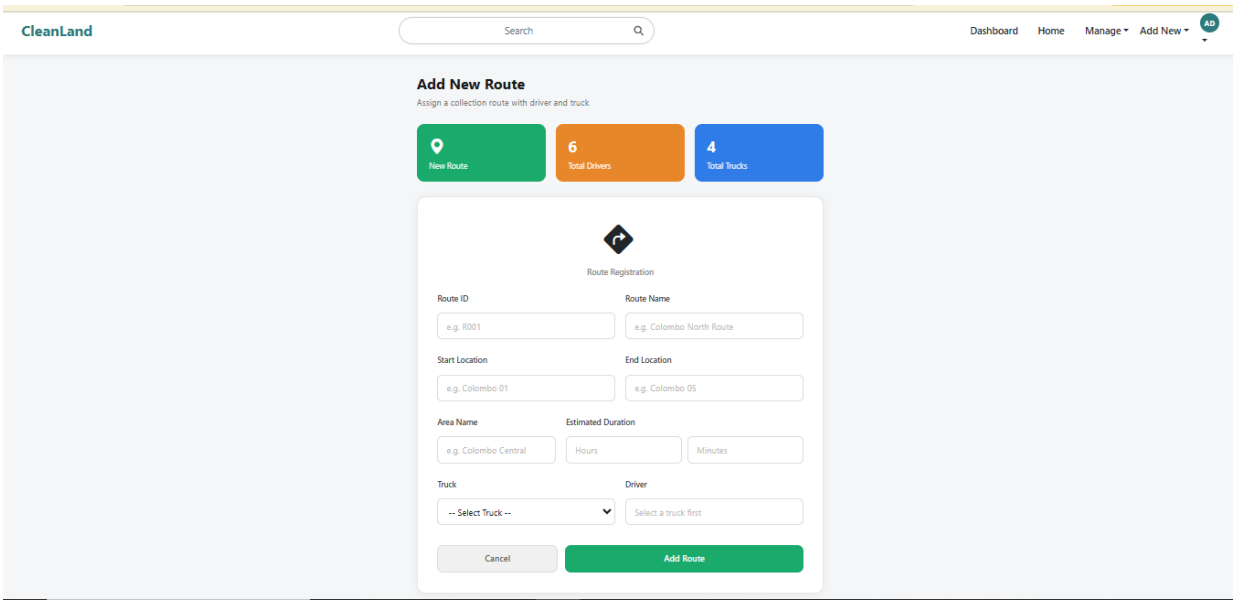


Figure 24: Add route page

Route management page

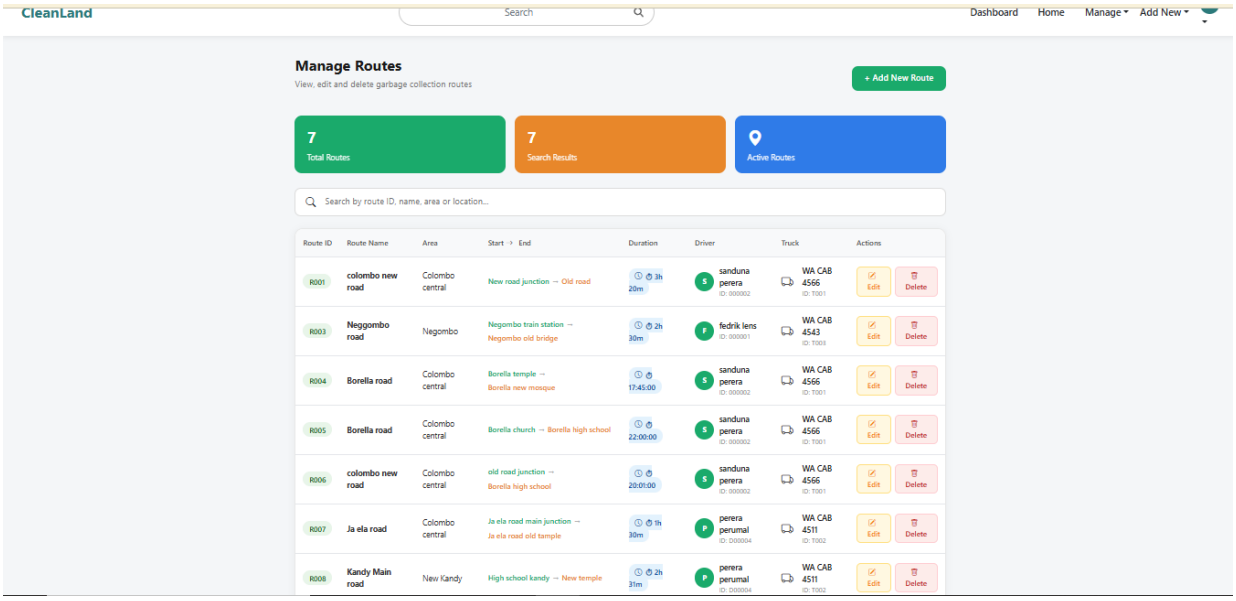


Figure 25: Route management page

Add Bin Page

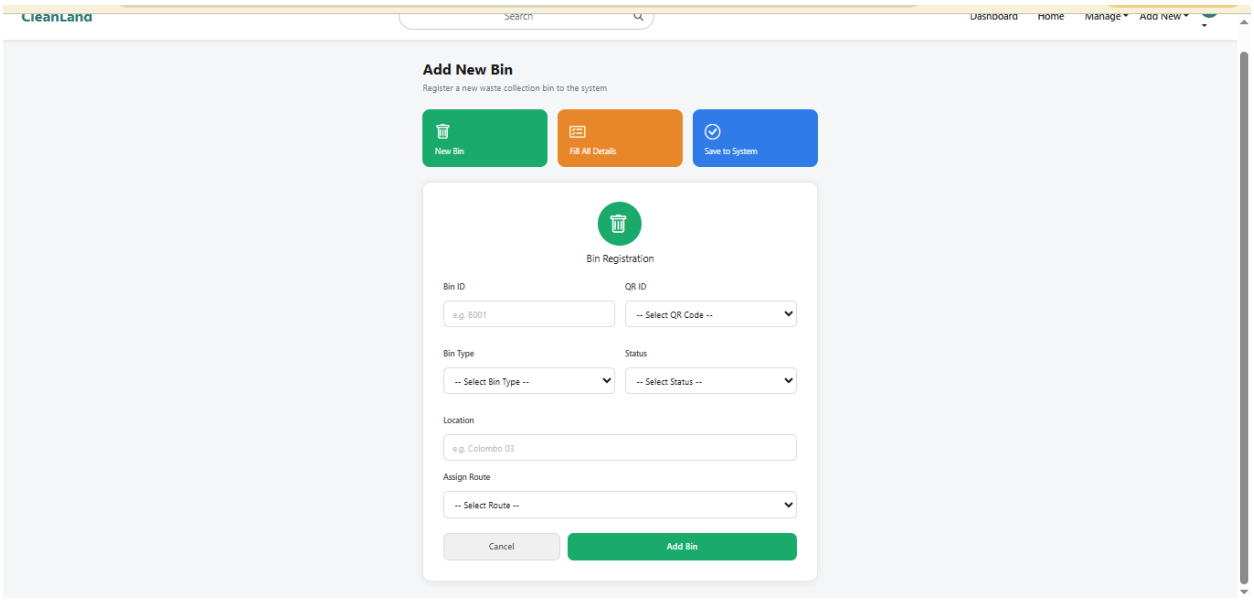


Figure 26: Add bin page

Bin management page

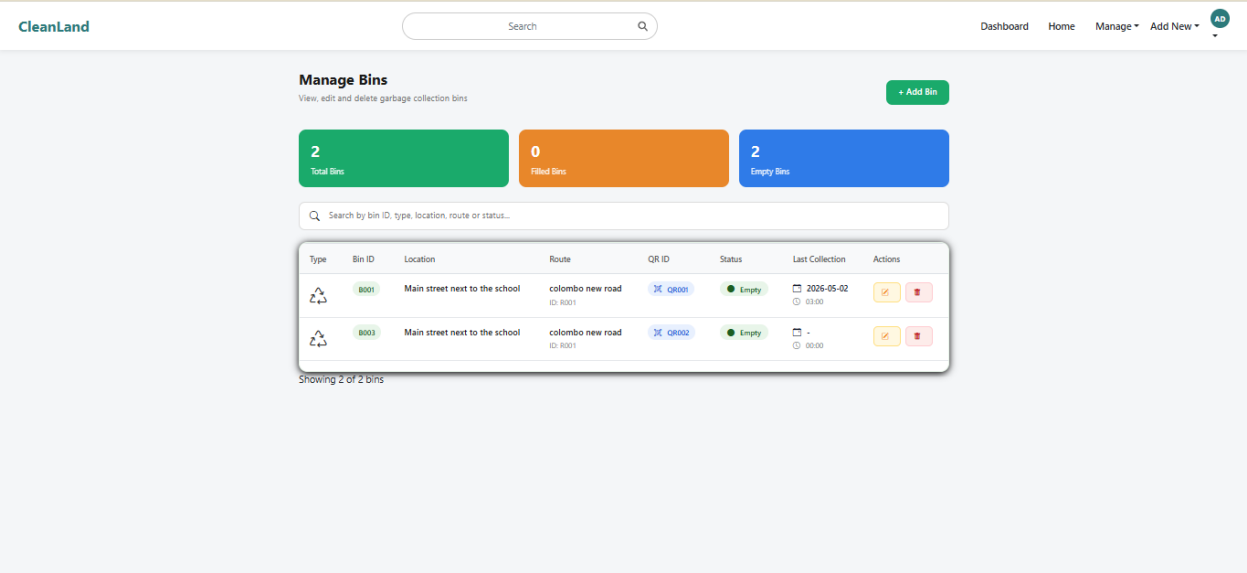


Figure 27: Bin management page

Update Bin Page

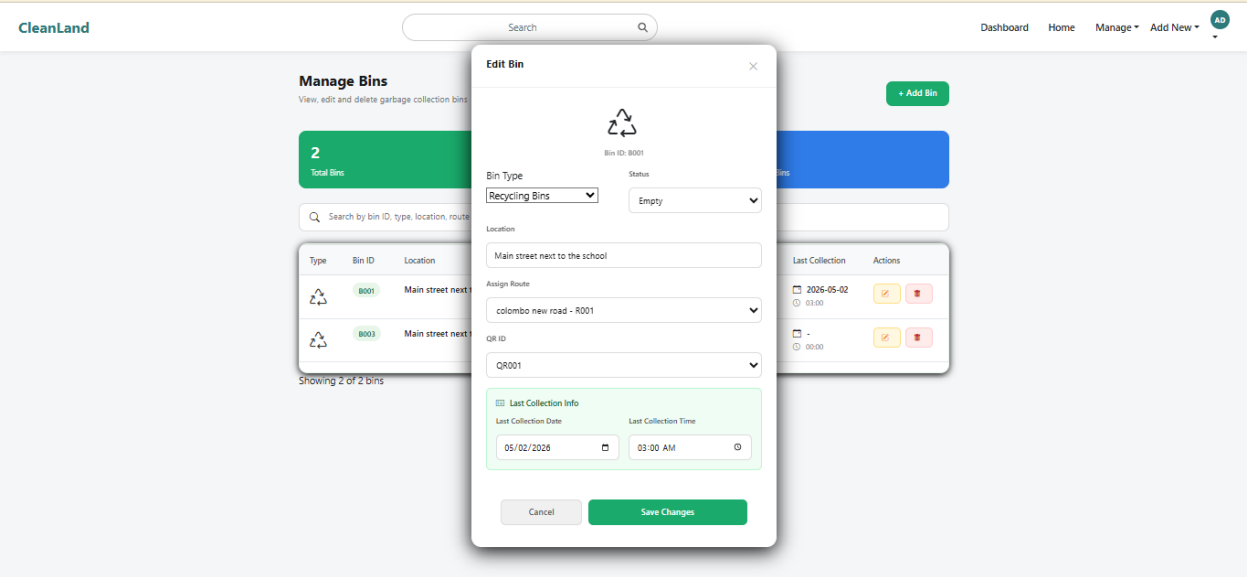


Figure 28: Bin update page

Add Content Page

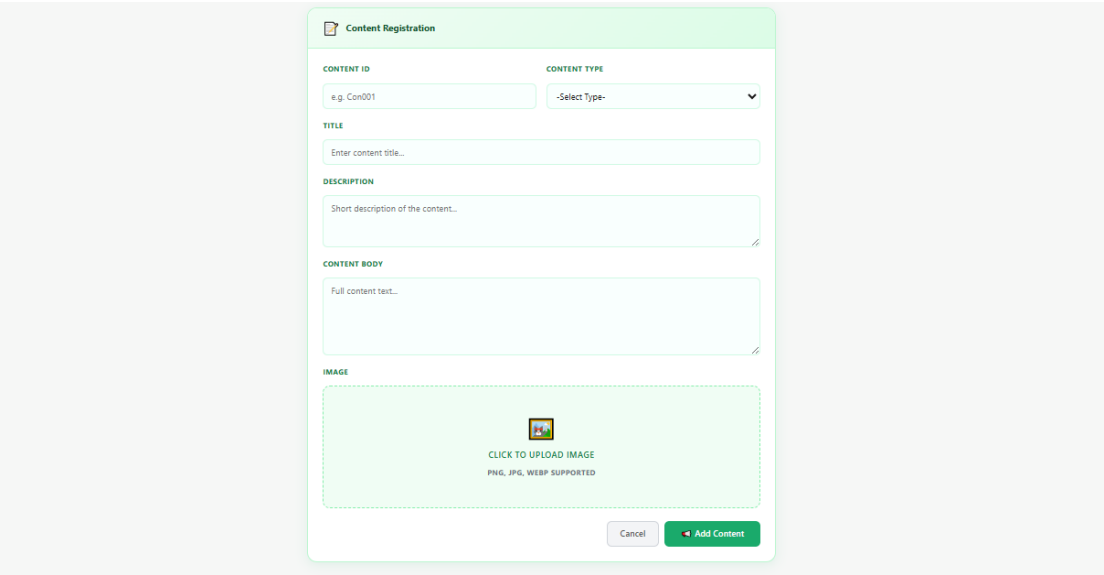


Figure 29: Add content page

Home Page

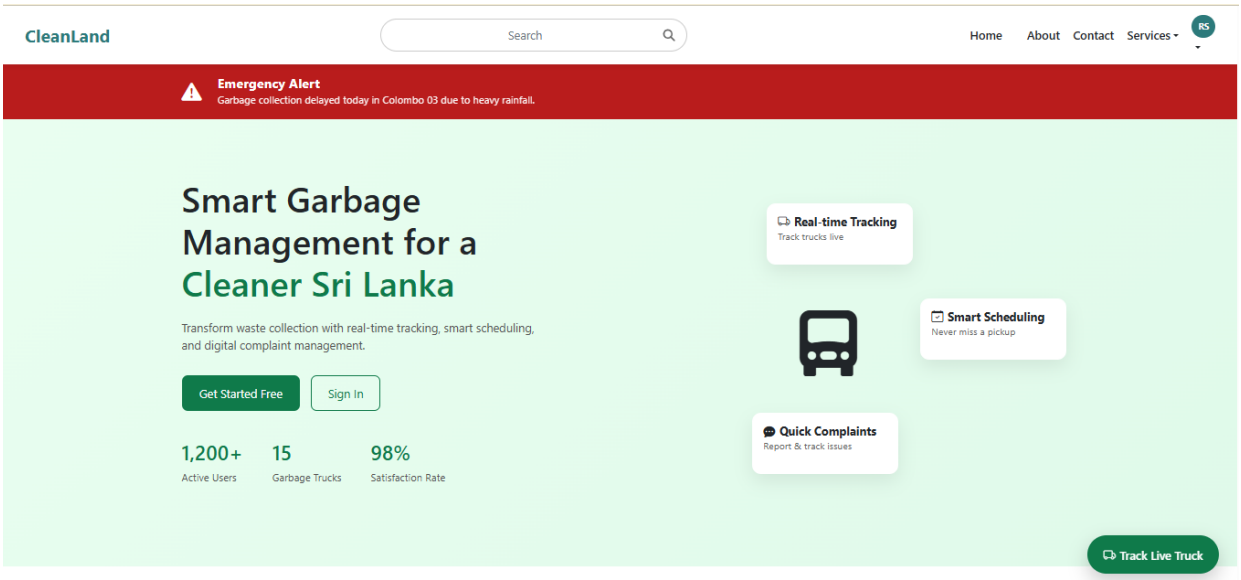


Figure 30: Home page 1

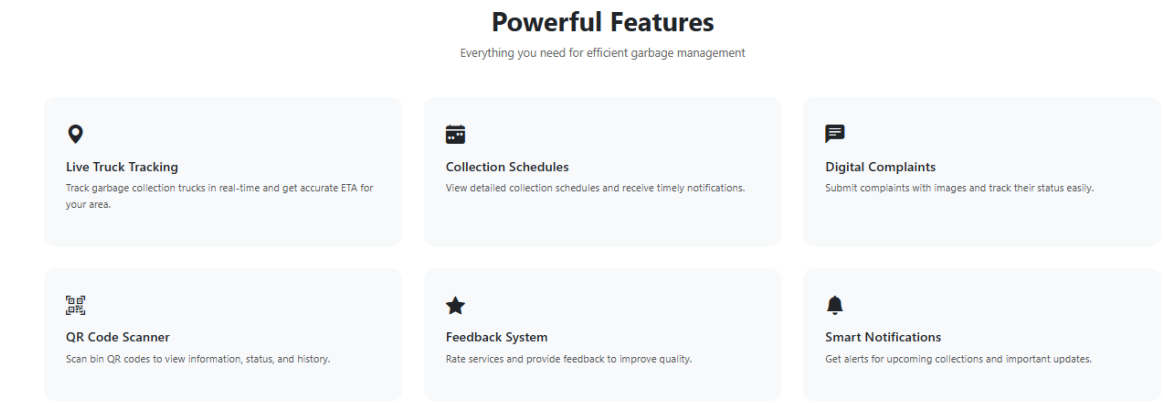


Figure 31: Home page 2

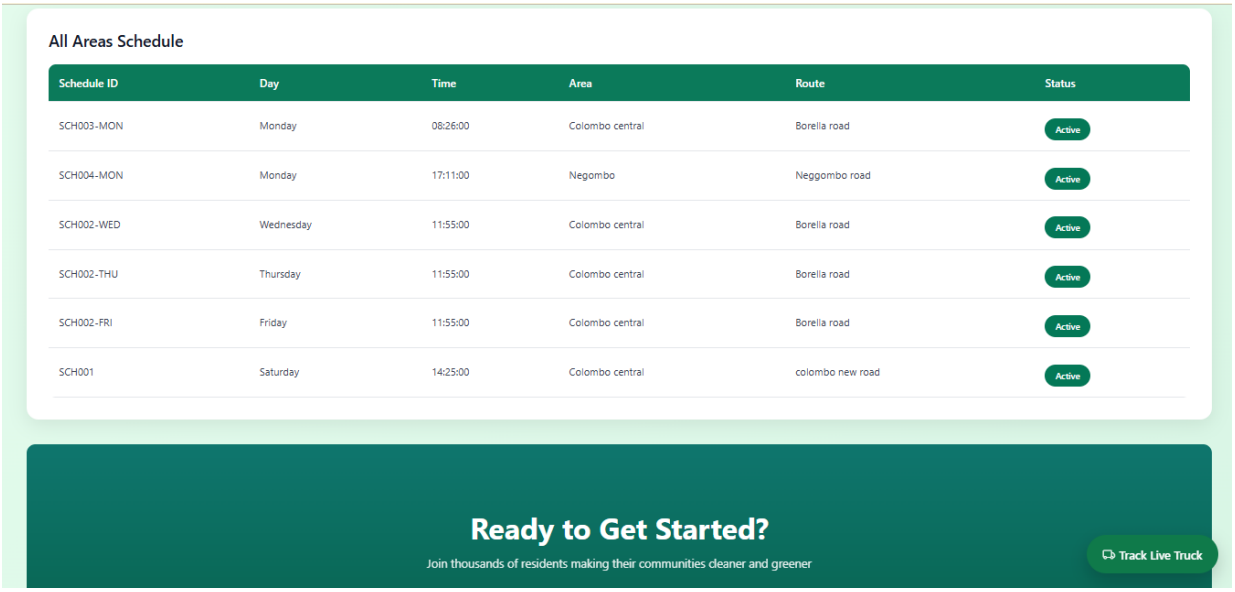


Figure 32: Home page 3

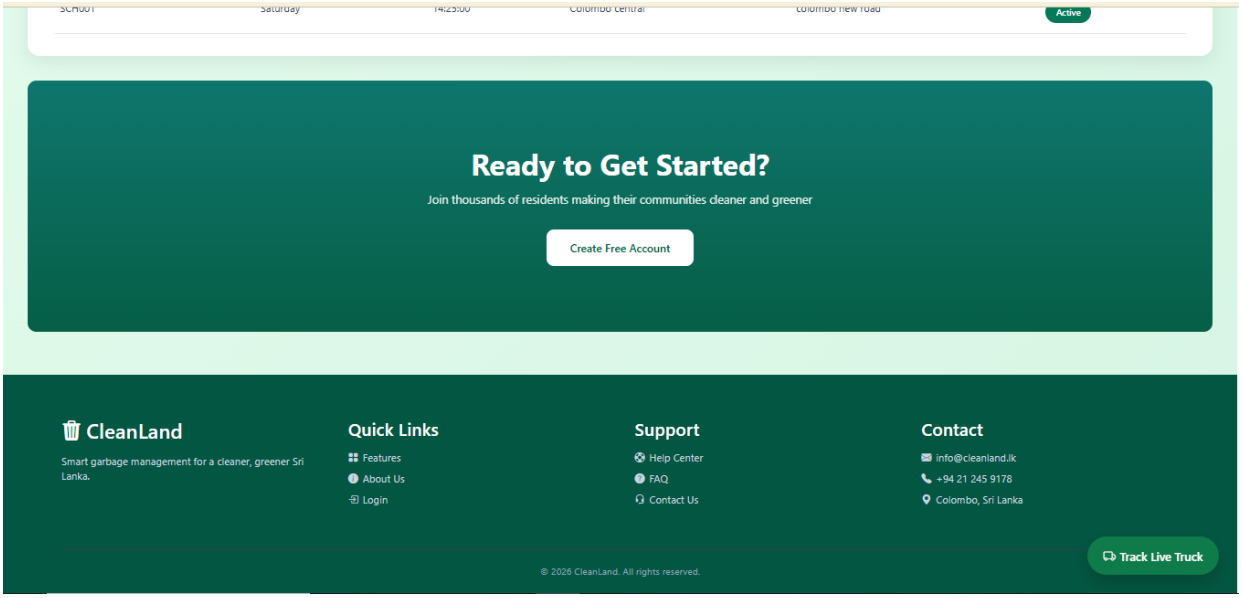


Figure 33: Home page 4

Schedule Page

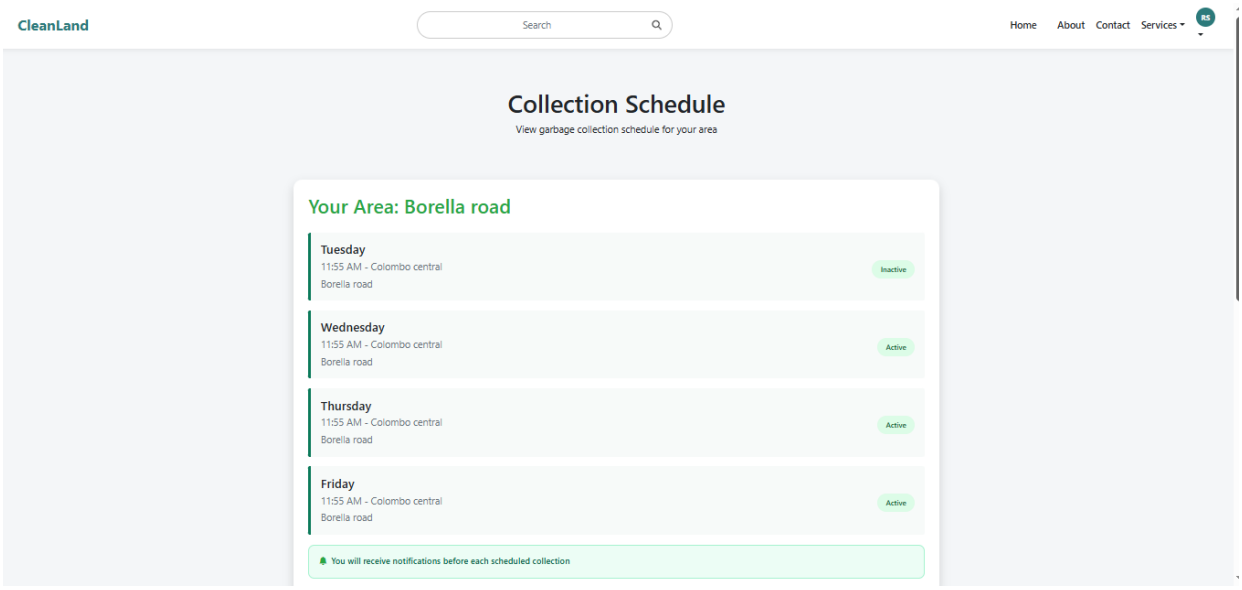


Figure 34:Schedule page

Complaint Management Page

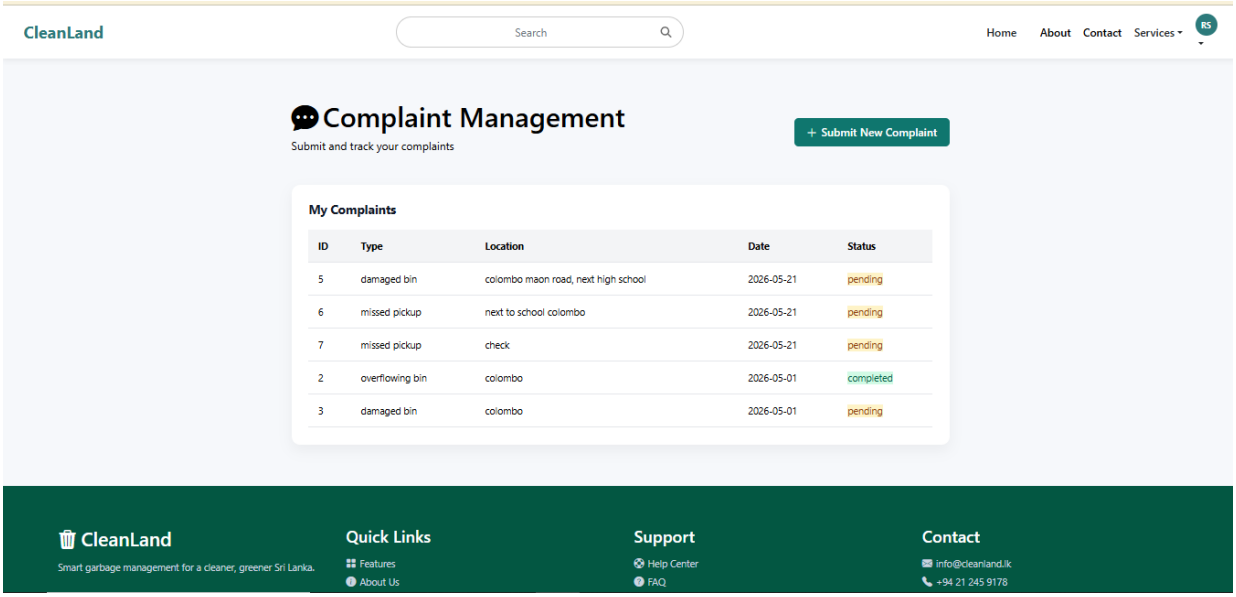


Figure 35: Complaint management page

Feedback Page

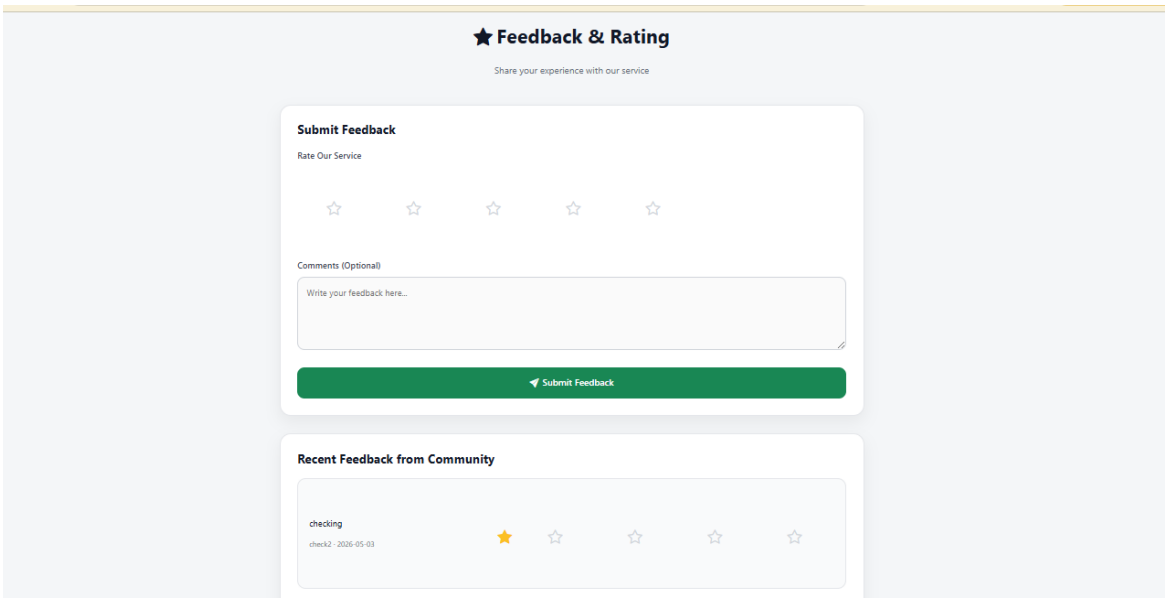


Figure 36: Feedback page

Garbage Pickup Request Page

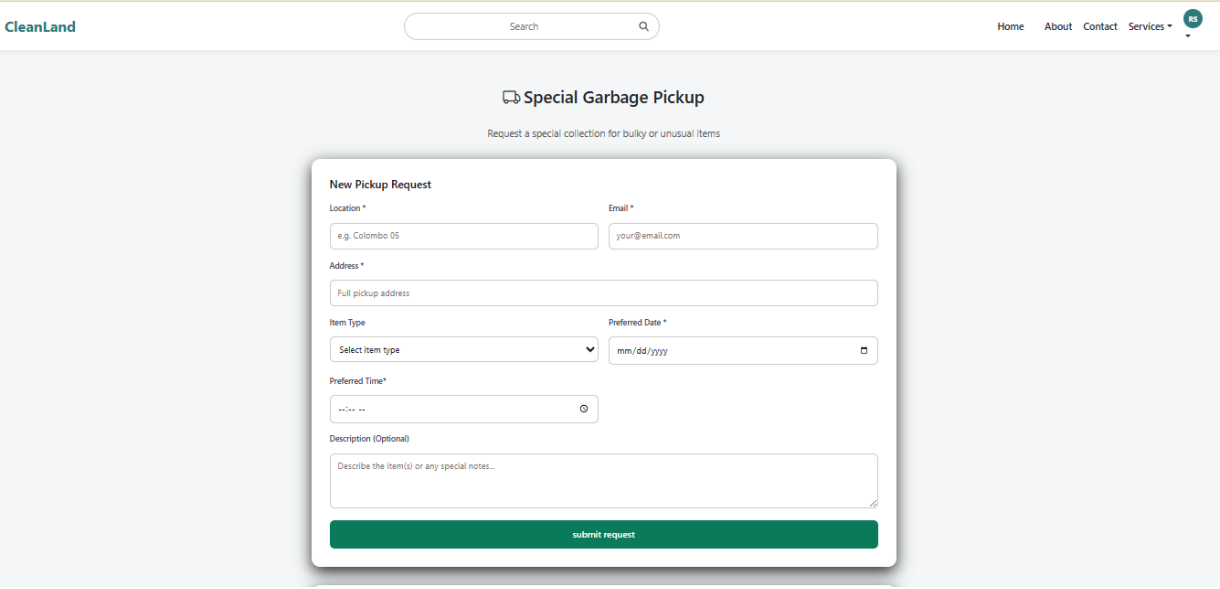


Figure 37: Garbage pickup request page

3.4.2 Database Component

In the database component of the proposed CleanLand Garbage Management System, all essential information related to residents, admin, drivers, garbage bins, complaints, schedules, feedback, notification, routes, special requests is stored and managed. The database ensures secure, organized, and efficient handling of data, enabling quick retrieval, modification, and maintenance of records when required.

The database serves as the central repository of the system and supports the smooth operation of all functionalities. Information stored includes user profiles, driver details, garbage collection schedules, complaint records and status updates, QR code based bin information, feedback and rating data, awareness content, notification records, and truck monitoring data. The database also maintains historical records of collection activities and complaint resolutions, allowing admin to monitor system performance and service quality.

The database supports report generation and data analysis, helping municipal authorities identify operational issues, evaluate collection efficiency, and make informed decisions. By providing

reliable data storage and management, the database plays a vital role in ensuring the effectiveness, transparency, and scalability of the CleanLand Garbage Management System.

3.4.2.1 Entities

admin, awarness_content, bin, complaint, driver, emergency_alert, feedback, gps_tracker, notification, qr_code, resident, route, schedule, special_request, truck

3.4.2.2 Relational Schema

Entity	Attributes
Admin	admin_id, email, name, password
awarness_content,	content_id, title, content_type, image, content_body_description
Bin	bin_id, type, status, location, route_id, qr_id, lst_clctn_time,lst_clctn_date
Complaint	complaint_id, complaint_type description, image, location, status, date, resident_id
Driver	driver_id, email, name, password, phone_no, license_no
emergency_alert	alert_id, is_active, expired_time_create_time, message, tittle, alert_type,resident_id
feedback	feedback_id, comment, rating, submitted_date, resident_id
gps_tracker	tracker_id, time_stamp, current_status, latitude, longtitude, speed, truck_id
notification	notification_id, title, message, sent_time, notification_type, resident_id
qr_code	qr_id, data
resident	resident_id, Email, Name, Password, Phone_no, Address
route	route_id, start_location, end_location, estimated_duration, route_name, area_name, driver_id, truck_id
Schedule	schedule_id, time, area, status, day_of_week, route_id
special_request	request_id, request_time, request_date, location, email, address, description, status, schedule_date, schedule_time, resident_id, item_type
Truck	truck_id, vehicle_number, driver_id

Table 2: Relational schema

3.4.2.3 Entity Relationship

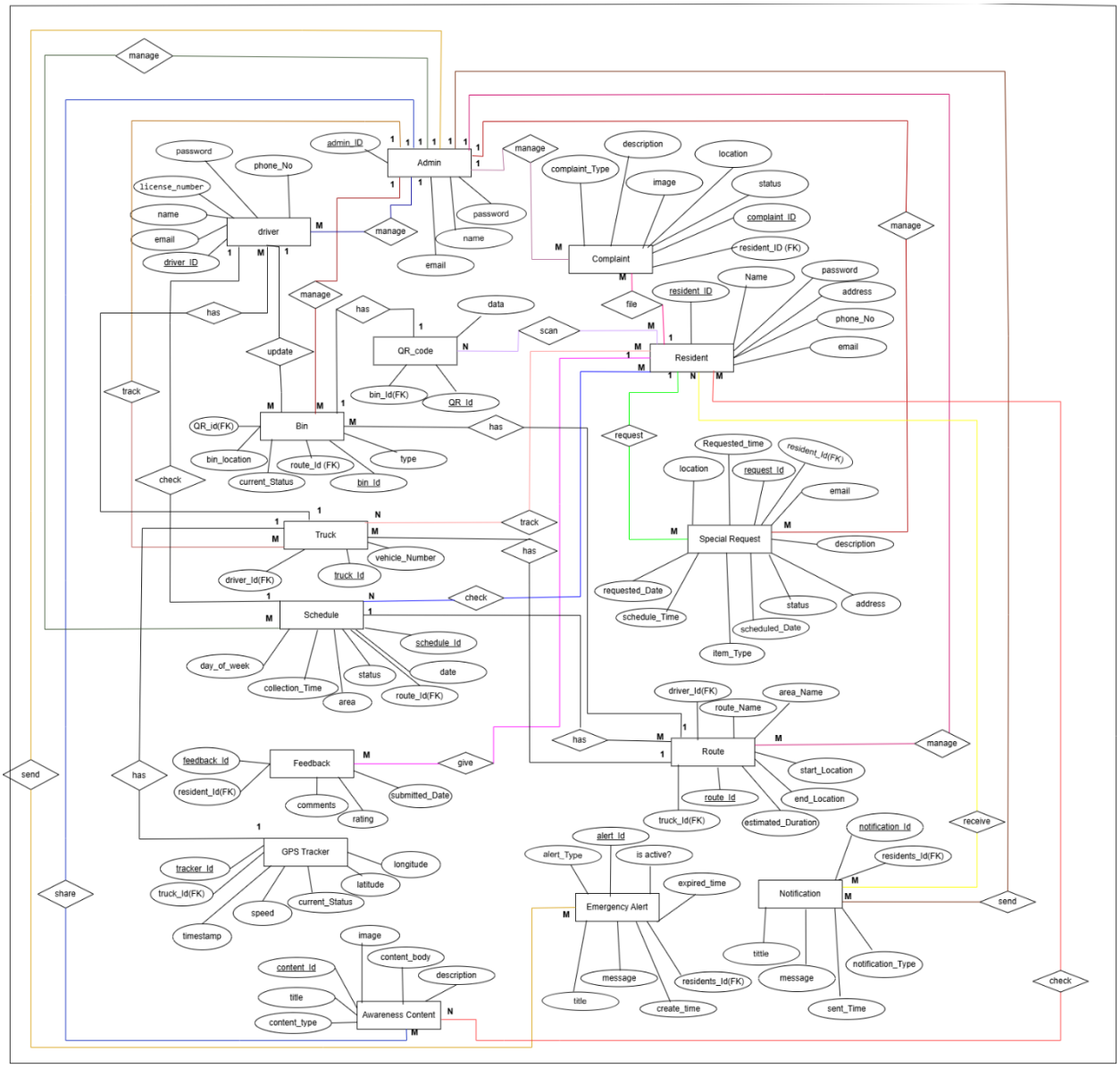


Figure 38: ER diagram

Assumptions

- Each resident, driver, and administrator has a unique ID, email address, and password for authentication and authorization purpose.

- Residents can submit multiple complaints, feedback entries, and special collection requests, all of which are linked to their unique Resident ID.
- Each complaint is assigned a unique Complaint ID and contains information such as complaint type, description, location, image, and status for tracking purposes.
- Each garbage bin has a unique Bin ID and is associated with a unique QR code, allowing collection activities to be recorded and monitored digitally.
- QR codes are generated and assigned to garbage bins, enabling drivers to scan and update bin collection information during garbage collection operations.
- Each garbage collection truck has a unique Truck ID and vehicle number. A truck can be assigned to one or more collection routes and schedules.
- Drivers are uniquely identified by a Driver ID and can be assigned to specific trucks, routes, and collection schedules.
- Collection schedules contain information such as collection date, time, area, route, and status to ensure organized garbage collection operations.
- Each route has a unique Route ID and includes route details such as route name, start location, end location, and estimated duration.
- • GPS tracking data is associated with garbage collection trucks and contains information such as location coordinates, speed, timestamp, and current status.
- Residents can submit feedback and ratings regarding garbage collection services, which are stored for service evaluation and improvement purposes.
- Awareness content is managed by administrators and includes waste management guidelines, recycling information, and public awareness materials.
- • Notifications are generated and delivered to residents regarding collection schedules, complaint status updates, special requests, and service announcements.
- The system assumes that users have internet connectivity when accessing web based services such as complaint submission, schedule viewing, QR code updates, and truck monitoring.
- All data stored within the system is maintained in a centralized MySQL database to ensure consistency, integrity, and efficient retrieval of information.

3.4.3 Hardware Component

The CleanLand Garbage Management System is a web-based application that can be accessed through desktop computers, laptops, tablets, and smartphones with internet connectivity. These devices serve as the primary interface for residents, drivers, and administrators to access the system's functionalities.

A server is required to host the backend application and MySQL database, enabling data processing, storage, and communication between system components. Additionally, devices with camera support can be used for QR code scanning and garbage bin identification.

These hardware components ensure reliable access to the system and support efficient garbage management operations.

3.5 Technologies Adapted

In the development of the CleanLand Garbage Management System, several technologies were utilized to support different aspects of the system.

- **Development Frameworks** – React.js was used for frontend development, while Node.js and Express.js were used for backend development and API management.
- **Database Management** – MySQL was used as the database management system to store and manage user data, complaints, schedules, feedback records, QR code information, and other system related data.
- **QR Code Technology** – QR code libraries were used to generate and manage unique QR codes for garbage bin identification and collection record updates.
- **Google Maps Integration** – Google Maps API was used to support truck monitoring functionality by displaying truck locations on an interactive map.
- **Notification Services** – Email services were integrated to send notifications regarding complaint status updates, collection schedules, and important announcements.
- **Version Control and Project Management** – Git and GitHub were used for version control, source code management, and collaborative development throughout the project lifecycle.

3.6 System Design

3.6.1 Use Case Diagram

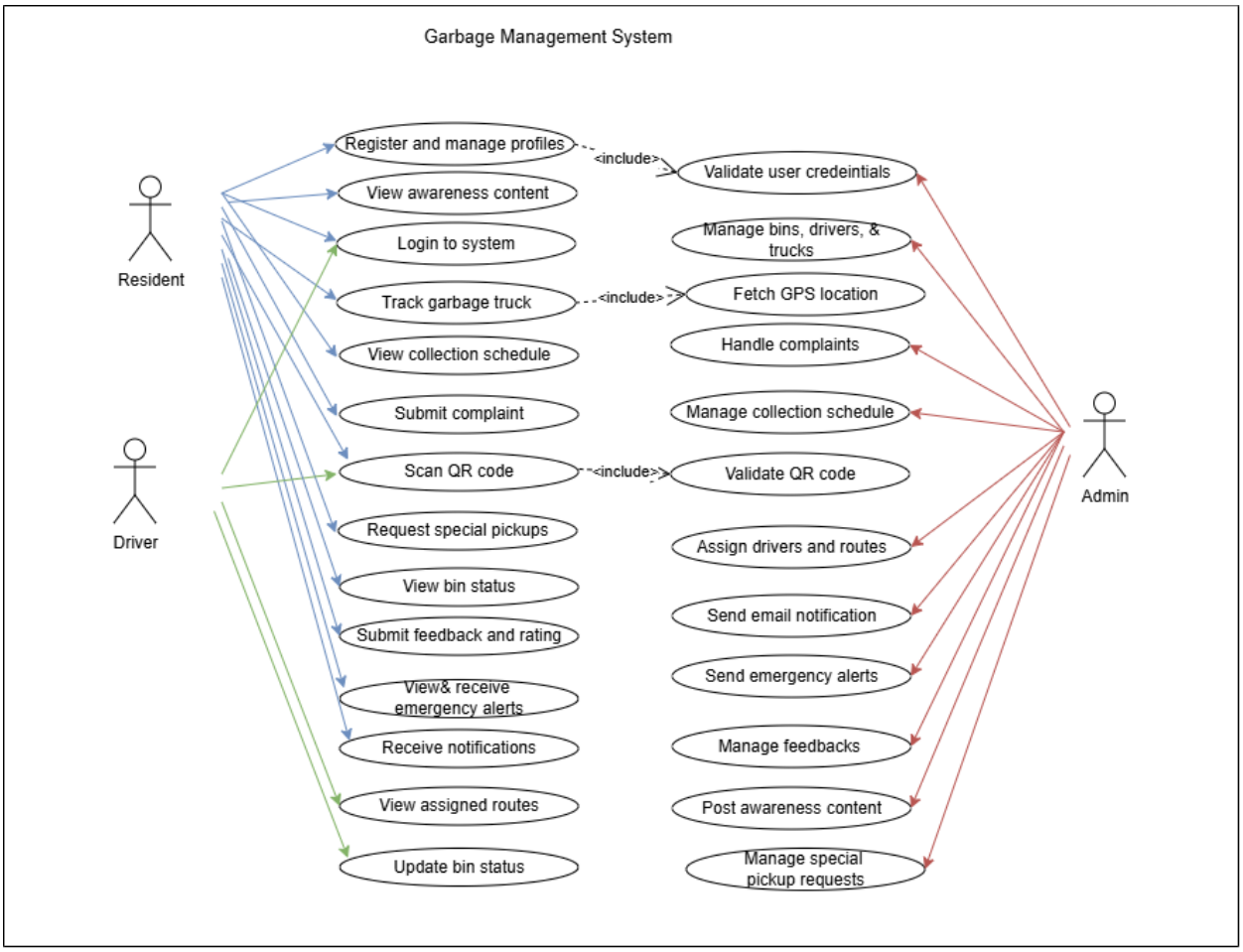


Figure 39: Use case diagram

3.6.2 Activity Diagram

Login

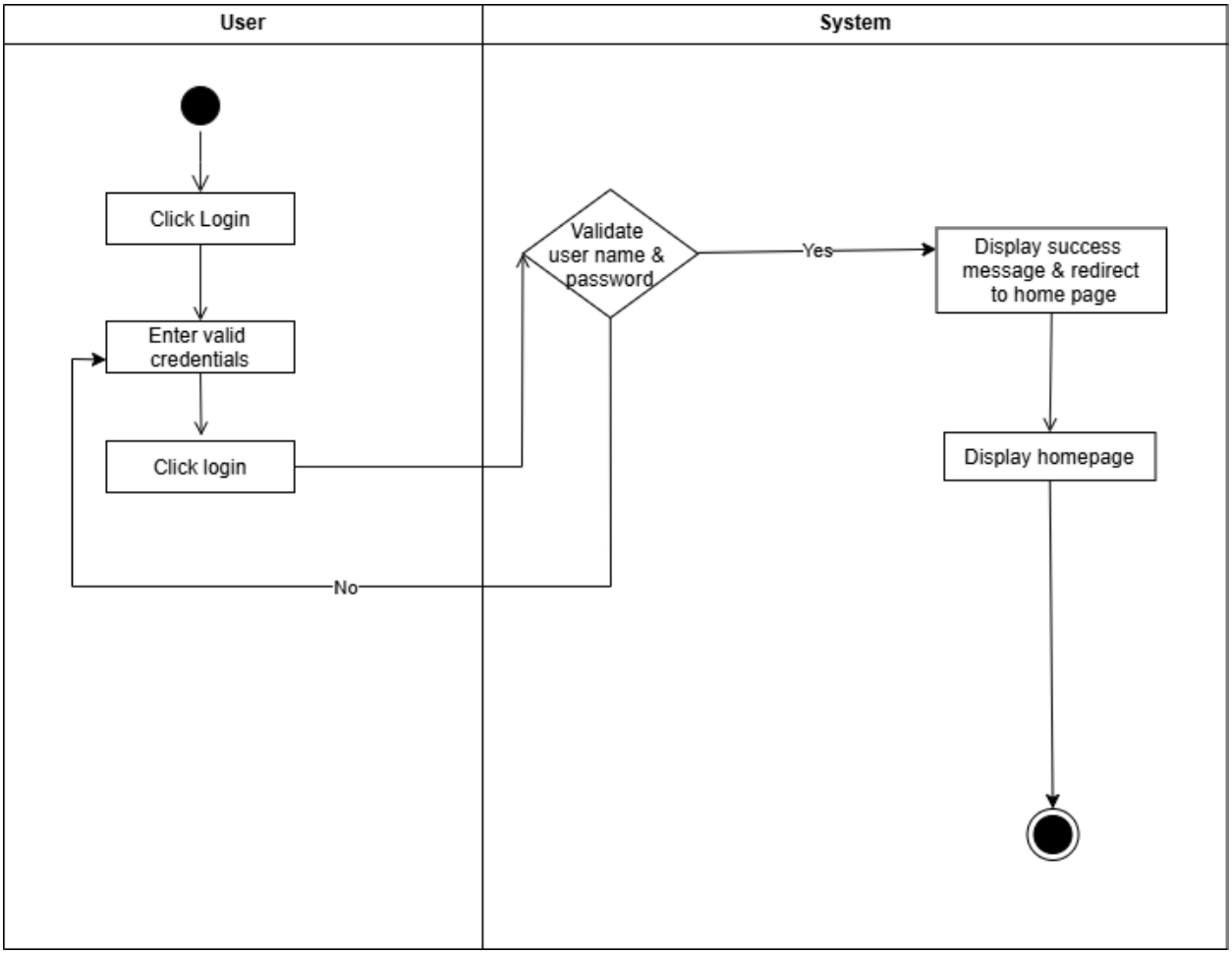


Figure 40:User login activity diagram

User registration

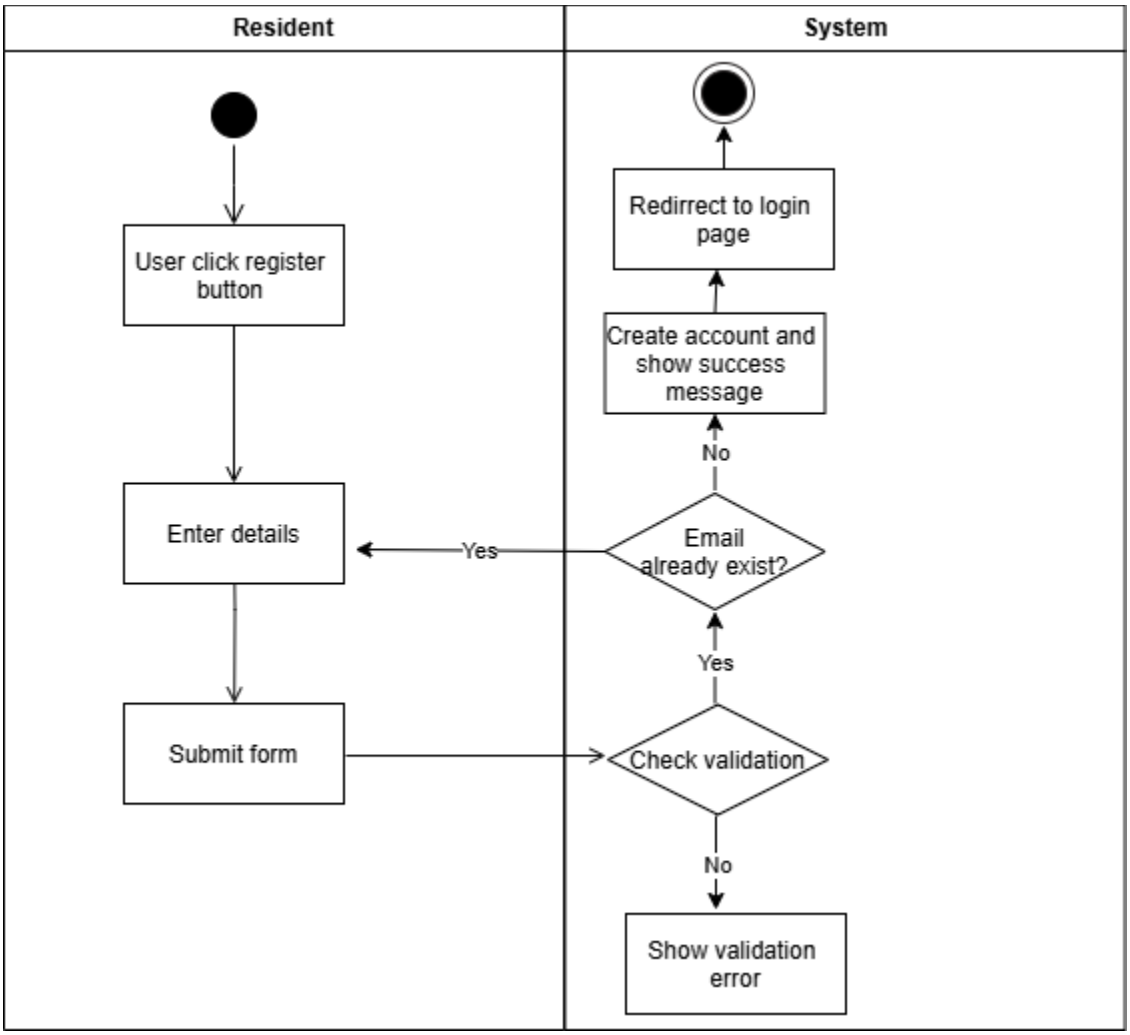


Figure 41: User registration activity diagram

Garbage Collection Scheduling

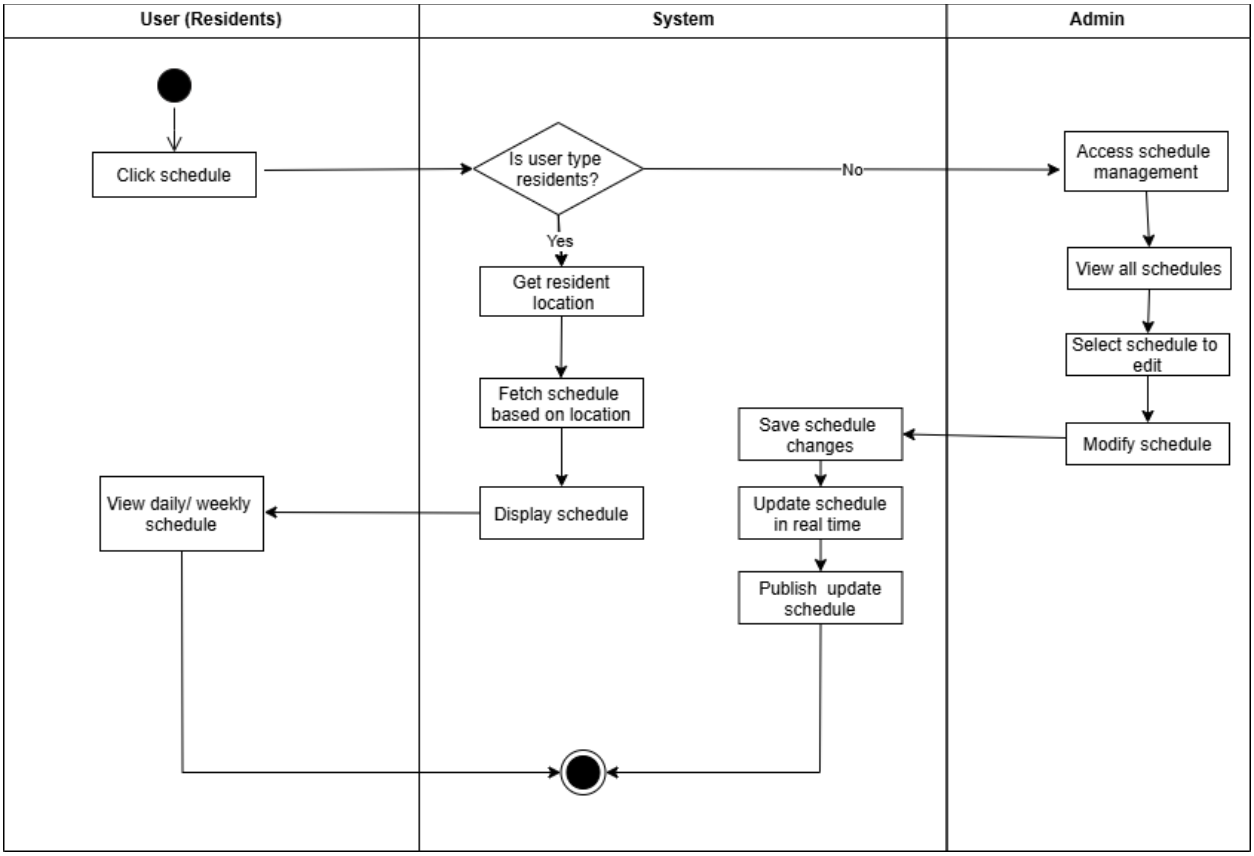


Figure 42: Garbage collection schedule activity diagram

Special pickup request

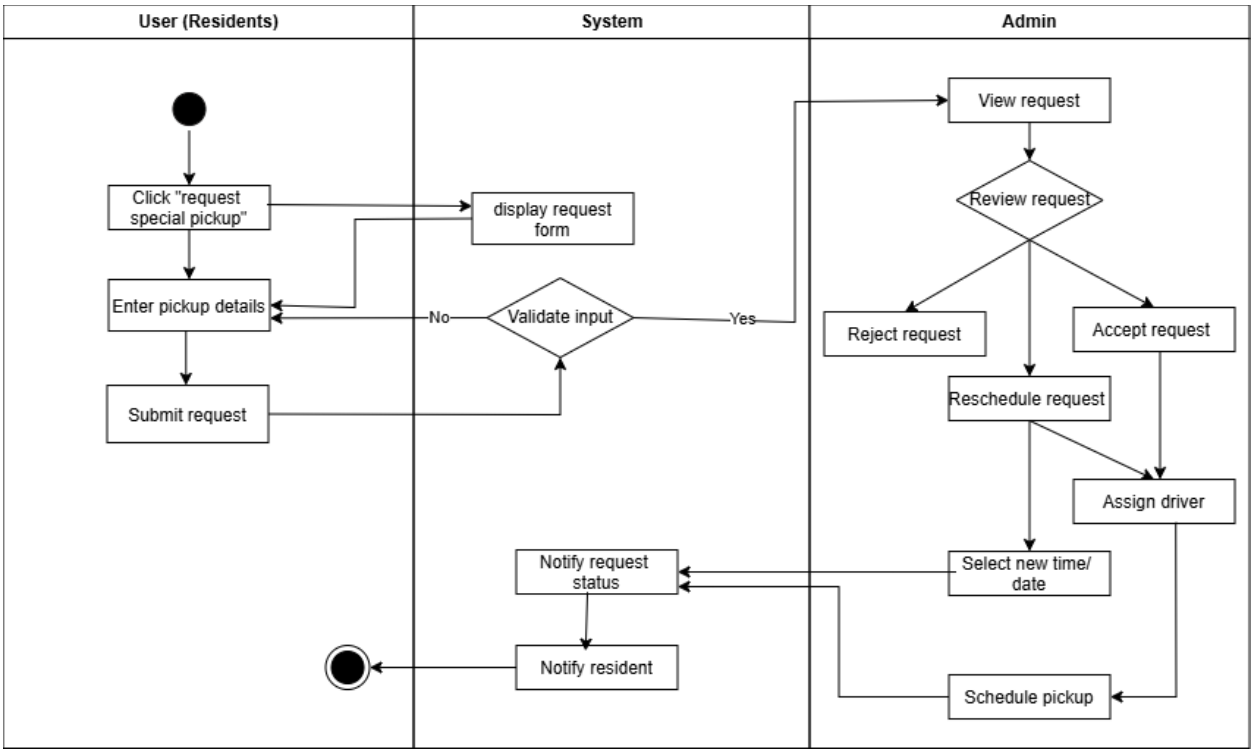


Figure 43: Special pickup request activity diagram

Awareness Content

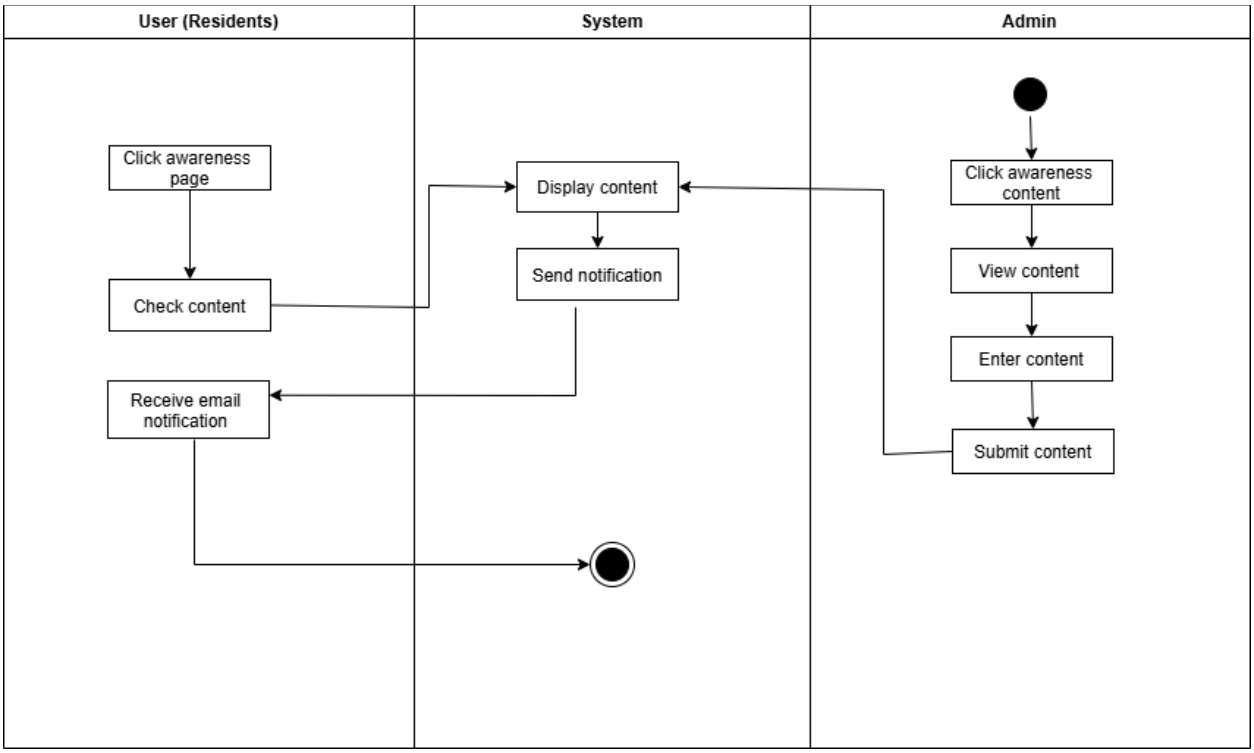


Figure 44: Awareness content activity diagram

Feedback

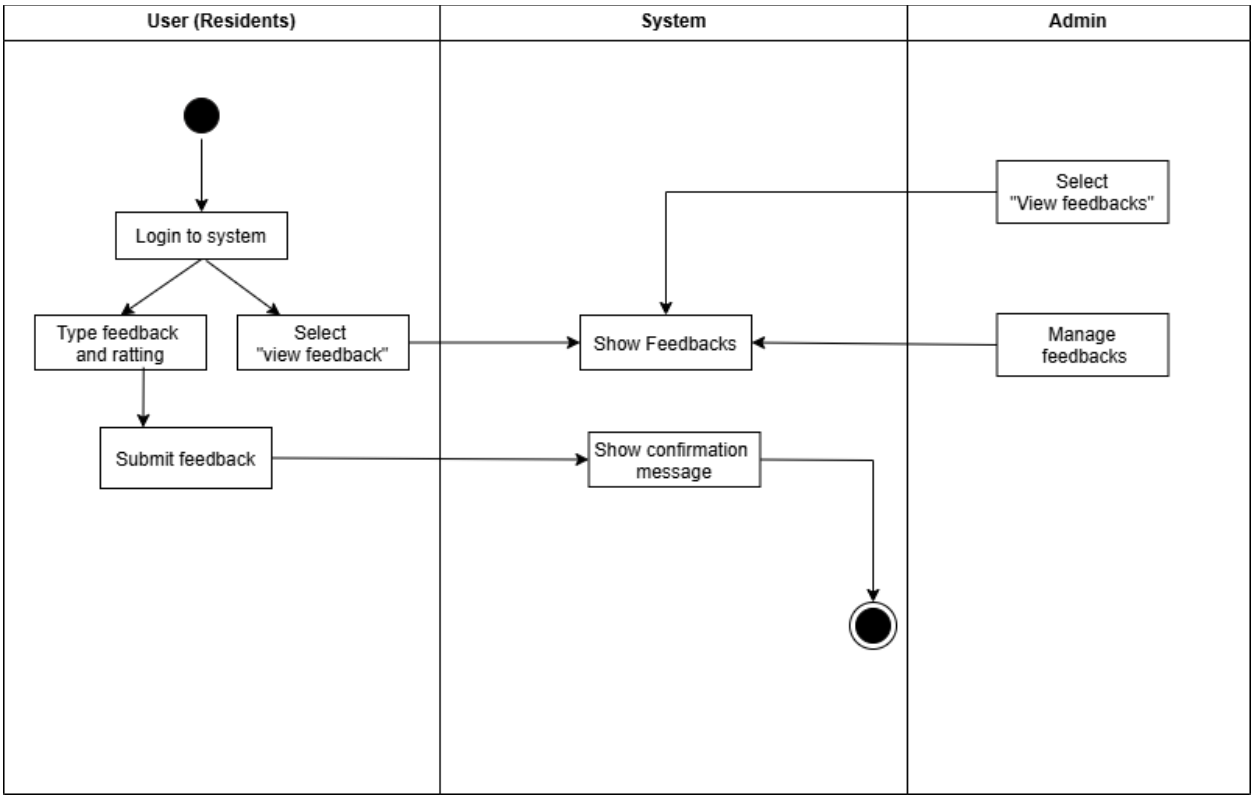


Figure 45: Feedback activity diagram

Complaint

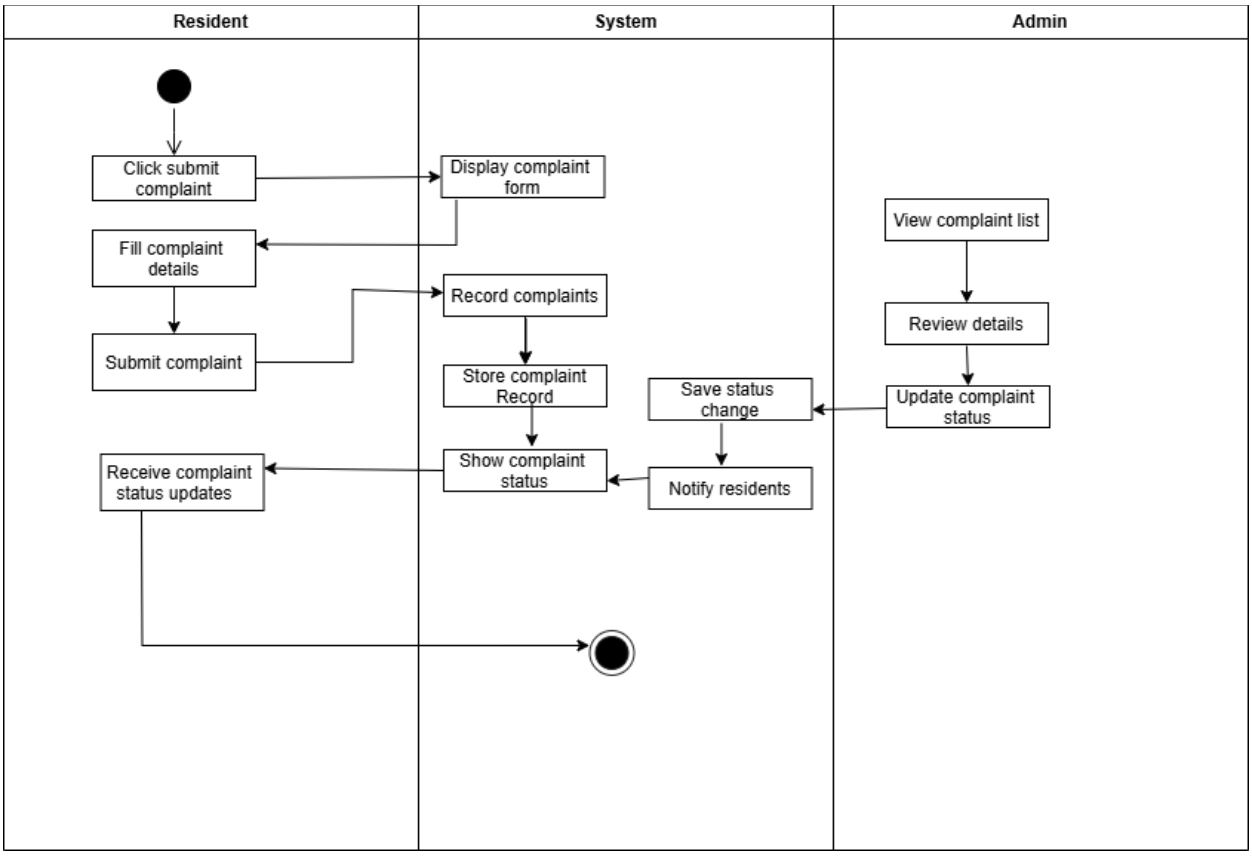


Figure 46: Complaint activity diagram

3.7 Summary

In this chapter, the system requirements and design methodology of the CleanLand Garbage Management System were discussed. The functional and non-functional requirements were identified to ensure that the system meets stakeholder needs while maintaining security, usability, and performance. The software, hardware, and database components of the system were also described, along with the technologies used for development, including React.js, Node.js, MySQL, Google Maps API, and QR code integration. Finally, the proposed Agile methodology and system design provided a clear foundation for the implementation of the CleanLand Garbage Management System.

CHAPTER 04

4. System Implementation

4.1 Introduction

The System Implementation chapter marks the transition from system design to the actual development of the CleanLand Garbage Management System. During this phase, the proposed design was transformed into a fully functional web based application that supports garbage collection management, complaint handling, collection scheduling, QR code based bin management, feedback management, notification services, and truck monitoring.

This chapter describes the implementation of the system components, including frontend development, backend development, database configuration, and the integration of supporting technologies such as Google Maps and QR code services. It also explains how the various modules were developed and integrated to provide a complete garbage management solution.

4.2 Code Features

The CleanLand Garbage Management System follows a three tier architecture consisting of the presentation layer, application layer, and database layer.

The presentation layer is implemented using React.js and provides user interfaces for residents, drivers, and administrators. Through this layer, users can register, log in, submit complaints, view collection schedules, manage feedback, access awareness content, and monitor garbage collection activities.

The application layer is developed using Node.js and Express.js. It handles business logic, user authentication, complaint processing, schedule management, QR code generation, notification services, and communication between the frontend and database.

The database layer utilizes MySQL to store and manage system data, including user accounts, complaints, schedules, garbage bin information, feedback records, notifications, and truck monitoring data. This architecture ensures scalability, maintainability, and efficient system performance.

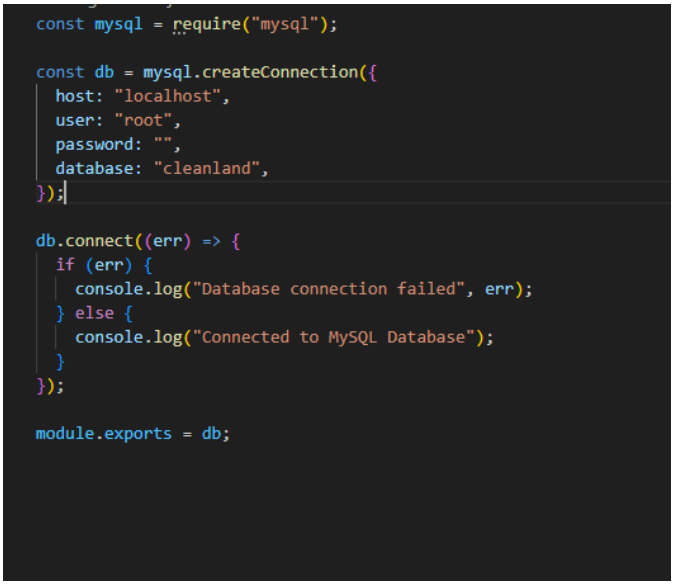
Communication between these layers enables seamless interaction, allowing users to perform system operations while ensuring secure and reliable data processing.

4.3 Database Connection

The CleanLand Garbage Management System utilizes MySQL as its database management system. MySQL provides reliable storage, retrieval, and management of all system data.

The backend server developed using Node.js and Express.js establishes a connection with the MySQL database and manages all database operations. This connection enables the system to perform user authentication, complaint management, schedule management, feedback processing, QR code management, and notification handling efficiently.

The database connection layer acts as the bridge between the frontend application and the database, ensuring secure and reliable communication between system components



```
const mysql = require("mysql");

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "cleanland",
});

db.connect((err) => {
  if (err) {
    console.log("Database connection failed", err);
  } else {
    console.log("Connected to MySQL Database");
  }
});

module.exports = db;
```

Figure 47: Database connection

4.4 Backend Server Implementation

```
const express = require("express");
const cors = require("cors");
const path = require("path");

const app = express();
app.use(cors());
app.use(express.json());

app.use("/uploads", express.static(path.join(__dirname, "uploads")));

app.use("/", require("./routes/authRoutes"));
app.use("/", require("./routes/driverRoutes"));
app.use("/", require("./routes/residentRoutes"));
app.use("/", require("./routes/truckRoutes"));
app.use("/", require("./routes/routeRoutes"));
app.use("/", require("./routes/binRoutes"));
app.use("/", require("./routes/qrRoutes"));
app.use("/", require("./routes/scheduleRoutes"));
app.use("/", require("./routes/awarenessRoutes"));
app.use("/", require("./routes/complaintRoutes"));
app.use("/", require("./routes/feedbackRoutes"));
app.use("/", require("./routes/specialRequestRoutes"));
app.use("/", require("./routes/dashboardRoute"));
app.use("/", require("./routes/emergencyAlertRoutes"));


app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Figure 48: Backend server implementation

4.5 Frontend Implementation

```
import 'bootstrap/dist/css/bootstrap.min.css';
import SignUp from './Signup_Page/signup';
import Home from './Home_Page/home';
import { BrowserRouter, Routes, Route } from "react-router-dom";
import ResidentLogin from './Login_Page/login';
import ResidentDashboard from './Resident_dashboard/resident_dashboard';
import LiveTracking from './track_page/track';
import CollectionSchedule from './collection_schedule_page/collection_schedule';
import ComplaintManagement from './Complaints_Page/complaints';
import Scanner from './Scanner_Page/scanner';
import Feedback from './Feedback_Page/Feedback';
import AdminDashboard from './Admin_Dashboard_Page/Admin_Dashboard';
import Drivr_dashboard from './Driver_Dashboard_Page/driver_dashboard';
import AddDriver from './Add_Driver_Page/add_driver';
import ManageDrivers from './Driver_Managment_page/driver_managment';
import ManageResidents from './Resident_Managment_Page/resident_managment';
import AddTruck from './Add_Truck_Page/add_truck';
import ManageTruck from './Truck_Management_Page/truck_management';
import AddRoute from './Add_Route_Page/add_route';
import ManageRoutes from './Route_Managment/route_managment';
import AddBin from './Add_Bin_Page/add_bin';
import ManageBin from './Bin_Managment_Page/bin_managment';
import AddQRCode from './Add_QR_Code_Page/add_qr_code';
import ManageQRCode from './QR_Code_Management_Page/qr_code_manage';
import AddSchedule from './Add_Schedule_Page/add_schedule';
import ManageSchedules from './Schedule_Managment_Page/schedule_management';
import AddContent from './Add_Content_Page/add_content';
import ManageContent from './Content_Management_Page/content_managment';
import AdminFeedback from './Feedback_Management_Page/feedback_management';
import ComplaintManagementAdmin from './Complaint_Managment_Page/complaint_managment';
import SpecialRequest from './Special_Pickup_Request_Page/pickup_request';
import AdminSpecialRequests from './Pickup_Request_Managment_Page/pickup_managment';
```

Figure 49: Frontend implementation

```
import AddEmergencyAlert from './Add_Emergency_Alert/add_emergency_alert';

function App() {
  return (
    <BrowserRouter>
      <Routes>
      <Route path="/home" element={<Home />} />
        <Route path="/" element={<SignUp />} />
        <Route path="/login" element={<ResidentLogin/>} />
        <Route path="/resident_dashboard" element={<ResidentDashboard/>} />
          <Route path="/track" element={<LiveTracking/>} />
        <Route path="/collection_schedule" element={<CollectionSchedule/>} />
        <Route path="/complaints" element={<ComplaintManagement/>} />
        <Route path="/scanner" element={<Scanner/>} />
        <Route path="/Feedback" element={<Feedback/>} />
        <Route path="/Admin_Dashboard" element={<AdminDashboard/>} />
        <Route path="/driver_dashboard" element={<Drivr_dashboard/>} />
        <Route path="/add_driver" element= {<AddDriver/>} />
       <Route path="/driver_managment" element= {<ManageDrivers/>} />
       <Route path="/resident_managment" element= {<ManageResidents/>} />
       <Route path="/add_truck" element= {<AddTruck/>} />
       <Route path="/truck_management" element= {<ManageTruck/>} />
       <Route path="/add_route" element= {<AddRoute/>} />
       <Route path="/route_managment" element= {<ManageRoutes/>} />
       <Route path="/add_bin" element= {<AddBin/>} />
       <Route path="/bin_managment" element= {<ManageBin/>} />
       <Route path="/add_qr_code" element= {<AddQRCode/>} />
       <Route path="/qr_code_manage" element= {<ManageQRCode/>} />
       <Route path="/add_schedule" element= {<AddSchedule/>} />
       <Route path="/schedule_management" element= {<ManageSchedules/>} />
       <Route path="/add_content" element= {<AddContent/>} />
```

Figure 50: Frontend implementation

```
          <Route path="/add_content" element= {<AddContent/>} />
          <Route path="/content_managment" element= {<ManageContent/>} />
          <Route path="/feedback_management" element= {<AdminFeedback/>} />
          <Route path="/complaint_managment" element= {<ComplaintManagementAdmin/>} />
          <Route path="/pickup_request" element= {<SpecialRequest/>} />
          <Route path="/pickup_managment" element= {<AdminSpecialRequests/>} />
          <Route path="/add_emergency_alert" element= {<AddEmergencyAlert/>} />
        </Routes>
      </BrowserRouter>
    );}
  export default App;
```

Figure 51: Frontend implementation

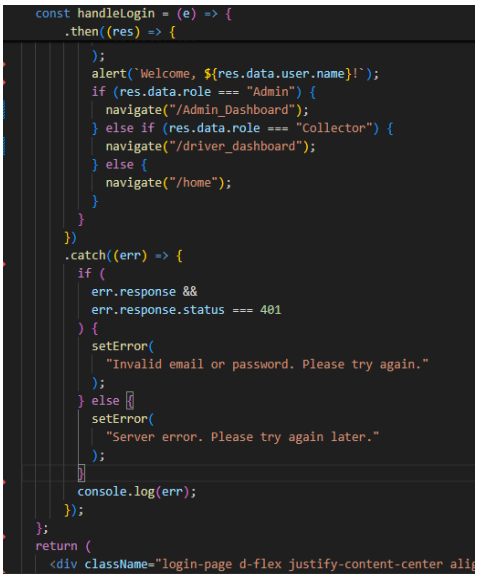
4.6 Login



```
import React, { useState } from "react";
import "./login.css";
import { Link, useNavigate } from "react-router-dom";
import axios from "axios";

function ResidentLogin() {
  const navigate = useNavigate();
  const [formData, setFormData] = useState({
    email: "",
    password: "",
    role: "Resident"
  });
  const [error, setError] = useState("");
  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
    setError("");
  };
  const handleLogin = (e) => {
    e.preventDefault();
    axios.post("http://localhost:5000/login", formData)
      .then((res) => {
        if (res.data.success) {
          localStorage.setItem(
            "user",
            JSON.stringify(res.data.user)
          );
          localStorage.setItem(
            "role",
            res.data.role
```

Figure 52: Login page 1



```
  const handleLogin = (e) => {
      .then((res) => {
          );
          alert(`Welcome, ${res.data.user.name}!`);
          if (res.data.role === "Admin") {
            navigate("/Admin_Dashboard");
          } else if (res.data.role === "Collector") {
            navigate("/driver_dashboard");
          } else {
            navigate("/home");
          }
        }
      })
      .catch((err) => {
        if (
          err.response &&
          err.response.status === 401
        ) {
          setError(
            "Invalid email or password. Please try again."
          );
        } else {
          setError(
            "Server error. Please try again later."
          );
        }
        console.log(err);
      });
  };
  return (
    <div className="login-page d-flex justify-content-center ali
```

Figure 53: Login page 2



```
function ResidentLogin() {
  return (
    <div className="login-page d-flex justify-content-center align-items-center">
      <div className="login-card shadow">
        <h2 className="text-center title">
          CleanLand
        </h2>
        <p className="text-center subtitle">
          Garbage Management System
        </p>
        <form onSubmit={handleLogin}>
          <div className="mb-3">
            <label className="form-label">
              Email Address
            </label>
            <input
              type="email"
              name="email"
              className="form-control"
              placeholder="resident@cleanland.lk"
              onChange={handleChange}
              value={formData.email}
              required
            />
          </div>
          <div className="mb-3">
            <label className="form-label">
              Password
            </label>
            <input
              type="password"
              name="password"
```

Figure 54: Login page 3



```
              className="form-control"
              placeholder="******"
              onChange={handleChange}
              value={formData.password}
              required
            />
          </div>
          <div className="mb-3 text-center">
            <label className="form-label-center">
              Login As
            </label>
  <br/>
            <select
              name="role"
              className="form-select"
              onChange={handleChange}
              value={formData.role}
            >
              <option>Resident</option>
              <option>Admin</option>
              <option>Collector</option>
            </select>
          </div>
          {error && (
            <div
              className="alert alert-danger py-2"
              role="alert"
            >
              {error}
            </div>
          )}
```

Figure 55: Login page

```
                <option>Collector</option>
              </select>
            </div>
            {error && (
              <div
                className="alert alert-danger py-2"
                role="alert"
              >
                {error}
              </div>
            )}
            <button
              type="submit"
              className="btn login-btn w-100 mb-2"
            >
              LOGIN
            </button>
            <p className="mt-2 text-center">
              Don't have an account?
              <Link to="/">
                Sign Up
              </Link>
            </p>
          </form>
        </div>
      </div>
    );
}
export default ResidentLogin;
```

Figure 56: Login page

Login backend

```
//login
const login = (req, res) => {
  const { email, password, role } = req.body;

  const roleMap = {
    Resident: { table: "resident", idField: "resident_ID" },
    Admin:    { table: "admin",    idField: "admin_ID"    },
    Collector:{ table: "driver",   idField: "driver_ID"   },
  };

  if (!roleMap[role])
    return res.status(400).json({ success: false, message: "Invalid role" });

  const { table, idField } = roleMap[role];

  db.query(`SELECT * FROM ${table} WHERE email = ? AND password = ?`, [email, password], (err, result) => {
    if (err) return res.status(500).json({ success: false, message: "Database error" });
    if (result.length > 0) {
      const user = result[0];
      return res.json({
        success: true,
        message: "Login successful",
        role,
        user: { id: user[idField], name: user.name, email: user.email },
      });
    }
    res.status(401).json({ success: false, message: "Invalid email or password" });
  });
};

module.exports = { signup, login };
```

Figure 57: Login page backend

4.7 SignUp

```jsx
import React, { useState } from "react";
import "./Signup.css";
import axios from "axios";
import { Link, useNavigate } from "react-router-dom";

function SignUp() {
  const navigate = useNavigate();
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    phone_no: "",
    address: "",
    password: ""
  });
  const [error, setError] = useState("");
  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
    setError("");
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    if (
      !formData.name ||
      !formData.email ||
      !formData.phone_no ||
      !formData.address ||
      !formData.password
    ) {
      setError("All fields are required");
```

Figure 58: Signup page 1

```jsx
const handleSubmit = (e) => {
  e.preventDefault();
  if (formData.phone_no.length < 10) {
    setError("Phone number must be at least 10 digits.");
    return;
  }
  if (formData.password.length < 6) {
    setError("Password must be at least 6 characters.");
    return;
  }
  axios
    .post("http://localhost:5000/signup", formData)
    .then((res) => {
      alert("Registration Successful");
      navigate("/login");
    })
    .catch((err) => {
      if (err.response && err.response.data) {
        setError(err.response.data);
      } else {
        setError("Error occurred. Please try again.");
      }
      console.log(err);
    });
};
return (
  <div className="register-container d-flex justify-content-center align-items-center">
    <div className="register-card shadow-lg">
      <h2 className="text-center title">
        CleanLand
      </h2>
      <p className="text-center subtitle">
```

Figure 59: Signup page 2

```
{error && (
  <div className="alert alert-danger text-center py-2">
    {error}
  </div>
)}
<form onSubmit={handleSubmit}>
  <div className="mb-3">
    <label className="form-label">
      Full Name
    </label>
    <input
      type="text"
      name="name"
      className="form-control"
      placeholder="Enter your full name"
      onChange={handleChange}
      value={formData.name}
      required
    />
  </div>
  <div className="mb-3">
    <label className="form-label">
      Email
    </label>
    <input
      type="email"
      name="email"
      className="form-control"
      placeholder="Enter your email"
      onChange={handleChange}
      value={formData.email}
```

Figure 60: Signup page 3

```
      required
    />
  </div>
  <div className="mb-3">
    <label className="form-label">
      Phone
    </label>
    <input
      type="text"
      name="phone_no"
      className="form-control"
      placeholder="Enter your phone number"
      onChange={handleChange}
      value={formData.phone_no}
      required
    />
  </div>
  <div className="mb-3">
    <label className="form-label">
      Address
    </label>
    <input
      type="text"
      name="address"
      className="form-control"
      placeholder="Enter your address"
      onChange={handleChange}
      value={formData.address}
      required
    />
  </div>
```

Figure 61: Signup page 4

```jsx
            <div className="mb-4">
              <label className="form-label">
                Password
              </label>
              <input
                type="password"
                name="password"
                className="form-control"
                placeholder="Minimum 6 characters"
                onChange={handleChange}
                value={formData.password}
                required          />
            </div>
            <div className="d-grid mb-3">
              <button
                type="submit"
                className="btn register-btn">
                Register
              </button>
            </div>
            <p className="text-center">
              Already have an account? <Link to="/login">Login</Link>
            </p>
          </form>
        </div>
      </div>
    );
  }
  export default SignUp;
```

Figure 62: Signup page 5

Backend

```jsx
const db = require("../config/db");

//signup
const signup = (req, res) => {
  const { name, email, phone_no, address, password } = req.body;

  if (!name || !email || !phone_no || !address || !password)
    return res.status(400).send("All fields are required.");

  if ([name, email, phone_no, address, password].some((f) => f.trim() === ""))
    return res.status(400).send("Fields cannot be empty or blank.");

  if (phone_no.length < 10)
    return res.status(400).send("Phone number must be at least 10 digits.");

  if (password.length < 6)
    return res.status(400).send("Password must be at least 6 characters.");

  db.query("SELECT * FROM resident WHERE email = ?", [email], (err, result) => {
    if (err) return res.status(500).send("Database error.");
    if (result.length > 0)
      return res.status(400).send("Email already registered. Please login.");

    const sql = `INSERT INTO resident (name, email, phone_no, address, password) VALUES (?, ?, ?, ?, ?)`;
    db.query(sql, [name.trim(), email.trim(), phone_no.trim(), address.trim(), password], (err) => {
      if (err) return res.status(500).send("Error inserting data.");
      res.send("Registration successful");
    });
  });
};
```

Figure 63: Signup page backend

4.8 Complaint Page

```jsx
import React, { useEffect, useRef, useState } from "react";
import "./complaints.css";
import NavBar from "../Navigation_Bar_Page/Navigation";
import Footer from "../Footer_Page/Footer";

const API_BASE = "http://localhost:5000";
const getResidentID = () => {
  try {
    const user = JSON.parse(localStorage.getItem("user") || "{}");
    return user.id || null;
  } catch {
    return null;
  }};
  const StatusBadge = ({ status }) => {
    const cls =
      status === "pending" ? "complaint_pending"
      : status === "in progress" ? "complaint_progress"
      : status === "completed" ? "complaint_completed"
      : "";
    return <span className={`complaint_status ${cls}`}>{status}</span>;
  };
const ComplaintManagement = () => {
  const residentID = getResidentID();
  const [showForm, setShowForm]     =useState(false);
  const [complaintType, setComplaintType] = useState("missed pickup");
  const [location, setLocation] = useState("");
  const [description, setDescription] = useState("");
  const [date, setDate] = useState(new Date().toISOString().split("T")[0]);
  const [imageFile, setImageFile] = useState(null);
  const [imagePreview, setImagePreview] = useState(null);
  const fileInputRef    = useRef(null);
  const [complaints, setComplaints] = useState([]);
```

Figure 64: complaint page 1

```jsx
  const [complaints, setComplaints] = useState([]);
  const [loading, setLoading] = useState(false);
  const [submitting, setSubmitting] = useState(false);
  const [error, setError]     =useState("");
  const [successMsg, setSuccessMsg] =useState("");

    useEffect(() => {
      if (residentID) fetchComplaints();
    }, []);
  const fetchComplaints = async () => {
    setLoading(true);
    try {
      const res = await fetch (`${API_BASE}/get-complaints/${residentID}`);
      const data = await res.json();
      if (data.success) setComplaints(data.complaints);
    } catch {
      setError("Failed to load complaints. Please try again.");
    } finally {
      setLoading(false);
    }};
  const handleImageChange = (e) => {
    const file = e.target.files [0];
    if (!file) return;
    setImageFile(file);
    setImagePreview(URL.createObjectURL(file));
  };
  const resetForm = () => {
    setComplaintType("missed pickup");
    setLocation("");
    setDescription("");
```

Figure 65: Complaint page 2

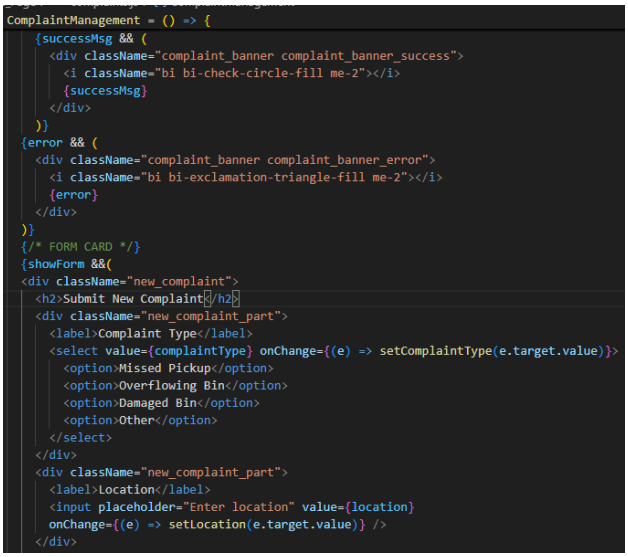
```
const ComplaintManagement = () => {
  const resetForm = () => {
    setDate(new Date().toISOString().split("T")[0]);
    setImageFile(null);
    setImagePreview(null);
    setError("");
  };
  const handleSubmit = async () => {
    setError("");
    setSuccessMsg("");
    if (!residentID) {
      setError("You must be logged in as a resident to submit a complaint.");
      return;
    }
    if (!location.trim()) {
      setError("Please enter a location.");
      return;
    }
    const formData = new FormData();
    formData.append("complaint_type", complaintType);
    formData.append("description", description);
    formData.append("location", location);
    formData.append("date", date);
    formData.append("resident_ID", residentID);
    if (imageFile) formData.append("image", imageFile);
    setSubmitting(true);
    try {
      const res = await fetch (`${API_BASE}/submit-complaint`, {method: "POST", body: formData});
      const data = await res.json();
      if (data.success) {
        setSuccessMsg(`Complaint ${data.complaint_ID} submitted successfully!`);
        resetForm();
```

Figure 66: Complaint page 3

```
const ComplaintManagement = () => {
  const handleSubmit = async () => {
        setShowForm(false);
        fetchComplaints();
      } else {
        setError(data.message || "Failed to submit complaint.");
      }
    } catch {
      setError("Network error. Please check your connection.");
    } finally {
      setSubmitting(false);
    }
  };

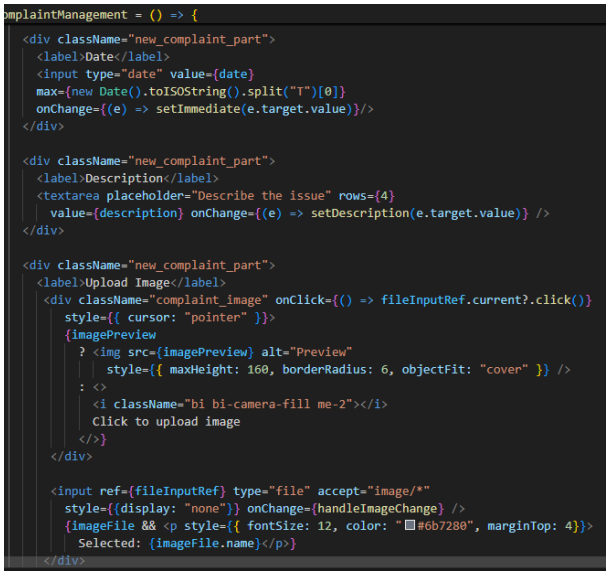
  return (
    <div className="complaint_page">
      <NavBar />
      <div className="complaint_page_container">
        {/* Header */}
        <div className="complaint_page_header">
          <div>
            <h1>
              <i className="bi bi-chat-dots-fill me-2"></i>
              Complaint Management
            </h1>
            <p>Submit and track your complaints</p>
          </div>
          <button className="submit_button" onClick={() => {setShowForm(!showForm); setError(""); setSuccessMsg("");}}>
            <>
              <i className={`bi ${showForm ? "bi-x-lg" : "bi-plus-lg"} me-2`}></i>
              {showForm ? "Close Form" : "Submit New Complaint"}
```

Figure 67: Complaint page 4



```
ComplaintManagement = () => {
    {successMsg && (
      <div className="complaint_banner complaint_banner_success">
        <i className="bi bi-check-circle-fill me-2"></i>
        {successMsg}
      </div>
    )}
  {error && (
    <div className="complaint_banner complaint_banner_error">
      <i className="bi bi-exclamation-triangle-fill me-2"></i>
      {error}
    </div>
  )}
  {/* FORM CARD */}
  {showForm &&(
  <div className="new_complaint">
    <h2>Submit New Complaint</h2>
    <div className="new_complaint_part">
      <label>Complaint Type</label>
      <select value={complaintType} onChange={(e) => setComplaintType(e.target.value)}>
        <option>Missed Pickup</option>
        <option>Overflowing Bin</option>
        <option>Damaged Bin</option>
        <option>Other</option>
      </select>
    </div>
    <div className="new_complaint_part">
      <label>Location</label>
      <input placeholder="Enter location" value={location}
      onChange={(e) => setLocation(e.target.value)} />
    </div>
```

Figure 68: Complaint page 5



```
mplaintManagement = () => {
  <div className="new_complaint_part">
    <label>Date</label>
    <input type="date" value={date}
    max={new Date().toISOString().split("T")[0]}
    onChange={(e) => setImmediate(e.target.value)}/>
  </div>

  <div className="new_complaint_part">
    <label>Description</label>
    <textarea placeholder="Describe the issue" rows={4}
      value={description} onChange={(e) => setDescription(e.target.value)} />
  </div>

  <div className="new_complaint_part">
    <label>Upload Image</label>
     <div className="complaint_image" onClick={() => fileInputRef.current?.click()}
        style={{ cursor: "pointer" }}>
        {imagePreview
          ? <img src={imagePreview} alt="Preview"
              style={{ maxHeight: 160, borderRadius: 6, objectFit: "cover" }} />
          : <>
            <i className="bi bi-camera-fill me-2"></i>
            Click to upload image
          </>}
      </div>

      <input ref={fileInputRef} type="file" accept="image/*"
        style={{display: "none"}} onChange={handleImageChange} />
        {imageFile && <p style={{ fontSize: 12, color: "#6b7280", marginTop: 4}}>
          Selected: {imageFile.name}</p>}
      </div>
```

Figure 69: Complaint page 6

```
    <div style={{ display: "flex", gap: 10, marginTop: 8 }}>
      <button className="submit_button" onClick={handleSubmit} disabled={submitting}>
        {submitting ? "Submitting..." : "Submit"}
      </button>
      <button className="complaint-button-outline" disabled={submitting}
        onClick={() => { resetForm(); setShowForm(false); }}>
        Cancel
      </button>
    </div>
  </div>
  )}
  {/* TABLE CARD (NOW GUARANTEED BELOW) */}
  <div className="complaints">
    <h2>My Complaints</h2>
  {loading ? (<p className="complaint_loading">Loading complaints...</p>
  ) : complaints.length === 0 ? (
    <p className="complaint_empty">No complaints found. Submit one above!</p>
  ) : (
    <table className="complaint_table">
      <thead>
        <tr>
          <th>ID</th>
          <th>Type</th>
          <th>Location</th>
          <th>Date</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody>
        {complaints.map((c) =>(
        <tr key={c.complaint_ID}>
```

Figure 70: Complaint page 7

```
            </thead>
            <tbody>
              {complaints.map((c) =>(
              <tr key={c.complaint_ID}>
                <td>{c.complaint_ID}</td>
                <td className="complaint_type_cell">{c.complaint_type}</td>
                <td>{c.location}</td>
                <td>{c.date ? c.date.split("T")[0] : "-"}</td>
                <td>
                 <span className={`complaint_status ${
                c.status === "pending" ? "complaint_pending"
                : c.status === "in progress" ? "complaint_progress"
                : c.status === "completed" ? "complaint_completed"
                : ""
              }`}>
                {c.status}
              </span>
              </td>
              </tr>
              ))}
            </tbody>
          </table>
        )}
        </div>
      </div>
      <Footer/>
    </div>
  );
};
export default ComplaintManagement;
```

Figure 71: Complaint page 8

Complaint Management



```
import axios from "axios";
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "./complaint_managment.css";
import "../extra_CSS_file.css";
import AdminNavBar from "../Admin_Navigation_Bar/admin_navigation";

function ComplaintManagementAdmin(){
    const navigate = useNavigate();
    const [complaints, setComplaints] = useState([]);
    const [search, setSearch] = useState("");
    const[loading, setLoading] =useState(true);
    const [error, setError] = useState("");
    const [success, setSuccess] = useState("");
    const [showModal, setShowModal] =useState(false);
    const [selectedComplaint, setSelectedComplaint] = useState(null);
    const [editStatus, setEditStatus] = useState("pending");
    const [showDeleteModal, setShowDeleteModal] = useState(false);
    const [deleteTarget, setDeleteTarget] = useState(null);
    const [ showViewModal, setShowViewModal] = useState(false);
    const [viewTarget, setViewTarget] = useState(null);
    const fetchComplaints = () => {
     setLoading(true);
     axios.get("http://localhost:5000/get-all-complaints")
        .then((res) => { setComplaints(res.data.complaints || res.data); setLoading(false); })
        .catch(() => { setError("Failed to load complaints"); setLoading(false); });
    };
    useEffect(() => {
        fetchComplaints();
        }, []);
    const filteredComplaints = complaints.filter((c) => {
```

Figure 72:Complaint management page 1



```
    const q = search.toLowerCase();
    return (
        (c.complaint_ID ||"").toString().toLowerCase().includes(q) ||
        (c.complaint_type || "").toLowerCase().includes(q) ||
        (c.status || "").toLowerCase().includes(q) ||
        (c.location || "").toLowerCase().includes(q) ||
        (c.resident_ID || "").toString().toLowerCase().includes(q)
    );
});
const getStatusClass= (status) => {
    if (!status) return "";
    if (status === "pending") return "cm_badge_pending";
    if (status === "in progress") return "cm_badge_progress";
    if (status === "completed") return "cm_badge_completed";
    return "";
};
const openEdit = (complaint) => {
    setSelectedComplaint(complaint);
    setEditStatus(complaint.status || "pending");
    setShowModal(true);
    setError("");
    setSuccess("");
};
const handleUpdate = () => {
    axios.put(`http://localhost:5000/update-complaint-status/${selectedComplaint.complaint_ID}`, {
  status: editStatus,
})
    .then(() => {
        setSuccess(`Complaint #${selectedComplaint.complaint_ID} updated to "${editStatus}" successfully!`);
        setShowModal(false);
```

Figure 73: Complaint management page 2



```
  const handleUpdate = () => {
    axios.put( http://localhost:5000/update-complaint-status/${selectedComplaint.complaint_ID} , {
    status: editStatus,
  })
      .then(() => {
          setSuccess(`Complaint #${selectedComplaint.complaint_ID} updated to "${editStatus}" successfully!`);
          setShowModal(false);
          fetchComplaints();
      })
      .catch((err) => setError(err.response?.data?.message || "Error updating complaint."));
};
    const openDelete = (complaint) => {
      setDeleteTarget(complaint);
      setShowDeleteModal(true);
    };
  const handleDelete = () => {
      axios.delete(`http://localhost:5000/delete-complaint/${deleteTarget.complaint_ID}`)
          .then(() => {
              setSuccess(`Complaint #${deleteTarget.complaint_ID} deleted successfully!`);
              setShowDeleteModal(false);
              fetchComplaints();
          })
          .catch(() => { setError("Error deleting complaint." );
              setShowDeleteModal(false);
          });};
  const openView = (complaint) => {
      setViewTarget(complaint);
      setShowViewModal(true);
  };
  const formatDate = (d) => (d ? d.split("T")[0] : "—");
```

Figure 74: Complaint management page 3

Complaint Backend

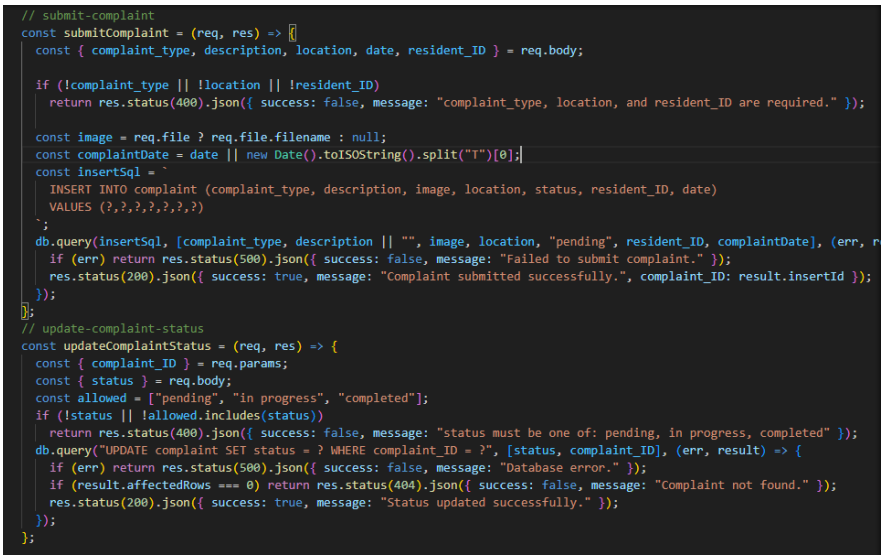


```
const db = require("../config/db");
const path = require("path");
const fs = require("fs");

// GET /get-complaints/:resident_ID
const getComplaintsByResident = (req, res) => {
  const { resident_ID } = req.params;
  const sql = `
    SELECT complaint_ID, resident_ID, complaint_type, description, image, location, status, date
    FROM complaint WHERE resident_ID = ? ORDER BY date DESC
  `;
  db.query(sql, [resident_ID], (err, results) => {
    if (err) return res.status(500).json({ success: false, message: "Database error." });
    res.json({ success: true, complaints: results });
  });
};

//get-all-complaints
const getAllComplaints = (req, res) => {
  const sql = `
    SELECT c.complaint_ID, c.resident_ID, c.complaint_type, c.description, c.image,
           c.location, c.status, c.date, r.name AS resident_name, r.email AS resident_email
    FROM complaint c
    LEFT JOIN resident r ON c.resident_ID = r.resident_ID
    ORDER BY c.date DESC
  `;
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ success: false, message: "Database error." });
    res.json({ success: true, complaints: results });
  });
};
```

Figure 75: Complaint backend



```
// submit-complaint
const submitComplaint = (req, res) => {
  const { complaint_type, description, location, date, resident_ID } = req.body;

  if (!complaint_type || !location || !resident_ID)
    return res.status(400).json({ success: false, message: "complaint_type, location, and resident_ID are required." });

  const image = req.file ? req.file.filename : null;
  const complaintDate = date || new Date().toISOString().split("T")[0];
  const insertSql = `
    INSERT INTO complaint (complaint_type, description, image, location, status, resident_ID, date)
    VALUES (?,?,?,?,?,?,?)
  `;
  db.query(insertSql, [complaint_type, description || "", image, location, "pending", resident_ID, complaintDate], (err, r
    if (err) return res.status(500).json({ success: false, message: "Failed to submit complaint." });
    res.status(200).json({ success: true, message: "Complaint submitted successfully.", complaint_ID: result.insertId });
  });
};
// update-complaint-status
const updateComplaintStatus = (req, res) => {
  const { complaint_ID } = req.params;
  const { status } = req.body;
  const allowed = ["pending", "in progress", "completed"];
  if (!status || !allowed.includes(status))
    return res.status(400).json({ success: false, message: "status must be one of: pending, in progress, completed" });
  db.query("UPDATE complaint SET status = ? WHERE complaint_ID = ?", [status, complaint_ID], (err, result) => {
    if (err) return res.status(500).json({ success: false, message: "Database error." });
    if (result.affectedRows === 0) return res.status(404).json({ success: false, message: "Complaint not found." });
    res.status(200).json({ success: true, message: "Status updated successfully." });
  });
};
```

Figure 76: Complaint backend



```
// delete complaint
const deleteComplaint = (req, res) => {
  const { complaint_ID } = req.params;
  db.query("SELECT image FROM complaint WHERE complaint_ID = ?", [complaint_ID], (err, rows) => {
    if (!err && rows.length > 0 && rows[0].image) {
      const imgPath = path.join(__dirname, "../uploads", rows[0].image);
      if (fs.existsSync(imgPath)) fs.unlinkSync(imgPath);
    }
    db.query("DELETE FROM complaint WHERE complaint_ID = ?", [complaint_ID], (err, result) => {
      if (err) return res.status(500).json({ success: false, message: "Error deleting complaint." });
      if (result.affectedRows === 0) return res.status(404).json({ success: false, message: "Complaint not found." });
      res.status(200).json({ success: true, message: "Complaint deleted successfully." });
    });
  });
};
// get recent complaints for admin dashboard
const getRecentComplaints = (req, res) => {
  const sql = `
    SELECT c.complaint_ID, c.complaint_type, c.status, c.date,
           r.name AS resident_name
    FROM complaint c
    LEFT JOIN resident r ON c.resident_ID = r.resident_ID
    WHERE c.status IN ('pending', 'in progress')
    ORDER BY c.date DESC
  `;
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ success: false, message: "Database error." });
    res.json({ success: true, complaints: results });
  });
};
module.exports = { getComplaintsByResident, getAllComplaints, submitComplaint, updateComplaintStatus, deleteComplaint, get
```

Figure 77: Complaint backend

4.9 Admin Dashboard

```jsx
import React, { useState, useEffect } from "react";
import "./Admin_Dashboard.css";
import { useNavigate } from "react-router-dom";
import AdminNavBar from "../Admin_Navigation_Bar/admin_navigation";

function StatBox({ value, label, color }) {
  return (
    <div className={`stat ${color}`}>
      <h2>{value ?? "..."}</h2>
      <p>{label}</p>
    </div>
  );
}

function AdminDashboard() {
  const navigate = useNavigate();
  const [stats, setStats] = useState({
    residents: null,
    truckCount: null,
    pendingComplaints: null,
    bins: null,
  });
  const [recentComplaints, setRecentComplaints] = useState([]);
  const [trucks, setTrucks] = useState([]);

  useEffect(() => {
    // Stats
    fetch("http://localhost:5000/get-dashboard-stats")
      .then((res) => res.json())
      .then((data) => setStats(data))
      .catch((err) => console.error("Failed to load stats:", err));
```

Figure 78: Admin dashboard 1

```jsx
    // Recent complaints
    fetch("http://localhost:5000/get-recent-complaints")
      .then((res) => res.json())
      .then((data) => {
        if (data.success) setRecentComplaints(data.complaints);
      })
      .catch((err) => console.error("Failed to load complaints:", err));
    // Trucks
    fetch("http://localhost:5000/get-trucks-with-driver")
      .then((res) => res.json())
      .then((data) => {
        console.log("Trucks loaded:", data);
        setTrucks(data);
      })
      .catch((err) => console.error("Failed to load trucks:", err));
  }, []);
  const getStatusClass = (status) => {
    if (status === "pending") return "pending";
    if (status === "in progress") return "progress";
    return "";
  };
  return (
    <div>
      <AdminNavBar />
      <div className="admin_page">

        <div className="admin_page_explain">
          <h1>Admin Dashboard</h1>
          <p>System overview and management</p>
        </div>
```

Figure 79: Admin dashboard 2

```
{/* Stats */}
<div className="stats_count">
  <StatBox value={stats.residents}         label="Total Residents"    color="green"  />
  <StatBox value={stats.trucks}            label="Active Trucks"      color="orange" />
  <StatBox value={stats.pendingComplaints} label="Pending Complaints" color="blue"   />
  <StatBox value={stats.bins}              label="Bins Managed"       color="teal"   />
</div>
{/* Main Section */}
<div className="main_section">
  {/* Recent Complaints */}
  <div className="complaint_truck_section">
    <h3>Recent Complaints</h3>
    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Type</th>
          <th>Resident</th>
          <th>Status</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {recentComplaints.length === 0 ? (
          <tr>
            <td colSpan="5" style={{ textAlign: "center" }}>
              No pending complaints
            </td>
          </tr>
        ) : (
          recentComplaints.map((c) => (
```

Figure 80: Admin dashboard 3

```
        ) : (
          recentComplaints.map((c) => (
            <tr key={c.complaint_ID}>
              <td>{c.complaint_ID}</td>
              <td>{c.complaint_type}</td>
              <td>{c.resident_name}</td>
              <td>
                <span className={`status ${getStatusClass(c.status)}`}>
                  {c.status}
                </span>
              </td>
              <td>
                <button
                  className="btn view"
                  onClick={() => navigate("/complaint_managment")}
                >
                  View
                </button>
              </td>
            </tr>
          ))
        )}
      </tbody>
    </table>
  </div>
  {/* Truck Status */}
  <div className="complaint_truck_section">
    <h3>Truck Status</h3>
    {trucks.length === 0 ? (
      <p style={{ textAlign: "center" }}>No trucks found</p>
    ) : (
```

Figure 81: Admin dashboard 4

```
        trucks.map((truck) => (
          <div className="truck_update" key={truck.truck_ID}>
            <div>
              <b>{truck.vehicle_number}</b>
              <p>{truck.truck_ID} · {truck.driver_name ?? "No driver assigned"}</p>
            </div>
            <span className="status active">Active</span>
          </div>
        ))
      )}
    </div> </div>
  {/* Quick Actions */}
  <div className="quick_action_part">
    <h3>Quick Actions</h3>
    <div className="actions_part">
      <button className="action green" onClick={() => navigate("/add_driver")}>
        <i className="bi bi-person-plus-fill"></i> Add Driver
      </button>
      <button className="action green" onClick={() => navigate("/driver_managment")}>
        <i className="bi bi-people-fill"></i> Manage Driver
      </button>
      <button className="action green" onClick={() => navigate("/resident_managment")}>
        <i className="bi bi-house-door-fill"></i> Manage Residents
      </button>
      <button className="action green" onClick={() => navigate("/add_truck")}>
        <i className="bi bi-truck"></i> Add Trucks
      </button>
      <button className="action green" onClick={() => navigate("/truck_management")}>
        <i className="bi bi-truck-flatbed"></i> Manage Trucks
      </button>
```

Figure 82: Admin dashboard 5

```
      <button className="action green" onClick={() => navigate("/add_route")}>
        <i className="bi bi-geo-alt-fill"></i> Add Route
      </button>
      <button className="action green" onClick={() => navigate("/route_managment")}>
        <i className="bi bi-map-fill"></i> Manage Routes
      </button>
      <button className="action green" onClick={() => navigate("/add_bin")}>
        <i className="bi bi-trash-fill"></i> Add Bins
      </button>
      <button className="action green" onClick={() => navigate("/bin_managment")}>
        <i className="bi bi-trash3-fill"></i> Manage Bins
      </button>
      <button className="action green" onClick={() => navigate("/add_qr_code")}>
        <i className="bi bi-qr-code-scan"></i> Add QR Code
      </button>
      <button className="action green" onClick={() => navigate("/qr_code_manage")}>
        <i className="bi bi-upc-scan"></i> Manage QR Code
      </button>
      <button className="action green" onClick={() => navigate("/add_schedule")}>
        <i className="bi bi-calendar-plus-fill"></i> Add Schedule
      </button>
      <button className="action green" onClick={() => navigate("/schedule_management")}>
        <i className="bi bi-calendar-check-fill"></i> Manage Schedule
      </button>
      <button className="action green" onClick={() => navigate("/add_content")}>
        <i className="bi bi-megaphone-fill"></i> Add Content
      </button>
      <button className="action green" onClick={() => navigate("/content_managment")}>
        <i className="bi bi-file-earmark-text-fill"></i> Manage Content
      </button>
      <button className="action green" onClick={() => navigate("/feedback_management")}>
```

Figure 83: Admin dashboard 6

4.10 Bin Management

```
import axios from "axios";
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import "./bin_managment.css";
import "../extra_CSS_file.css";
import AdminNavBar from "../Admin_Navigation_Bar/admin_navigation";
function ManageBin() {
    const navigate = useNavigate();
    const [bins, setBins] = useState([]);
    const [routes, setRoutes] = useState([]);
    const [qrCodes, setQrCodes]= useState([]);
    const [search, setSearch] = useState ("");
    const [loading, setLoading] = useState ("");
    const [error, setError] = useState("");
    const [success, setSuccess] = useState("");
    const [showModal, setShowModal] = useState(false);
    const [selectedBin, setSelectedBin] = useState(null);
    const [editData, setEditData] = useState({});
    const [showDeleteModal, setShowDeleteModal] = useState(false);
    const [deleteTarget, setDeleteTarget] = useState(null);
    // Fetch all bins
      const fetchBins = () => {
        setLoading(true);
        axios.get("http://localhost:5000/get-bins")
          .then(res => {
            setBins(res.data);
            setLoading(false);
          })
          .catch(() => {
            setError("Failed to load bins.");
            setLoading(false);
          });
```

Figure 84: Bin management 1

```
//Fetcj all routes
const fetchRoutes =() => {
    axios.get("http://localhost:5000/get-routes")
        .then(res =>
            setRoutes(res.data))
        .catch(() =>
            console.log("Failed to load routes."));
};
// Fetch all QR codes
const fetchQRCodes = () => {
    axios.get("http://localhost:5000/get-qrcodes")
        .then(res =>
            setQrCodes(res.data))
        .catch(() =>
            console.log ("Failed to load QR codes"));
};
useEffect(() => {
    fetchBins();
    fetchRoutes();
    fetchQRCodes();
}, []);
//Filter bins by search
const filteredBins = bins.filter (b =>
    (b.bin_ID || "").toLowerCase().includes(search.toLowerCase()) ||
    (b.type || "").toLowerCase().include(search.toLowerCase()) ||
    (b.location || "").toLowerCase().includes(search.toLowerCase()) ||
    (b.route_ID || "").toLowerCase().includes(search.toLowerCase()) ||
    (b.status ||"").toLowerCase().includes(search.toLowerCase())
);
```

Figure 85: Bin management 2

```
    // Get route name by ID
    const getRouteName = (route_ID) => {
        const route = routes.find(r => r.route_ID === route_ID);
        return route ? route.route_name : route_ID;
    };
    // Bin type icon
    const getBinIcon = (type) => {
    if (type === "Recycling bins") {
        return <i className="bi bi-recycle"></i>;
    }
    if (type === "Organic baste bins") {
        return <i className="bi bi-flower1"></i>;
    }
    if (type === "Hazardous waste bins") {
        return <i className="bi bi-exclamation-octagon-fill"></i>;
    }
    return <i className="bi bi-trash-fill"></i>;
};
    const formatDate = (d) => d? d.split("T")[0] : "-";
    const formatTime = (t) => t ? t.slice(0,5) : "-";
    //count by status
    const emptyCount = bins.filter(b => b.status === "Empty").length;
    const filledCount = bins.filter(b => b.status === "Filled").length;
    //Open edit modal
    const openEdit = (bin) => {
        setSelectedBin(bin);
        setEditData({
            type: bin.type,
            status: bin.status,
            location: bin.location,
```

Figure 86: Bin management 3

```
        QR_ID: bin.QR_ID,
        lst_clctn_date: bin.lst_clctn_date ? bin.lst_clctn_date.split("T")[0] : "",
        lst_clctn_time: bin.lst_clctn_time ? bin.lst_clctn_time.slice(0, 5) : "",
    });
    setShowModal(true);
    setError("");
    setSuccess("");
};
const handleEditChange =(e) => {
    setEditData({...editData, [e.target.name]: e.target.value});
};
//submit edit
const handleUpdate = () => {
    if (!editData.type || !editData.status || !editData.location ||
        !editData.route_ID || !editData.QR_ID) {
    setError ("All fields are required");
    return;
}
axios.put(`http://localhost:5000/update-bin/${selectedBin.bin_ID}`, editData)
    .then(() => {
        setSuccess(`Bin "${selectedBin.bin_ID}" updated successfully!`);
        setShowModal(false);
        fetchBins();
    })
    .catch(err => {
        setError(err.response?.data || "Error updating bin.");
    });
};
```

Figure 87: Bin management 4

```
};
//Open delete modal
const openDelete = (bin) => {
    setDeleteTarget(bin);
    setShowDeleteModal(true);
};
 // Confirm delete
  const handleDelete = () => {
    axios.delete(`http://localhost:5000/delete-bin/${deleteTarget.bin_ID}`)
      .then(() => {
        setSuccess(`Bin "${deleteTarget.bin_ID}" deleted successfully!`);
        setShowDeleteModal(false);
        setDeleteTarget(null);
        fetchBins();
      })
      .catch(err => {
        setError(err.response?.data || "Error deleting bin.");
        setShowDeleteModal(false);
      });
  };
```

Figure 88: Bin management 5

Add bin

```
function AddBin() {
  const navigate = useNavigate();
  const [formData, setFormData] = useState({
      bin_ID: "",
      type: "",
      status: "",
      location: "",
      route_ID: "",
      QR_ID: ""
  });
  const [routes, setRoutes] = useState([]);
  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const [loadingRoutes, setLoadingRoutes] = useState(true);
  const [qrCodes, setQrCodes] = useState([]);
  const [loadingQR, setLoadingQR] = useState(true);
  useEffect(() => {
      axios.get("http://localhost:5000/get-routes")
        .then(res => {
          setRoutes(res.data);
          setLoadingRoutes(false);
        })
        .catch(() => {
          setError("Failed to load routes.");
          setLoadingRoutes(false);
        });
      axios.get("http://localhost:5000/get-qrcodes")
        .then(res => {
          setQrCodes(res.data);
          setLoadingQR(false);
        })
        .catch(() => {
```

Figure 89: Add bin 1



```
        })
        .catch(() => {
          setError("Failed to load QR codea.");
          setLoadingQR(false);
        });
  }, []);
  const handleChange = (e) => {
      setFormData({ ...formData, [e.target.name]: e.target.value });
      setError("");
      setSuccess("");
  };
  const handleSubmit = (e) => {
      e.preventDefault();
      if (!formData.bin_ID || !formData.type || !formData.status ||
          !formData.location || !formData.route_ID || !formData.QR_ID) {
          setError("All fields are required.");
          return;
      }
      axios.post("http://localhost:5000/add-bin", formData)
          .then(() => {
              setSuccess(`Bin "${formData.bin_ID}" added successfully!`);
              setFormData({
                  bin_ID: "",
                  type: "",
                  status: "",
                  location: "",
                  route_ID: "",
                  QR_ID: ""
              });
          })
```

Figure 90: Add bin 2



```
          .catch(err => {
              setError(err.response?.data || "Server error. Please try again");
          });
  };
  const getBinIcon = () => {
      if (formData.type === "Recycling bins") return "bi-recycle";
      if (formData.type === "Organic baste bins") return "bi-tree";
      if (formData.type === "Hazardous waste bins") return "bi-radioactive";
      return "bi-trash";
  };
```

Figure 91: Add bin 3

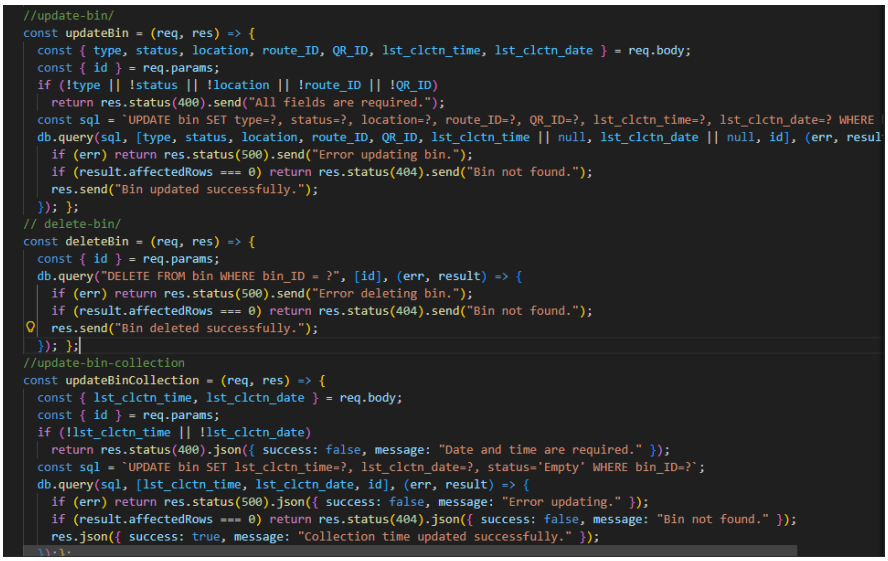
Bin backend



```
const db = require("../config/db");
//get-bins
const getBins = (req, res) => {
  db.query("SELECT * FROM bin", (err, result) => {
    if (err) return res.status(500).send("Database error.");
    res.json(result);
  });
};
//  add-bin
const addBin = (req, res) => {
  const { bin_ID, type, status, location, route_ID, QR_ID } = req.body;

  if (!bin_ID || !type || !status || !location || !route_ID || !QR_ID)
    return res.status(400).send("All fields are required.");
  db.query("SELECT * FROM bin WHERE bin_ID = ?", [bin_ID.trim()], (err, result) => {
    if (err) return res.status(500).send("Database error.");
    if (result.length > 0) return res.status(400).send("Bin ID already exists.");
    db.query("SELECT * FROM bin WHERE QR_ID = ?", [QR_ID.trim()], (err, result) => {
      if (err) return res.status(500).send("Database error.");
      if (result.length > 0) return res.status(400).send("QR ID already exists.");
      db.query("SELECT * FROM route WHERE route_ID = ?", [route_ID.trim()], (err, result) => {
        if (err) return res.status(500).send("Database error.");
        if (result.length === 0) return res.status(400).send("Route ID does not exist.");
        const sql = `INSERT INTO bin (bin_ID, type, status, location, route_ID, QR_ID) VALUES (?,?,?,?,?,?)`;
        db.query(sql, [bin_ID.trim(), type, status, location.trim(), route_ID.trim(), QR_ID.trim()], (err) => {
          if (err) return res.status(500).send("Error adding bin.");
          res.send("Bin added successfully.");
        });   }); }); }); };
```

Figure 92: Bin backend



```
//update-bin/
const updateBin = (req, res) => {
  const { type, status, location, route_ID, QR_ID, lst_clctn_time, lst_clctn_date } = req.body;
  const { id } = req.params;
  if (!type || !status || !location || !route_ID || !QR_ID)
    return res.status(400).send("All fields are required.");
  const sql = `UPDATE bin SET type=?, status=?, location=?, route_ID=?, QR_ID=?, lst_clctn_time=?, lst_clctn_date=? WHERE
  db.query(sql, [type, status, location, route_ID, QR_ID, lst_clctn_time || null, lst_clctn_date || null, id], (err, resul
    if (err) return res.status(500).send("Error updating bin.");
    if (result.affectedRows === 0) return res.status(404).send("Bin not found.");
    res.send("Bin updated successfully.");
  }); };
// delete-bin/
const deleteBin = (req, res) => {
  const { id } = req.params;
  db.query("DELETE FROM bin WHERE bin_ID = ?", [id], (err, result) => {
    if (err) return res.status(500).send("Error deleting bin.");
    if (result.affectedRows === 0) return res.status(404).send("Bin not found.");
    res.send("Bin deleted successfully.");
  }); };
//update-bin-collection
const updateBinCollection = (req, res) => {
  const { lst_clctn_time, lst_clctn_date } = req.body;
  const { id } = req.params;
  if (!lst_clctn_time || !lst_clctn_date)
    return res.status(400).json({ success: false, message: "Date and time are required." });
  const sql = `UPDATE bin SET lst_clctn_time=?, lst_clctn_date=?, status='Empty' WHERE bin_ID=?`;
  db.query(sql, [lst_clctn_time, lst_clctn_date, id], (err, result) => {
    if (err) return res.status(500).json({ success: false, message: "Error updating." });
    if (result.affectedRows === 0) return res.status(404).json({ success: false, message: "Bin not found." });
    res.json({ success: true, message: "Collection time updated successfully." });
```

Figure 93: Bin backend



```
//get-bin-by-qr
const getBinByQR = (req, res) => {
  const { qr_id } = req.params;
  const sql = `
    SELECT b.*, r.route_name
    FROM bin b
    LEFT JOIN route r ON b.route_ID = r.route_ID
    WHERE b.QR_ID = ? LIMIT 1
  `;
  db.query(sql, [qr_id], (err, result) => {
    if (err) return res.status(500).json({ success: false, message: "Database error." });
    if (result.length === 0) return res.status(404).json({ success: false, message: "No bin found for this QR code." });
    res.json({ success: true, bin: result[0] });
  });
};

module.exports = { getBins, addBin, updateBin, deleteBin, updateBinCollection, getBinByQR };
```

Figure 94: Bin backend

4.11 Special Pickup



```
import axios from "axios";
import { useNavigate } from "react-router-dom";
import { useState, useEffect } from "react";
import "./pickup_managment.css";
import "../extra_CSS_file.css";
import AdminNavBar from "../Admin_Navigation_Bar/admin_navigation";

const STATUS_OPTIONS = ["Pending", "Scheduled", "Completed", "Rejected"];
function AdminSpecialRequests() {
    const navigate = useNavigate();
    const [requests, setRequests]   =useState([]);
    const [search, setSearch]       =useState("");
    const [loading, setLoading]     =useState(true);
    const [error, setError]         = useState("");
    const [success, setSuccess]     = useState("");
    const [showModal, setShowModal] = useState(false);
    const [selectedRequest, setSelectedRequest] =useState(null);
    const [editData, setEditData]   =useState({});
    const [showDeleteModal, setShowDeleteModal] = useState(false);
    const [deleteTarget, setDeleteTarget]   = useState(null);
    //fetch
        const fetchRequests = () => {
        setLoading(true);
        axios.get("http://localhost:5000/admin/special-requests")
        .then((res) => { setRequests(res.data.success ? res.data.requests : []); setLoading(false); })
        .catch(() => { setError("Failed to load requests."); setLoading(false); });
    };
    useEffect(() => { fetchRequests(); }, []);
    // Filter
    const filteredRequests = requests.filter((r) => {
        const q = search.toLowerCase();
```

Figure 95: Special pickup page 1



```
    return(
        String(r.request_ID).includes(q) ||
        (r.resident_name || "").toLowerCase().includes(q) ||
        (r.item_type    || "").toLowerCase().includes(q) ||
        (r.location || "").toLowerCase().includes(q) ||
        (r.status        || "").toLowerCase().includes(q)
    );
});
//  Status badge class
const getStatusClass = (status) => {
    switch (status) {
    case "Pending":   return "asr_badge_pending";
    case "Scheduled": return "asr_badge_scheduled";
    case "Completed": return "asr_badge_completed";
    case "Rejected":  return "asr_badge_rejected";
    default:          return "asr_badge_pending";
    } };
//  Open edit modal
    const openEdit = (req) => {
        setSelectedRequest(req);
        setEditData({
        status:        req.status        || "Pending",
        schedule_date: req.schedule_date ? req.schedule_date.split("T")[0] : "",
        schedule_time: req.schedule_time || "",
        });
        setError("");
        setSuccess("");
        setShowModal(true);
    };
```

Figure 96: : Special pickup page 2



```
    const handleEditChange = (e) => {
        setError("");
        const { name, value } = e.target;
        setEditData((prev) => ({ ...prev, [name]: value }));
    };
    // Submit update
const handleUpdate = () => {
    if (!editData.status) { setError("Please select a status."); return; }

    if (editData.status === "Scheduled" && (!editData.schedule_date || !editData.schedule_time)) {
    setError("Please provide both schedule date and time when status is Scheduled.");
    return;
    }
    axios.patch(`http://localhost:5000/admin/special-requests/${selectedRequest.request_ID}`, editData)
    .then(() => {
        setSuccess(`Request #${selectedRequest.request_ID} updated successfully!`);
        setShowModal(false);
        fetchRequests();
    })
    .catch((err) => setError(err.response?.data?.message || "Error updating request."));
};
//  Delete
const openDelete = (req) => { setDeleteTarget(req); setShowDeleteModal(true); };
  const handleDelete = () => {
    axios.delete(`http://localhost:5000/admin/special-requests/${deleteTarget.request_ID}`)
      .then(() => {
        setSuccess(`Request #${deleteTarget.request_ID} deleted successfully!`);
        setShowDeleteModal(false);
        fetchRequests();
      })
```

Figure 97: Special pickup page 3



```
        .catch(() => { setError("Error deleting request."); setShowDeleteModal(false); });
    };

    const formatDate = (d) => (d ? d.split("T")[0] : "—");
    const formatTime = (t) => {
      if (!t) return "—";
      const [h, m] = t.split(":");
      const hour = parseInt(h);
      return `${hour % 12 || 12}:${m} ${hour >= 12 ? "PM" : "AM"}`;
    };
    const countByStatus = (s) => requests.filter((r) => r.status === s).length;
```

Figure 98: Special pickup page 4

4.12 Driver management page

Add driver

```
function AddDriver() {

  const navigate = useNavigate();
  const [formData, setFormData] = useState({
    driver_ID: "",
    name: "",
    email: "",
    phone_no: "",
    license_no: "",
    password: ""
  });
  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
    setError("");
    setSuccess("");
  };

  const getInitials = () => {
    const parts = formData.name.trim().split(" ");
    if (parts.length >= 2 && parts[1].length > 0)
      return parts[0][0].toUpperCase() + parts[1][0].toUpperCase();
    if (formData.name.trim().length > 0)
      return formData.name.trim()[0].toUpperCase();
    return "DR";
  };

  const handleSubmit = (e) => {
  e.preventDefault();
```

Figure 99: Add driver 1

```
  if (!formData.driver_ID || !formData.name || !formData.email || !formData.phone_no ||
      !formData.license_no || !formData.password) {
    setError("All fields are required.");
    return;
  }

  if (formData.driver_ID.toString().trim() === "") {
    setError("Driver ID cannot be empty.");
    return;
  }
  if (formData.phone_no.length < 10) {
    setError("Phone number must be at least 10 digits.");
    return;
  }
  if (formData.password.length < 6) {
    setError("Password must be at least 6 characters.");
    return;
  }

  axios.post("http://localhost:5000/add-driver", formData)
    .then(() => {
      setSuccess(`Driver "${formData.name}" added successfully!`);
      setFormData({ driver_ID: "", name: "", email: "", phone_no: "", license_no: "", password: "" });
    })
    .catch(err => {
      setError(err.response?.data || "Server error. Please try again.");
    });
};
```

Figure 100: Add driver 2

Driver management

```
function ManageDrivers() {
  const navigate = useNavigate();
  const [drivers, setDrivers] = useState([]);
  const [search, setSearch] = useState("");
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [success, setSuccess] = useState("");
  const [showModal, setShowModal] = useState(false);
  const [selectedDriver, setSelectedDriver] = useState(null);
  const [editData, setEditData] = useState({});
  const [showDeleteModal, setShowDeleteModal] = useState(false);
  const [deleteTarget, setDeleteTarget] = useState(null);

  const getDriverId = (driver) => {
    if (!driver) return null;
    return driver.driver_ID || driver.driver_id || driver.id || null;
  };
  // Fetch  drivers
  const fetchDrivers = () => {
    setLoading(true);
    axios.get("http://localhost:5000/get-drivers")
      .then(res => {
        console.log("Drivers from DB:", res.data);
        setDrivers(res.data);
        setLoading(false);
      })
      .catch(() => {
        setError("Failed to load drivers.");
        setLoading(false);
      });
  };
```

Figure 101: Driver management 1

```
  });
};
useEffect(() => {
  fetchDrivers();
}, []);
// Filter drivers
const filteredDrivers = drivers.filter(d =>
  d.name.toLowerCase().includes(search.toLowerCase()) ||
  d.email.toLowerCase().includes(search.toLowerCase()) ||
  (getDriverId(d) || "").toString().toLowerCase().includes(search.toLowerCase()) ||
  d.license_no.toLowerCase().includes(search.toLowerCase())
);
// Open edit modal
const openEdit = (driver) => {
  console.log("Opening edit for driver:", driver);
  console.log("Driver ID:", getDriverId(driver));
  setSelectedDriver(driver);
  setEditData({
    name: driver.name,
    email: driver.email,
    phone_no: driver.phone_no,
    license_no: driver.license_no
  });
  setShowModal(true);
  setError("");
  setSuccess("");
};
// Handle edit input change
const handleEditChange = (e) => {
  setEditData({ ...editData, [e.target.name]: e.target.value });
};
```

Figure 102: Driver management 2



```
// Submit edit
const handleUpdate = () => {
  const driverID = getDriverId(selectedDriver);
  console.log("Updating driver ID:", driverID);

  if (!editData.name || !editData.email || !editData.phone_no || !editData.license_no) {
    setError("All fields are required.");
    return;
  }
  if (!driverID) {
    setError("Driver ID is missing. Please close and try again.");
    return;
  }
  if (editData.phone_no.length < 10) {
    setError("Phone number must be at least 10 digits.");
    return;
  }

  axios.put(`http://localhost:5000/update-driver/${driverID}`, editData)
    .then(() => {
      setSuccess(`Driver "${editData.name}" updated successfully!`);
      setShowModal(false);
      fetchDrivers();
    })
    .catch(err => {
      setError(err.response?.data || "Error updating driver.");
    });
};
// Open delete confirm
```

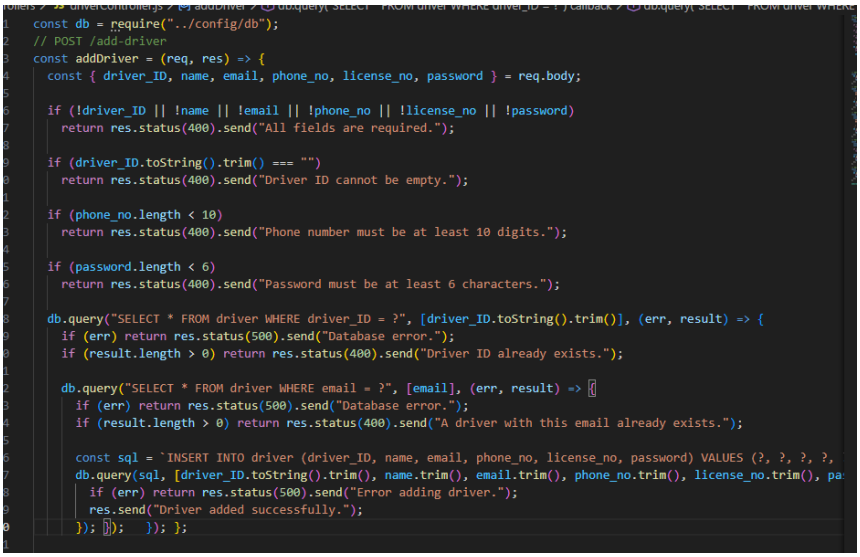
Figure 103: Driver management 3



```
  // Open delete confirm
  const openDelete = (driver) => {
    setDeleteTarget(driver);
    setShowDeleteModal(true);
  };
  // Confirm delete
  const handleDelete = () => {
    const driverID = getDriverId(deleteTarget);
    axios.delete(`http://localhost:5000/delete-driver/${driverID}`)
      .then(() => {
        setSuccess(`Driver "${deleteTarget.name}" deleted successfully!`);
        setShowDeleteModal(false);
        fetchDrivers();
      })
      .catch(() => {
        setError("Error deleting driver.");
        setShowDeleteModal(false);
      });
  };
  const getInitials = (name) => {
    if (!name) return "DR";
    const parts = name.trim().split(" ");
    if (parts.length >= 2 && parts[1].length > 0)
      return parts[0][0].toUpperCase() + parts[1][0].toUpperCase();
    return name.trim()[0].toUpperCase();
  };
```

Figure 104: Driver management 4

Driver backend



```
const db = require("../config/db");
// POST /add-driver
const addDriver = (req, res) => {
  const { driver_ID, name, email, phone_no, license_no, password } = req.body;

  if (!driver_ID || !name || !email || !phone_no || !license_no || !password)
    return res.status(400).send("All fields are required.");

  if (driver_ID.toString().trim() === "")
    return res.status(400).send("Driver ID cannot be empty.");

  if (phone_no.length < 10)
    return res.status(400).send("Phone number must be at least 10 digits.");

  if (password.length < 6)
    return res.status(400).send("Password must be at least 6 characters.");

  db.query("SELECT * FROM driver WHERE driver_ID = ?", [driver_ID.toString().trim()], (err, result) => {
    if (err) return res.status(500).send("Database error.");
    if (result.length > 0) return res.status(400).send("Driver ID already exists.");

    db.query("SELECT * FROM driver WHERE email = ?", [email], (err, result) => {
      if (err) return res.status(500).send("Database error.");
      if (result.length > 0) return res.status(400).send("A driver with this email already exists.");

      const sql = `INSERT INTO driver (driver_ID, name, email, phone_no, license_no, password) VALUES (?, ?, ?, ?,
      db.query(sql, [driver_ID.toString().trim(), name.trim(), email.trim(), phone_no.trim(), license_no.trim(), pa
        if (err) return res.status(500).send("Error adding driver.");
        res.send("Driver added successfully.");
      }); });   }); };
```

Figure 105:: Driver backend



```
// get drivers
const getDrivers = (req, res) => {
  db.query("SELECT * FROM driver", (err, result) => {
    if (err) return res.status(500).send("Database error.");
    res.json(result);
  }); };
//get-unassigned drivers
const getUnassignedDrivers = (req, res) => {
  const sql = `
    SELECT * FROM driver
    WHERE driver_ID NOT IN (
      SELECT driver_id FROM truck WHERE driver_id IS NOT NULL AND driver_id != ''
    )
  `;
  db.query(sql, (err, result) => {
    if (err) return res.status(500).send("Database error.");
    res.json(result);
  }); };
//get-available-drivers/truckID
const getAvailableDrivers = (req, res) => {
  const { truckID } = req.params;
  const sql = `
    SELECT * FROM driver
    WHERE driver_ID NOT IN (
      SELECT driver_id FROM truck WHERE driver_id IS NOT NULL AND truck_ID != ?
    )
  `;
  db.query(sql, [truckID], (err, result) => {
    if (err) return res.status(500).send("Database error.");
    res.json(result);
  }); };
```

Figure 106: : Driver backend

```
//update-driver
const updateDriver = (req, res) => {
  const { name, email, phone_no, license_no } = req.body;
  const { id } = req.params;

  if (!name || !email || !phone_no || !license_no)
    return res.status(400).send("All fields are required.");

  const sql = `UPDATE driver SET name=?, email=?, phone_no=?, license_no=? WHERE driver_ID=?`;
  db.query(sql, [name, email, phone_no, license_no, id], (err) => {
    if (err) return res.status(500).send("Error updating driver.");
    res.send("Driver updated successfully.");
  });
};

//delete-driver
const deleteDriver = (req, res) => {
  const { id } = req.params;
  db.query("DELETE FROM driver WHERE driver_ID = ?", [id], (err) => {
    if (err) return res.status(500).send("Error deleting driver.");
    res.send("Driver deleted successfully.");
  });
};

module.exports = { addDriver, getDrivers, getUnassignedDrivers, getAvailableDrivers, updateDriver, deleteDriver };
```

Figure 107 : Driver backend

4.13 Rout Page

```
const db = require("../config/db");
//get-routes
const getRoutes = (req, res) => {
  db.query("SELECT * FROM route", (err, result) => {
    if (err) return res.status(500).send("Database error.");
    res.json(result);
  }); };
//add-route
const addRoute = (req, res) => {
  const { route_ID, route_name, start_location, end_location,
          estimated_duration, area_name, driver_ID, truck_ID } = req.body;
  if (!route_ID || !route_name || !start_location || !end_location ||
      !estimated_duration || !area_name || !driver_ID || !truck_ID)
    return res.status(400).send("All fields are required.");
  db.query("SELECT * FROM route WHERE route_ID = ?", [route_ID], (err, result) => {
    if (err) return res.status(500).send("Database error.");
    if (result.length > 0) return res.status(400).send("Route ID already exists.");
    db.query("SELECT COUNT(*) AS count FROM route WHERE truck_ID = ?", [truck_ID], (err, truckResult) => {
      if (err) return res.status(500).send("Database error.");
      if (truckResult[0].count >= 3)
        return res.status(400).send("This truck is already assigned to 3 routes. Maximum limit reached.");
      db.query("SELECT COUNT(*) AS count FROM route WHERE driver_ID = ?", [driver_ID], (err, driverResult) => {
        if (err) return res.status(500).send("Database error.");
        if (driverResult[0].count >= 3)
          return res.status(400).send("This driver is already assigned to 3 routes. Maximum limit reached.");
        const sql = `INSERT INTO route (route_ID, route_name, start_location, end_location,
                     estimated_duration, area_name, driver_ID, truck_ID) VALUES (?, ?, ?, ?, ?, ?, ?, ?)`;
        db.query(sql, [route_ID, route_name, start_location, end_location,
                       estimated_duration, area_name, driver_ID, truck_ID], (err) => {
          if (err) return res.status(500).send("Error adding route.");
          res.send("Route added successfully.");
        }); }); }); }); };
```

Figure 108: Route backend

```
//update route
const updateRoute = (req, res) => {
  const { route_name, start_location, end_location,
          estimated_duration, area_name, driver_ID, truck_ID } = req.body;
  const { id } = req.params;

  const sql = `UPDATE route SET route_name=?, start_location=?, end_location=?,
               estimated_duration=?, area_name=?, driver_ID=?, truck_ID=? WHERE route_ID=?`;

  db.query(sql, [route_name, start_location, end_location,
                 estimated_duration, area_name, driver_ID, truck_ID, id], (err, result) => {
    if (err) return res.status(500).send("Error updating route.");
    if (result.affectedRows === 0) return res.status(404).send("Route not found.");
    res.send("Route updated successfully.");
  });
};
//delete route
const deleteRoute = (req, res) => {
  const { id } = req.params;
  db.query("DELETE FROM route WHERE route_ID = ?", [id], (err, result) => {
    if (err) return res.status(500).send("Error deleting route.");
    if (result.affectedRows === 0) return res.status(404).send("Route not found.");
    res.send("Route deleted successfully.");
  });
};

module.exports = { getRoutes, addRoute, updateRoute, deleteRoute };
```

Figure 109: Route backend

4.14 Schedule page

```
const db = require("../config/db");

//check truck availability
const checkTruckAvailability = (req, res) => {
  const { truck_ID, time, day_of_week } = req.query;

  if (!truck_ID || !day_of_week || !time)
    return res.status(400).json({ error: "truck_ID, day_of_week, and time are required." });

  const query = `
    SELECT s.schedule_id, s.time, s.day_of_week, r.route_name, r.route_ID
    FROM schedule s
    JOIN route r ON s.route_ID = r.route_ID
    WHERE r.truck_ID = ? AND s.day_of_week = ? AND s.time = ?
    LIMIT 1
  `;
  db.query(query, [truck_ID, day_of_week, time], (err, results) => {
    if (err) return res.status(500).json({ error: "DB error checking truck availability." });
    if (results.length > 0) {
      return res.json({
        conflict: true,
        conflictDetails: {
          schedule_id: results[0].schedule_id,
          route_name:  results[0].route_name,
          route_ID:    results[0].route_ID,
          time:        results[0].time,
          day_of_week: results[0].day_of_week,
        },  });   }
    res.json({ conflict: false, conflictDetails: null });
  }); };
```

Figure 110: Schedule backend 1



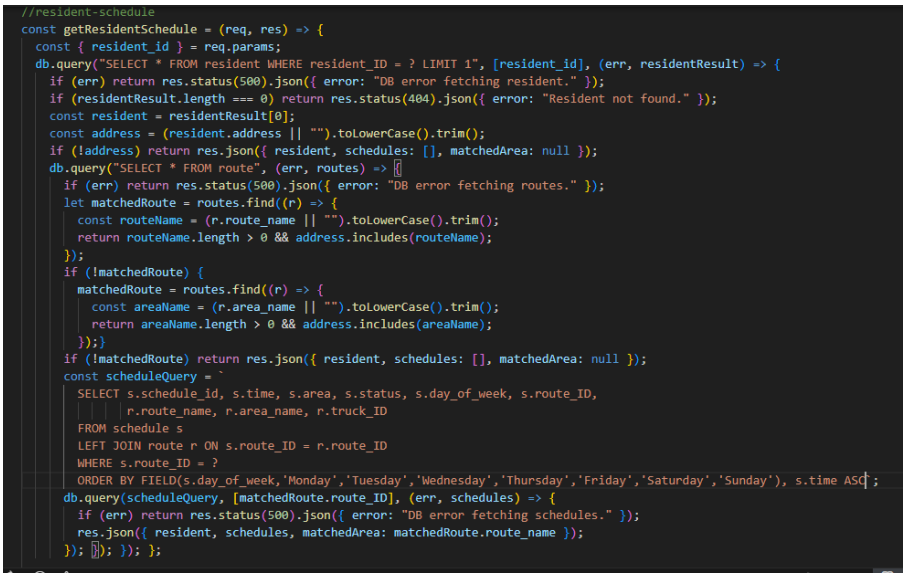
```
//add-schedule
const addSchedule = (req, res) => {
  const { schedule_id, time, area, status, day_of_week, route_ID } = req.body;
  if (!schedule_id || !time || !area || !status || !day_of_week || !route_ID)
    return res.status(400).json("All fields are required.");
  db.query("SELECT * FROM schedule WHERE schedule_id = ?", [schedule_id], (err, result) => {
    if (err) return res.status(500).json("Database error.");
    if (result.length > 0) return res.status(409).json(`Schedule ID "${schedule_id}" already exists.`);
    const truckConflictCheck = `
      SELECT s.schedule_id, s.day_of_week, s.time, r.route_name
      FROM schedule s
      JOIN route r ON s.route_ID = r.route_ID
      WHERE r.truck_ID = (SELECT truck_ID FROM route WHERE route_ID = ?)
        AND s.day_of_week = ? AND s.time = ?
      LIMIT 1
    `;
    db.query(truckConflictCheck, [route_ID, day_of_week, time], (err, conflicts) => {
      if (err) return res.status(500).json("Server error checking truck conflicts.");
      if (conflicts.length > 0)
        return res.status(409).json(
          `Truck is already assigned to route "${conflicts[0].route_name}" on ${day_of_week} at ${time}. Please cho
        );
      const insert = `INSERT INTO schedule (schedule_id, time, area, status, day_of_week, route_ID) VALUES (?, ?, ?
      db.query(insert, [schedule_id, time, area, status, day_of_week, route_ID], (err) => {
        if (err) {
          if (err.code === "ER_DUP_ENTRY") return res.status(409).json(`Schedule ID "${schedule_id}" already exists
          return res.status(500).json("Failed to add schedule.");
        }
        res.status(200).json("Schedule added successfully.");
      });   });  });  };
```

Figure 111: Schedule backend 2



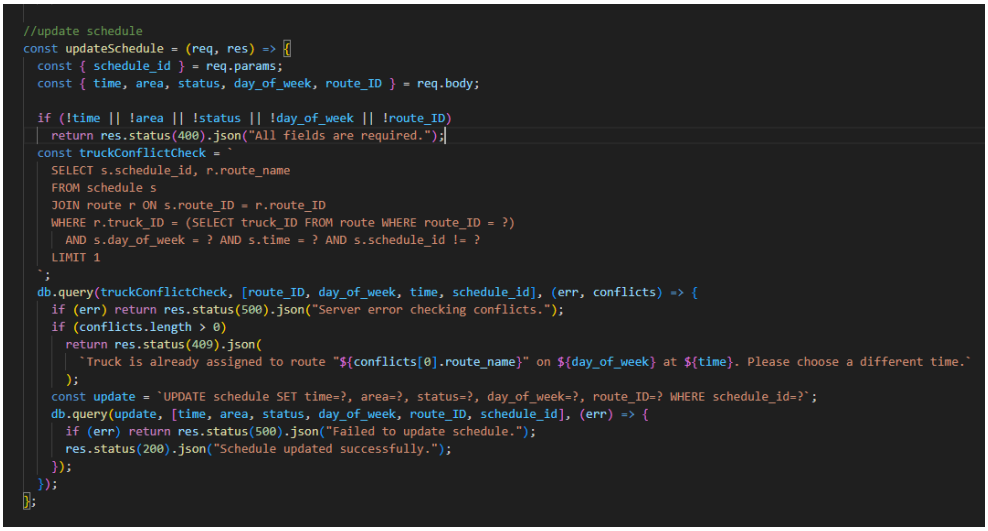
```
//get schedules
const getSchedules = (req, res) => {
  const query = `
    SELECT s.schedule_id, s.time, s.area, s.status, s.day_of_week, s.route_ID,
           r.route_name, r.area_name, r.truck_ID
    FROM schedule s
    LEFT JOIN route r ON s.route_ID = r.route_ID
    ORDER BY FIELD(s.day_of_week,'Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'), s.time AS
  `;
  db.query(query, (err, results) => {
    if (err) return res.status(500).json("Failed to fetch schedules.");
    res.json(results);
  }); };
```

Figure 112: Schedule backend 3



```
//resident-schedule
const getResidentSchedule = (req, res) => {
  const { resident_id } = req.params;
  db.query("SELECT * FROM resident WHERE resident_ID = ? LIMIT 1", [resident_id], (err, residentResult) => {
    if (err) return res.status(500).json({ error: "DB error fetching resident." });
    if (residentResult.length === 0) return res.status(404).json({ error: "Resident not found." });
    const resident = residentResult[0];
    const address = (resident.address || "").toLowerCase().trim();
    if (!address) return res.json({ resident, schedules: [], matchedArea: null });
    db.query("SELECT * FROM route", (err, routes) => {
      if (err) return res.status(500).json({ error: "DB error fetching routes." });
      let matchedRoute = routes.find((r) => {
        const routeName = (r.route_name || "").toLowerCase().trim();
        return routeName.length > 0 && address.includes(routeName);
      });
      if (!matchedRoute) {
        matchedRoute = routes.find((r) => {
          const areaName = (r.area_name || "").toLowerCase().trim();
          return areaName.length > 0 && address.includes(areaName);
        });}
      if (!matchedRoute) return res.json({ resident, schedules: [], matchedArea: null });
      const scheduleQuery = `
        SELECT s.schedule_id, s.time, s.area, s.status, s.day_of_week, s.route_ID,
               r.route_name, r.area_name, r.truck_ID
        FROM schedule s
        LEFT JOIN route r ON s.route_ID = r.route_ID
        WHERE s.route_ID = ?
        ORDER BY FIELD(s.day_of_week,'Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'), s.time ASC`;
      db.query(scheduleQuery, [matchedRoute.route_ID], (err, schedules) => {
        if (err) return res.status(500).json({ error: "DB error fetching schedules." });
        res.json({ resident, schedules, matchedArea: matchedRoute.route_name });
      }); }); }); };
```

Figure 113: Schedule backend 4



```
//update schedule
const updateSchedule = (req, res) => {
  const { schedule_id } = req.params;
  const { time, area, status, day_of_week, route_ID } = req.body;

  if (!time || !area || !status || !day_of_week || !route_ID)
    return res.status(400).json("All fields are required.");
  const truckConflictCheck = `
    SELECT s.schedule_id, r.route_name
    FROM schedule s
    JOIN route r ON s.route_ID = r.route_ID
    WHERE r.truck_ID = (SELECT truck_ID FROM route WHERE route_ID = ?)
      AND s.day_of_week = ? AND s.time = ? AND s.schedule_id != ?
    LIMIT 1
  `;
  db.query(truckConflictCheck, [route_ID, day_of_week, time, schedule_id], (err, conflicts) => {
    if (err) return res.status(500).json("Server error checking conflicts.");
    if (conflicts.length > 0)
      return res.status(409).json(
        `Truck is already assigned to route "${conflicts[0].route_name}" on ${day_of_week} at ${time}. Please choose a different time.`
      );
    const update = `UPDATE schedule SET time=?, area=?, status=?, day_of_week=?, route_ID=? WHERE schedule_id=?`;
    db.query(update, [time, area, status, day_of_week, route_ID, schedule_id], (err) => {
      if (err) return res.status(500).json("Failed to update schedule.");
      res.status(200).json("Schedule updated successfully.");
    });
  });
};
```

Figure 114: Schedule backend 5

```
//delete schedule
const deleteSchedule = (req, res) => {
  const { schedule_id } = req.params;
  db.query("DELETE FROM schedule WHERE schedule_id = ?", [schedule_id], (err, result) => {
    if (err) return res.status(500).json("Failed to delete schedule.");
    if (result.affectedRows === 0) return res.status(404).json("Schedule not found.");
    res.status(200).json("Schedule deleted successfully.");
  });
};

// get active schedules (for home page)
const getActiveSchedules = (req, res) => {
  const query = `
    SELECT s.schedule_id, s.time, s.area, s.status, s.day_of_week, s.route_ID,
           r.route_name, r.area_name, r.truck_ID
    FROM schedule s
    LEFT JOIN route r ON s.route_ID = r.route_ID
    WHERE s.status = 'Active'
    ORDER BY FIELD(s.day_of_week,'Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'), s.time ASC
  `;
  db.query(query, (err, results) => {
    if (err) return res.status(500).json("Failed to fetch schedules.");
    res.json(results);
  });
};

module.exports = { checkTruckAvailability, addSchedule, getSchedules, getResidentSchedule, updateSchedule, deleteSchedule, getActiveSchedules };
```

Figure 115: Schedule backend 6

4.15 Data Validation

Data validation is an important process that ensures the system operates using accurate, complete, and reliable data. The CleanLand Garbage Management System applies various validation rules to prevent incorrect or duplicate information from being stored in the database.

The system validates user inputs during registration, login, complaint submission, feedback submission, and schedule management. Email addresses are checked to ensure uniqueness, required fields are validated before form submission, and data formats such as phone numbers and dates are verified. The system also ensures that authorized users can only access and modify data according to their roles. These validation mechanisms improve data integrity, system reliability, and overall user experience.

4.16 Reusable Components of the System

The CleanLand Garbage Management System was developed using reusable components to improve development efficiency, maintainability, and consistency throughout the application. Several common components are shared across multiple modules of the system.

- Navigation Bar – Provides easy navigation between different system modules.
- Dashboard Cards and Statistics Widgets – Display summary information such as complaints, residents, drivers, trucks, and schedules.
- Form Validation Components – Ensure accurate user input during registration, complaints, feedback, and management operations.
- Data Tables – Used for displaying and managing residents, drivers, trucks, routes, schedules, complaints, and QR codes.

• Modal and Confirmation Dialogs – Used for update, delete, and confirmation actions.

• Notification Messages – Provide success, warning, and error messages to users.

• Responsive Layout Components – Ensure proper display across desktop, tablet, and mobile devices.

These reusable components promote code consistency, reduce development effort, and simplify future system enhancements.

4.17 Difficulties of the Software

During the development of the CleanLand Garbage Management System, several challenges were encountered:

• **QR Code Management:** Generating, storing, and linking unique QR codes with garbage bins required careful database design to ensure accurate identification and status updates.

• **Database Management:** Designing relationships between residents, drivers, trucks, routes, schedules, complaints, feedback, and bins was challenging due to the large number of interconnected modules.

• **Data Validation and Security:** Ensuring data accuracy, preventing duplicate records, implementing user authentication, and restricting access based on user roles required additional security measures.

• **Google Maps Integration:** Implementing truck monitoring using Google Maps required configuring APIs, managing map rendering, and handling location related data effectively.

• **System Integration:** Integrating multiple modules such as complaint management, schedule management, QR code management, feedback management, and truck monitoring into a single platform required careful planning and testing to ensure smooth operation.

4.18 Summary

This chapter discussed the implementation of the CleanLand Garbage Management System. The system was developed as a web based application using React.js for the frontend, Node.js and Express.js for the backend, and MySQL as the database management system. The implementation included frontend routing, backend server configuration, database connectivity, data validation mechanisms, and reusable system components.

Several key features, including complaint management, collection schedule management, QR code based bin management, feedback handling, awareness content management, and truck monitoring,

were successfully integrated into the system. Although challenges were encountered during development, appropriate solutions were implemented to ensure a functional, reliable, and user-friendly garbage management platform.

CHAPTER 05

5. RESULTS AND EVALUATION

5.1 Introduction

This chapter evaluates the performance of the proposed CleanLand Garbage Management System and presents the testing activities carried out to verify that the system functions according to the specified requirements.

Several testing methods were conducted to assess the system:

- **Component Testing** – Individual modules such as resident registration, login, complaint management, schedule management, QR code management, truck management, and feedback management were tested independently.
- **Integration Testing** – Communication between the frontend, backend server, and MySQL database was tested to ensure smooth data exchange and functionality.
- **User Testing** – Residents, drivers, and administrator used the system to evaluate usability, functionality, and overall user experience.
- **Security Testing** – Authentication, role based access control, and data validation mechanisms were tested to protect system data and prevent unauthorized access.

Evaluation Results

The testing results demonstrated that:

- All system modules functioned correctly and interacted successfully with each other.
- Users were able to register, log in, submit complaints, view schedules, and provide feedback without errors.
- Administrator could efficiently manage residents, drivers, trucks, routes, schedules, awareness content, complaints, and special pickup requests.
- The MySQL database correctly stored and retrieved system information.
- Authentication and authorization mechanisms successfully protected sensitive data.

Areas for Future Improvement

Although the system achieved its objectives, several improvements can be considered:

- Mobile application development to complement the web platform.

- Integration of automated SMS notifications.
- Enhanced reporting and analytics dashboards.
- Improved scalability for larger municipal deployments.

5.2 Test and Deployment Plan

System Testing

- Test all functional modules individually.
- Verify communication between frontend, backend, and database.
- Validate user authentication and authorization processes.
- Test complaint submission and tracking functionalities.
- Verify QR code generation and scanning features.
- Test Google Maps truck tracking functionality.
- Validate schedule management and notification features.

Deployment

The CleanLand system is designed to be deployed on a web server environment. The deployment process includes:

- Configuring the backend server and database.
- Deploying the React frontend application.
- Setting up the MySQL database.
- Configuring API connections between frontend and backend.
- Performing final system testing before release.

Maintenance Plan

- Monitor system performance regularly.
- Perform database backups periodically.
- Apply bug fixes and security updates.
- Improve system features based on stakeholder feedback.

- Maintain server availability and data integrity.

5.3 Testing Procedure

In the Agile methodology, testing was carried out continuously throughout the development of the CleanLand Garbage Management System. Each module was tested during every development iteration to identify and resolve issues at an early stage. This approach helped ensure that newly developed features functioned correctly before being integrated with other system components. The testing process incorporated both black-box and white-box testing techniques to verify the system's functionality, reliability, and overall quality. Continuous testing throughout the Agile development cycle ensured that the final system met all specified requirements and was ready for deployment.

Unit Testing: Individual modules of the system were tested separately to ensure that they performed their intended functions correctly. Modules such as user registration, login, complaint management, schedule management, QR code management, feedback management, and truck management were tested independently. White-box testing techniques were used to examine the internal logic of these modules, allowing errors to be detected and corrected before integration with other components.

White-Box Testing: White-box testing focused on examining the internal structure and logic of the system. Source code, database queries, input validation mechanisms, authentication processes, and API functions were carefully tested by providing different inputs and verifying the expected outputs. This testing method helped ensure the accuracy, reliability, and security of the system while identifying coding errors and logical issues.

Black-Box Testing: Black-box testing evaluated the system from the user's perspective without considering the internal implementation details. The testing focused on verifying that all system functionalities operated according to user requirements. Features such as resident registration, login, complaint submission, complaint tracking, QR code scanning, schedule viewing, truck tracking, and feedback submission were tested to ensure that the expected results were produced for different user interactions.

Integration Testing: Integration testing was conducted to verify the communication and data flow between different components of the system. The interaction between the React frontend, Node.js backend server, MySQL database, and Google Maps integration was thoroughly tested. This phase ensured that all modules worked together seamlessly and that information was correctly transmitted and processed across the system.

User Acceptance Testing: The final stage of testing involved selected stakeholders, including residents, drivers, and administrators, evaluating the system in a real-world environment. Users tested the system's features and provided feedback regarding usability, functionality, and

performance. The results confirmed that the CleanLand Garbage Management System successfully met user requirements and provided an effective solution for improving communication, transparency, complaint management, and operational efficiency within municipal waste management services.

Testing Outcome

The testing process confirmed that the CleanLand Garbage Management System successfully fulfills its intended objectives by improving communication, transparency, complaint handling, and operational efficiency within municipal waste management services. The system was found to be stable, user-friendly, and suitable for deployment in a real world environment.

5.4 Test Cases

5.4.1 Test Cases for Registration

Test case ID	REG_01
Title	Resident registration with valid credentials
Description	Verify that a resident can successfully register by providing all required valid information.
Test steps	1. Navigate to the Registration page. 2. Enter Name. 3. Enter Email Address. 4. Enter Phone Number. 5. Enter Address. 6. Enter Password. 7. Click the Register button
Input data	Name: Harin Fedrik Email: harinfedrik@gmail.com Phone Number: 0773453451 Address: No. 6, Borella Road, Colombo Password: 123456
Expected result	The resident account should be successfully created and redirected to the login page
Actual result	Resident account created successfully and redirected to the login page
Status	Pass

Table 3: Test case for registration 1

Test case ID	REG_02
Title	Resident Registration with Empty Fields
Description	Verify registration with empty fields

Test steps	1. Navigate to the Registration page. 2. Leave Name field empty. 3. Leave Email field empty. 4. Leave Phone Number field empty. 5. Leave Address field empty. 6. Leave Password field empty. 7. Click the Register button.
Input data	Name: Email: Phone Number: Address: Password:
Expected result	The system should display a message such as **"Please fill required fields."** Registration should not proceed.
Actual result	As expected.
Status	Pass

Table 4: Test case for registration 2

5.4.2 Test Case for login

Test case ID	LOG_01
Title	Resident Login with Valid Credentials
Description	Verify that a resident can successfully log in using valid email, password, and role selection.
Test steps	1. Navigate to the Login page. 2. Enter a valid email address. 3. Enter a valid password. 4. Select "Resident" from the role selection dropdown. 5. Click the Login button.

Input data	Email Address: harinfedrik@gmail.com Password: 123456 Login As: Resident
Expected result	The system should authenticate the resident successfully and redirect the user to the home page.
Actual result	As expected.
Status	Pass

Table 5: Test case for login 1

Test case ID	LOG_02
Title	Resident login with invalid credentials
Description	Verify that the system prevents login when incorrect credentials are entered.
Test steps	1. Navigate to the Login screen. 2. Enter an invalid email address. 3. Enter an incorrect password. 4. Select Resident from the Login As dropdown. 5. Click the Login button.
Input data	Email Address: lamey@gmail.com Password: abcdefg Login As: Resident
Expected result	The system should display an error message such as "Invalid email or password" and deny access.
Actual result	As expected.
Status	Pass

Table 6: Test case for login 2

5.4.3 Special pickup request test case

Test case ID	SP_REQ_01

Title	Submit Special Pickup Request with Valid Details
Description	Verify that a resident can successfully submit a special pickup request by providing all required information.
Test steps	1. Navigate to the Special Pickup Request page. 2. Enter a valid location. 3. Enter a valid email address. 4. Enter a valid address. 5. Select an item type. 6. Select a preferred pickup date. 7. Select a preferred pickup time. 8. Enter a description (optional). 9. Click the Submit Request button.
Input data	**Location:** Main street next to the school **Email:** check1@gmail.com **Address:** No 1, Check1 Lane **Item Type:** Furniture **Preferred Date:** 2026-05-12 **Preferred Time:** 21:18 **Description:** I have an overload of furniture to dispose of
Expected result	The system should successfully save the pickup request, generate a unique Request ID, set the status as Pending, and display a success message and request should display on request management page & resident submitted request page .
Actual result	As expected.
Status	Pass

Table 7: Test case for special pickup request 1

Test case ID	SP_REQ_02

Title	Submit Special Pickup Request with Empty Required Fields
Description	Verify that the system prevents submission when required fields are left empty.
Test steps	1. Navigate to the New Pickup Request page. 2. Leave Location empty. 3. Leave Email empty. 4. Leave Address empty. 5. Do not select an Item Type. 6. Do not select Date and Time. 7. Click the Submit Request button.
Input data	Location: Empty Email: Empty Address: Empty Item Type: Empty Preferred Date: Empty Preferred Time: Empty
Expected result	The system should display validation messages such as "Please fill all required fields" and prevent request submission.
Actual result	As expected.
Status	Pass

Table 8: Test case for special pickup request 2

5.4.4 Complaint submission test case

Test case ID	COM_REQ_01
Title	Submit Complaint with Valid Details
Description	Verify that a resident can successfully submit a complaint by providing all required information.
Test steps	1. Navigate to the Submit New Complaint page. 2. Select a complaint type.

	3. Enter a valid location. 4. Select a date. 5. Enter a complaint description. 6. Upload an image. 7. Click the Submit button.
Input data	**Complaint Type:** Damaged Bin **Location:** Colombo **Date:** 2026-05-02 **Description:** Damage bin near temple **Image:** 1777791687231-744963832.png
Expected result	The complaint should be successfully submitted, stored in the database, assigned a Complaint ID, and the status should be set to Pending and it should display on complaint management & resident submitted complaint page.
Actual result	As expected.
Status	Pass

Table 9: Test case for complaint submission 1

Test case ID	COM_REQ_02
Title	Submit Complaint with Empty Required Fields
Description	Verify that the system prevents complaint submission when required fields are empty.
Test steps	1. Navigate to the Submit New Complaint page. 2. Leave Complaint Type empty. 3. Leave Location empty. 4. Leave Description empty. 5. Do not upload an image. 6. Click the Submit button.
Input data	Complaint Type: Empty Location: Empty Description: Empty Image: Empty

Expected result	The system should display an error message such as "Please fill all required fields" and prevent complaint submission.
Actual result	As expected.
Status	Pass

Table 10: Test case for complaint submission 2

5.4.5 Feedback submission test case

Test case ID	FB_01
Title	Submit Feedback with Valid Rating and Comment
Description	Verify that a resident can successfully submit feedback with a star rating and comment.
Test steps	1. Navigate to the **Submit Feedback** page. 2. Select a star rating. 3. Enter a comment. 4. Click the **Submit Feedback** button.
Input data	**Rating:** 1 Star **Comment:** Better
Expected result	The feedback should be successfully submitted and stored in the database with a unique Feedback ID and submission date and it should display on feedback management page and resident feedback page .
Actual result	As expected.
Status	Pass

Table 11: Test case for feedback submission 1

Test case ID	FB_REQ_02
Title	Submit Feedback Without Selecting Rating
Description	Verify that the system prevents feedback submission when no star rating is selected.
Test steps	1. Navigate to the Submit Feedback page. 2. Leave the star rating unselected. 3. Enter a comment. 4. Click the Submit Feedback button.
Input data	Rating**:** Empty Comment**:** Good service
Expected result	The system should display the error message "Please select a star rating" and prevent feedback submission.
Actual result	As expected.
Status	Pass

Table 12: Test case for feedback submission 2

5.4.6. Add Schedule test case

Test case ID	SCH_AD_01
Title	Add Schedule with Valid Details
Description	Verify that the administrator can successfully create a garbage collection schedule with valid information.
Test steps	1. Log in as Admin. 2. Navigate to the Add Schedule page. 3. Enter a Schedule ID. 4. Select Status as Active. 5. Select a day of the week.

	6. Select a collection time. 7. Select a route. 8. Click the Add Schedule button.
Input data	**Schedule ID:** SCH004-MON **Status:** Active **Day:** Monday **Time:** 17:11 PM **Route ID:** R003
Expected result	The schedule should be successfully saved in the database and displayed in the Schedule Management page and resident garbage collection schedule page.
Actual result	As expected.
Status	Pass

Table 13: Test case for add schedule 1

Test case ID	SCH_AD_02
Title	Add Schedule with Multiple Selected Days
Description	Verify that schedules can be generated for multiple selected days.
Test steps	1. Navigate to the Add Schedule page. 2. Enter a Schedule ID. 3. Select multiple days (Tuesday, Wednesday, Thursday, Friday). 4. Select a route and time. 5. Click Add Schedule**.**
Input data	**Schedule ID:** SCH002 **Days:** Tuesday, Wednesday, Thursday, Friday **Time:** 11:55 AM **Route ID:** R004

Expected result	Separate schedule records should be created for each selected day and displayed in the Schedule Management page and resident garbage collection schedule page (e.g., SCH002-TUE, SCH002-WED, SCH002-THU, SCH002-FRI).
Actual result	As expected.
Status	Pass

Table 14: Test case for add schedule 2

5.4.7 Test case for route registration

Test case ID	ROUTE_ADD_01
Title	Add Route with Valid Details
Description	Verify that the administrator can successfully create a new route and that the assigned driver is automatically selected based on the chosen truck.
Test steps	1. Navigate to the Add Route page. 2. Enter Route ID. 3. Enter Route Name as. 4. Enter Start Location and End Location. 5. Enter Area Name. 6. Enter Estimated Duration. 7. Select a Truck from the dropdown list. 8. Verify that the driver assigned to the selected truck is automatically displayed. 9. Click Add Route.
Input data	**Route ID:** R009 **Route Name:** Colombo South Route **Start Location:** Colombo 03 **End Location:** Colombo 07 **Area Name:** Colombo Central

	Estimated Duration: 2h 30m **Truck:** T001
Expected result	The route should be successfully created. The driver assigned to the selected truck should be automatically populated. The new route should appear in the Route Management page.
Actual result	As expected.
Status	Pass

Table 15: Test case for route registration 1

Test case ID	ROUTE_ADD_02
Title	Verify Automatic Driver Assignment Based on Selected Truck
Description	Verify that the system automatically displays the driver assigned to the selected truck.
Test steps	1. Navigate to the Add Route page. 2. Select Truck T001 from the dropdown. 3. Observe the Driver field.
Input data	**Truck:** T001
Expected result	The Driver field should automatically display the driver assigned to Truck T001 (e.g., Driver ID 000002). The field should not require manual selection.
Actual result	As expected.
Status	Pass

Table 16: Test case for route registration 2

5.5 Summary

This chapter evaluated the performance and functionality of the CleanLand Garbage Management System through various testing activities, including unit testing, integration testing, system testing, and user acceptance testing. The results confirmed that the system successfully performs its core functions such as resident registration, complaint management, feedback submission, route scheduling, truck management, QR code integration, and special pickup request handling. The system demonstrated reliable performance, secure data management, and user-friendly operation across different modules. Overall, the testing outcomes indicate that the proposed solution meets the identified requirements and provides an effective, efficient, and transparent platform for municipal waste management services.

CHAPTER 06

6. Future Work

6.1 Introduction

Although the CleanLand Garbage Management System successfully achieves its primary objectives of improving garbage collection management, complaint handling, communication, and operational transparency, there are several opportunities for future enhancements. As technology continues to evolve, additional features can be incorporated to further improve efficiency, user experience, and service quality. The following areas have been identified for future development.

• Real Time GPS Tracking Enhancement

The current system utilizes Google Maps to monitor garbage trucks. Future versions can incorporate real-time GPS tracking devices installed in collection vehicles to provide accurate live location updates. This will allow residents to monitor truck movements in real time and enable administrators to optimize collection operations more effectively.

• Mobile Application Development

Currently, the CleanLand system operates as a web-based platform. A dedicated mobile application for Android and iOS devices can be developed in the future to improve accessibility and provide users with a more convenient way to access schedules, submit complaints, scan QR codes, and receive notifications.

• Automated Notification System

Future enhancements can include automated SMS, email, and push notification services. Residents can receive reminders about upcoming garbage collection schedules, complaint status updates, special collection announcements, and emergency service notifications without having to log into the system.

• Advanced Analytics and Reporting

Future versions of the system can incorporate advanced reporting and data analytics features. Administrators will be able to analyze collection performance, complaint trends, resident feedback, and operational efficiency through interactive dashboards and graphical reports, supporting better decision-making.

• Smart Bin Monitoring Integration

The system can be extended by integrating IoT-enabled smart bins equipped with sensors to monitor waste levels. These sensors can automatically notify authorities when bins become full, helping optimize collection schedules and reduce overflowing waste bins.

• Security and Data Protection Improvements

Future developments can focus on strengthening system security through advanced encryption methods, enhanced authentication mechanisms, and secure cloud-based backups. These improvements will help protect sensitive user information and ensure the reliability of system data.

6.3 Summary

This chapter discussed several future enhancements that can further improve the CleanLand Garbage Management System. Proposed improvements include real-time GPS based truck tracking, mobile application development, automated notification services, advanced analytics and reporting, IoT based smart bin monitoring, and enhanced security mechanisms. Implementing these features will increase system efficiency, improve communication between stakeholders, and support more effective and sustainable municipal waste management operations.

CHAPTER 07

7. Conclusion

The successful development of the CleanLand Garbage Management System represents a significant step towards improving municipal waste management through digital technology. The system successfully integrates key features such as resident registration, complaint management, collection schedule management, QR code-based bin identification, truck monitoring, feedback management, and awareness content delivery within a centralized web-based platform.

The implementation of the system addresses several challenges identified during the stakeholder survey, including poor communication, missed garbage collections, inefficient complaint handling, and limited operational transparency. By providing residents, drivers, and administrators with an accessible and user-friendly platform, the system enhances service efficiency, accountability, and communication between stakeholders.

The testing and evaluation results confirmed that the system performs according to the specified requirements and successfully supports the core waste management operations. The integration of technologies such as React, Node.js, MySQL, QR codes, and Google Maps has enabled the development of a reliable and scalable solution.

Although the project has achieved its primary objectives, there are opportunities for future enhancements, including real-time GPS tracking, mobile application development, automated notifications, advanced reporting, and IoT-based smart bin monitoring. These improvements will further increase the effectiveness and sustainability of waste management operations.

Overall, the CleanLand Garbage Management System demonstrates how modern web technologies can be utilized to improve waste collection services, enhance citizen engagement, and support smarter municipal waste management practices. The project provides a strong foundation for future development and innovation in the field of digital waste management systems.

CHAPTER 8

8. Reflection

During this final year project, I had the opportunity to design and develop the CleanLand Garbage Management System, a web-based platform aimed at improving municipal waste management through digital technology. This project allowed me to apply the knowledge and skills I gained throughout my academic studies while gaining valuable practical experience in software development, database management, and system design.

I am sincerely grateful to the University of Moratuwa for providing the knowledge, guidance, and learning environment that made this project possible. Through this project, I enhanced my technical skills in web development using React, Node.js, Express.js, MySQL, and Google Maps integration. I also gained practical experience in system analysis, requirement gathering, database design, testing, and project documentation.

I would like to express my heartfelt gratitude to my supervisor for providing continuous guidance, valuable feedback, and encouragement throughout the project. I am also thankful to my lecturers, classmates, friends, and family members who supported and motivated me during the development of this system.

Throughout the project, I encountered several challenges, including database integration, route management, QR code implementation, and ensuring smooth communication between the frontend and backend components. Overcoming these challenges helped me improve my problem-solving abilities, technical knowledge, and confidence as a software developer.

Looking ahead, I hope to further enhance my skills in software engineering, web application development, and emerging technologies. This project has provided a strong foundation for my future career in the Information Technology industry and has strengthened my passion for developing technology based solutions that address real world problems.

9. Reference

[1] “myWaste.” Accessed: Jan. 03, 2026. [Online]. Available: https://www.mywaste.app/#

[2] “Solid Waste Management - Cleantech.” Accessed: Jan. 03, 2026. [Online]. Available: https://cleantech.lk/services/solid-waste-management/

[3] “Colombo Municipal Council.” Accessed: Jan. 03, 2026. [Online]. Available: https://www.colombo.mc.gov.lk/garbage-collection.php

[4] “Kurunegala Municipal Council.” Accessed: Jan. 03, 2026. [Online]. Available: https://www.mckswd.com/
[5] “Solid Waste Management Centre - Galigamuwa Pradeshiya Sabha.” Accessed: Jan. 03, 2026. [Online]. Available: https://galigamuwa.ps.gov.lk/01-draft-page/
[6] “galle.mc.gov.lk/node/122.” Accessed: Jan. 03, 2026. [Online]. Available: https://galle.mc.gov.lk/node/122